Abstract

From First Principles to Real-World Impact:

Advancing LLM Reasoning from Logic to Law

Simeng Han

2025

Large language models (LLMs) have demonstrated remarkable progress across natural language understanding and generation, with especially visible gains in areas such as mathematical and commonsense reasoning. Yet, the reliability of their reasoning processes remain open questions in logical and legal reasoning. Current evaluations often emphasize final answers, but these can obscure whether models have engaged in rigorous inference, or relied on fragile shortcuts. As a result, measuring not only what answer is produced but also how it is derived is crucial.

This dissertation advances the study of reasoning in LLMs through a progression from first principles to real-world applications. We begin by introducing a novel dataset, FOLIO, which captures natural language problems annotated with first-order logic, enabling precise evaluation of deductive validity with both natural language and symbolic reasoning. Having established FOLIO, we then focus on step-by-step reasoning evaluation, where the models are assessed in how they reach an answer, rather than solely evaluating the final answer. To do so, we introduce P-FOLIO, a process-based logical reasoning benchmark

involving human written proofs. P-FOLIO supports fine-grained assessment of inference steps and provides structured supervision to train models toward more rigorous reasoning.

Having examined the strengths and weaknesses of both natural language and symbolic reasoning, we identify adaptability as the next critical challenge: enabling models to integrate and transition between these modes as the problem demands. HYBRIDMIND offers an adaptive framework that chooses dynamically between natural language and symbolic approaches such as code or formal logic, improving performance on both logical and mathematical problems.

At the real-world frontier, LLMs are increasingly used in the legal domain for tasks like summarizing case law or offering basic legal guidance. However, their ability to generate full judicial analyses, such as complete reasoning sections in U.S. court opinions, remains largely unexplored. CourtReasoner introduces the first benchmark for full-length judicial-style reasoning in U.S. court opinions, evaluating whether models can construct coherent, precedent-grounded arguments under adversarial conditions. Together, this dissertation makes significant contributions toward advancing LLMs in logical and legal reasoning. By combining benchmarks that span formal logic and legal argumentation with methods that integrate symbolic and natural language reasoning, this dissertation provides both diagnostic tools and algorithmic strategies, moving from first principles to high-stakes, real-world applications.

From First Principles to Real-World Applications: Advancing LLM Reasoning from Logic to Law

A Dissertation

Presented to the Faculty of the Graduate School

Of

Yale University

In Candidacy for the Degree of

Doctor of Philosophy

By

Simeng Han

Dissertation Director: Arman Cohan

December, 2025

Copyright © 2025 by Simeng Han

All rights reserved.

Acknowledgments

This dissertation would not have been possible without the support, guidance, and encouragement of many mentors, colleagues, friends, and family members.

I have been incredibly fortunate to be advised by Arman Cohan, whose unwavering support carried me through some of the most difficult moments of my academic life. Arman has been more than just an advisor—he has been a mentor, collaborator, and role model. With his deep expertise in Natural Language Processing and Representation Learning, Arman has helped me refine my research ideas at their earliest stages, challenged me to ask deeper questions, and guided me in transforming rough drafts into polished, impactful papers. His feedback has consistently pushed me to craft stronger narratives, sharpen my arguments, and communicate my work more effectively, whether in writing or through presentations. Beyond research, Arman has taught me what it means to be a thoughtful scholar: balancing ambition with humility, rigor with openness, and persistence with patience. He is the most patient, kind, and calm advisor I know, always ready to listen to my doubts and uncertainties, no

matter how big or small, and to offer thoughtful guidance on both research and career decisions. His mentorship has not only shaped my dissertation but has also profoundly influenced the way I think about scholarship, collaboration, and mentorship itself. I could not have asked for a better advisor to accompany me on this journey.

I am deeply grateful to the late Dragomir Radev, who guided me during the first two years of my Ph.D. In 2020, while I was still an undergraduate working remotely from Singapore, Drago supported my Ph.D. application with thoughtful advice and even organized an additional information session to accommodate time-zone difference for students in Asia. His school recommendations also gave me good directions. At Yale, Drago offered unwavering mentorship. He trusted me to lead a large project, bringing together students and researchers from academia and industry. By guiding me at the very beginning of my doctoral journey, he set me on an excellent research path that opened doors into interdisciplinary topics and connected me with a community of remarkable mentors across diverse disciplines, including Ruzica Piskac, Scott Shapiro, Tom McCoy, Denny Zhou, Alex Lew, Jiayuan Mao and Pan Lu. Outside of work, Drago shared glimpses of his life like chatting about languages and hobbies. I will remember Drago as a remarkable academic and a warm, humorous person full of passion for life.

I owe special thanks to Ruzica Piskac, Scott Shapiro, and Yoshiki Takashima

for introducing me to the exciting field of Legal NLP and for so generously teaching me the foundational knowledge I needed to navigate this new domain. Each of them brought a unique perspective—Ruzica with her expertise in formal methods and programming languages, Scott with his deep insights into legal theory and jurisprudence, and Yoshiki with his dedication to bridging law and computational approaches. Together, they opened my eyes to how methods from computer science and legal scholarship can be brought into conversation with one another. Their mentorship has been instrumental in shaping one of the core directions of my research.

I am deeply grateful to Shafiq Rayhan Joty, my undergraduate advisor, who has been a constant mentor and friend since the very beginning of my research journey. From patiently teaching me the intricacies of text generation with RNNs to pulling all-nighters together before paper deadlines, Shafiq showed me both the rigor and the camaraderie of research life. His guidance during my undergraduate years helped me build the technical foundation I would rely on in graduate school. Shafiq also played a pivotal role in shaping the course of my academic career. It was through his introduction that I connected with the advisor who would ultimately guide my Ph.D., a gesture that changed the trajectory of my life. Even after I left for Yale, Shafiq remained a steadfast supporter, always willing to offer advice, encouragement, and perspective whenever I reached out.

ACKNOWLEDGMENTS

I am also grateful to my internship mentors at Meta FAIR, Google DeepMind and AWS: Richard Pang, Shiqi Wang, Nathaniel Schaerli, Nathan Scales, Gustavo Hernandez Abrego, Daniel Cer, Tu Vu, Pramuditha Perera, Ben Athiwaratkun, Xiaopeng Li, Zijian Wang, William Yang Wang, Dongzhuo Li and Yuandong Tian. While I ultimately decided to dedicate most of my time to academic research, I learned invaluable lessons from them and from the experience of conducting research in an industry setting. Their mentorship has contributed greatly to my growth as a researcher.

I would like to thank my high school Math Olympiad coach, Chao Wang, who first instilled in me the importance of first-principles thinking. Instead of teaching me only how to solve competition problems, he encouraged me to reflect on the very process of problem solving itself. This mindset fostered both my curiosity and my enduring passion for learning and discovery.

Last but not the least, I would like to thank my family and my partner. My mother put her own career growth on hold to ensure I had the best educational opportunities, all the while making it seem as though her sacrifices were effortless. She is the most optimistic and intelligent woman I know. My father taught me the importance of long-term thinking and steadiness—values that have sustained me through the many challenges of academic research. He often reminded me of the saying, “Even if an elk were to leap before him, his heart would remain undisturbed.” I am also deeply grateful to my partner. From

celebrating small victories to standing by me in difficult times, he has been my unwavering source of love and encouragement.

To all those mentioned here, and many others who have encouraged and inspired me along the way, I am profoundly grateful.

Contents

Acknowledgments **5**

List of Tables **16**

List of Figures **20**

1 Introduction **21**

2 Natural Language Reasoning with First-Order Logic **26**

2.1 Introduction 26

2.2 Related Work 30

2.2.1 Datasets for reasoning from text 30

2.2.2 Reasoning using large language models 31

2.3 FOLIO Corpus Construction 33

2.3.1 Example collection 34

2.3.2 Quality control for NL sentences 36

2.3.3 Quality control for FOL formulas 39

2.3.4 FOL modeling conventions 39
2.3.5 NL-FOL alignment review 41
2.3.6 FOL verification 41
2.3.7 Dataset statistics 42
2.4 Task Definition 45
2.4.1 Natural language reasoning with first-order logic 46
2.4.2 NL-FOL translation 46
2.5 Experiments 47
2.5.1 Experimental setup 48
2.5.2 Models 48
2.5.3 Main results 50
2.5.4 Case Studies 53
2.6 Error Analysis 55
2.6.1 Human performance 58
2.6.2 Further Model Performance Analysis 59
2.7 Conclusion 61
2.8 Limitations 61

3 Evaluating and Improving Logical Reasoning with Abundant Human-Written Reasoning Chains 63

3.1 Introduction 63
3.2 Related Work 67

3.3 P-FOLIO 69
3.3.1 Inference Rules 70
3.3.2 Dataset Annotation 71
3.3.3 Dataset statistics 74
3.4 Reasoning Evaluation 75
3.4.1 Single-step inference-rule classification 75
3.4.2 Single-step derivation reasoning 75
3.4.3 Proof generation 76
3.4.4 Proof generation evaluation 76
3.4.5 Task comparison 76
3.5 Experimental Results 77
3.5.1 Single-step evaluation 78
3.5.2 Proof generation 79
3.6 Analysis 85
3.7 Conclusion 88

4 Meta Selection of Natural Language and Symbolic Language for Enhanced LLM Reasoning 90

4.1 Introduction 90
4.1.1 Related Work 93
4.2 Method 95
4.2.1 Approaches to reasoning 95

4.2.2 Choice of symbolic language 97
4.2.3 Training data generation 98
4.3 Experiments 99
4.3.1 Datasets 100
4.3.2 Model choice 100
4.3.3 NL, SL, NLSymbol and SymbolNL 101
4.3.4 Experiments on FOLIO 102
4.3.5 Experiments on MATH 105
4.4 Analysis 108
4.4.1 Analysis on FOLIO 108
4.4.2 Analysis on MATH 110
4.4.3 Comparing FOLIO with MATH 112
4.5 Conclusion and Future Work 113

5 Evaluating LLM Agents' Capabilities of Reasoning like Judges 114

5.1 Introduction 114
5.2 Related Work 121
5.2.1 LLM reasoning benchmarks in the legal domain 121
5.2.2 Legal reasoning with LLMs 122
5.3 CourtReasoner construction 123
5.3.1 Overall input and output annotation 123
5.3.2 Adversarial complex legal reasoning task construction . . 126

5.3.3 Dataset statistics 130
5.3.4 Meta-evaluation 132
5.4 Experiments with LLM Evaluators 136
5.4.1 LLM as an evaluator 136
5.5 Conclusion 138
5.6 Limitations 139
5.7 Ethical considerations 141

6 Conclusion **143**

6.1 Summary of Findings 143
6.2 Overarching Lessons and Retropsection 147
6.3 Limitations and Future Directions 148
6.3.1 Disentangling Memorized Knowledge from Higher-Level Problem-Solving 148
6.3.2 Constructing a Reasoning Taxonomy for Improving Large Reasoning Models 151

A Appendix **184**

A.1 P-FOLIO Annotation Guidelines 184
A.2 P-FOLIO Hyper-parameter settings 184
A.3 HybridMind implementation details 185
A.4 Statistics of testing data in HybridMind 186

A.5 Significance test for HybridMind 188

A.6 Analysis between SL and NL with Case Studies for HybridMind . 188

A.6.0.1 SL 196

A.6.0.2 Second stage of SymbolNL 205

A.6.0.3 Second stage of NLSymbol 212

A.6.1 Meta-Selection prompt 220

A.7 Scoring Rubric for CourtReasoner Meta-Evaluation 237

A.8 Per-citation Relevance Rubric used in CourtReasoner 244

A.9 Example of model input and output for the CourtReasoner benchmark245

List of Tables

2.1 Comparison of FOLIO with other datasets related to logical reasoning. #Distinct AST stands for the number of distinct abstract syntax trees, representing the number of distinct sentence-level logic structures in the corpus. FOLIO is the first expert-written dataset for FOL reasoning equipped with parallel FOL formulas. The examples are mostly aligned with real-world knowledge and use highly natural wordings. It also has a greater variety than the previous datasets in terms of reasoning depths with a larger number of distinct logic patterns and a large vocabulary. 27
2.2 An example story in FOLIO based on the knowledge from the Wikipedia page on wild turkeys. The story consists of five premises and three conclusions with their corresponding FOL formulas and labels for the conclusions. All five premises are needed to infer the conclusions. The model needs to reason under logic patterns with universal quantification ($\forall$), negation ($\neg$), conjunction ($\wedge$), and disjunction ($\vee$). 27
2.3 Statistics based on different data collection methods of FOLIO. #*Words* is the average number of words per NL sentence. 42
51table.caption.10
2.5 NL-FOL translation results on FOLIO. SynV measures syntactic validity and ExcAcc measures the inference engine execution accuracy. 52
2.6 A WikiLogic story and model predictions. 53
2.7 A HybLogic story and model predictions. 54
2.8 Performance differences on the WikiLogic and HybLogic subset of FOLIO. WikiLogic has more diverse logical structures while HybLogic stories have higher reasoning depths. 56
2.9 Human evaluation on GPT-4 model outputs with incorrect truth value predictions 57

2.10 Comparison of the results across different input formats with few-shot prompting. NL, NL-FOL, FOL, NL + FOL stands for NL prompting, execution accuracy of NL-FOL translation, using only FOL in the prompt and using concatenated NL and FOL in the prompt respectively. 60
3.1 Comparison of P-FOLIO and other datasets of proofs 63
3.2 An example story in FOLIO consisting of five premises and one conclusion with the label for the conclusion. 64
3.3 Proof written for the example story in FOLIO. “P No” is premise number and “D” stands for derivation. 69
3.4 Categories of inference rules used in our annotation protocol. . . 69
3.5 Single-step inference rule classification and single-step derivation reasoning results. 78
3.6 Instruction used for prompting GPT-4 for zero-shot proof generation. 79
3.7 Result of many-shot prompting with GPT-4. 79
3.8 Comparison of final answer accuracy and proof accuracy (%). . . . 80
3.9 Fine-tuning results and out-of-domain generalization capabilities of LLMs trained on FOLIO and P-FOLIO. "w/o Proofs" indicates the model is trained with FOLIO and "w/Proofs" indicates the model is trained with P-FOLIO. 80
3.10 Pass@k results for GPT-3.5 and GPT-4 80
3.11 Final answer accuracy of GPT-4 with human-written derivations 81
3.12 Case Study 82
3.13 Example including prompt, GPT-4 model input and output for evaluating whether two model outputs following the same reasoning path. 84
4.1 Model performance on FOLIO *test* set. **Bold** and underlined numbers indicate the best and second-best performance in each category, respectively. MV: Majority Vote. Best-of-2 (NL/SL): At least one of NL/SL is correct. 102
4.2 Model performance on MATH *test* set. **Bold** and underlined numbers indicate the best and second-best performance in each category, respectively. MV: Majority Vote. Best-of-2 (NL/SL): At least one of NL/SL is correct. Best-of-4: At least of one of NL/SL/SymbolNL/NLSymbol is correct. 105
4.3 Analysis on FOLIO 108
4.4 FOLIO output example 110
5.1 Distribution of question lengths. 125
5.2 Distribution of model output lengths. 125

5.3 Performance of LLMs evaluated by human annotators, reported in mean. . . . 128
5.4 Spearman correlation (ρ) between two annotators across evaluation conditions. `o3-search` shows the strongest correlation, followed by `open-deep-research`. . . . 129
5.5 Performance of LLMs evaluated by human annotators, reported in percentage. Higher values are colored in darker shades of red. Overall, the removal of key facts results in a noticeable decline in performance. . . . 130
5.6 Average per-citation relevance score (%) for Gemini Deep Research (Gemini DR) and Open-Deep-Research (Open DR). . . . 131
5.7 Meta-evaluation on citation relevance (CT), constraint extraction (CS), and argument validity (AG) across various perturbation settings. **Spearman correlations** are reported, in percentage. Values are colored by the blue-to-red `coolwarm` colormap. . . . 137

S1 Annotation guidelines . . . 238
S2 Number of samples in the testing set by categories for MATH. . . 239
S3 The frequency and proportion of selected choices by HybridMind on MATH and FOLIO *test* set. We implemented 4-choice selector for MATH dataset and 2-choice selector for FOLIO. . . . 239
S4 Number of samples in the testing set by categories for FOLIO. . . 239
S5 Testing of significance levels of MATH dataset between FOLIO and NL. . . . 239
S6 Testing of significance levels of FOLIO dataset between FOLIO and NL. . . . 239
S7 An example where SL outperforms NL. While NL reasoning leads to logical errors in analysis and overlooks some possible scenarios, resulting in incorrect conclusions, writing Python code is more elegant and intuitive. . . . 240
S8 An example where SL outperforms NL. Python code offers a more accurate and structured approach. While NL reasoning overlooks some edge cases and leads to incorrect conclusions, SL systematically handles the details and produces the correct result. 241
S9 An example where natural language (NL) outperforms symbolic logic (SL) arises when solving problems through Python code. A key factor in such cases is that the language model must not only understand the problem but also possess a strong command of the syntax and semantics of the symbolic language. Without this, the model may generate code that is syntactically invalid or semantically incorrect, even if its underlying reasoning about the problem is sound. . . . 242

LIST OF TABLES

S10 An example where NL outperforms SL. SL doesn't consider the case where the numerator equals zero, which highlights the weakness of SL reasoning when facing certain edge cases or special conditions. 243

List of Figures

2.1 Distribution of reasoning depths 43
2.2 Dale-Chall Readability Distribution. 44
2.3 Accuracies of different models categorized into examples with different reasoning depths. 56
2.4 Confusion matrices for the results of fine-tuning RoBERTa-Large and few-shot prompting GPT-4. 59

3.1 Distribution of the number of correct & wrong answers vs the number of human-written proof steps 81
3.2 Distribution of the number of proof steps 85
3.3 Distribution of widely used inference rules 86

4.1 Illustration of HybridMind. HybridMind enables the model to analyze the problem and choose the most suitable approach among NL, SL, NLSymbol, and SymbolNL. We have different reasoning paths for logic and mathematical reasoning problems. In this example, HybridMind selects SL for a mathematical reasoning problem, which leads to the right solution (105). 95

5.1 Overview of the CourtReasoner pipeline for judicial reasoning evaluation of LLM agents. 123

Chapter 1

Introduction

Large language models (LLMs) have made striking progress on a wide range of NLP tasks, yet their reasoning remains under-specified and inconsistently measured.

This dissertation introduces a unified trajectory for advancing robust and interpretable reasoning in language models, beginning with foundational evaluation and extending to adaptive methods and real-world applications. FOLIO and P-FOLIO establish a foundational framework for evaluating and improving logical reasoning: FOLIO probes outcome-based deductive validity by pairing natural language problems with first-order logic, while P-FOLIO enriches this paradigm with human-written proofs that enable process-based assessment and supervision. Building on insights from these benchmarks, HYBRIDMIND demonstrates that no single reasoning modality is universally sufficient, and

proposes a meta-selection mechanism that dynamically chooses between natural-language and symbolic reasoning pathways, reflecting how effective human reasoning flexibly blends intuitive and formal strategies. Finally, CourtReasoner brings these advances into a high-stakes legal context by evaluating whether LLM agents can generate full-length judicial decisions grounded in precedent, doctrinal consistency, and adversarial reasoning. Together, these contributions form a coherent program for understanding how models reason, guiding them toward more reliable, rigorous, and context-aware inference.

FOLIO addresses foundational gaps in existing benchmarks for logical reasoning. Prior datasets are often synthetic, limited in reasoning depth and logical variety, or entangle logic with other forms of knowledge and commonsense. FOLIO responds by aligning expert-written, natural-language "stories" with parallel first-order logic (FOL) formulas so that conclusions can be assessed for formal validity, not just surface plausibility. The dataset emphasizes natural wording, real-world grounding, diverse abstract syntax structures, and varying reasoning depth; it also introduces an NL$\rightarrow$FOL translation task to make the underlying logic explicit. Benchmarking strong LMs under both fine-tuning and few-shot prompting reveals that fluent models still produce formally invalid conclusions when variable binding, quantifier scope, or multi-step deduction are required—clarifying that logical soundness must be measured directly, not inferred from outcome accuracy. Because correct labels alone cannot certify the

process that produced them, the next step is P-FOLIO.

P-FOLIO builds on this by shifting from outcome-based to process-based evaluation. If correctness cannot establish reasoning ability, we must examine the reasoning chains that lead to those labels. P-FOLIO contributes expert-written, step-by-step natural-language proofs for FOLIO's realistic logic stories, collected with an annotation protocol that encourages clear, forward-moving derivations without proof by contradiction or opaque case splits. These proofs span a wide range of inference rules and step counts and support evaluation at multiple granularities (e.g., single-step inference classification and end-to-end proof generation). Using these chains as supervision—via many-shot prompting or fine-tuning—improves models' logical fidelity and reveals where even strong LLMs still fail (e.g., skipping intermediate premises, conflating closely related rules, or losing track of bindings in longer chains). Yet knowing how to reason does not decide which representation to use on a given problem, which motivates HybridMIND.

HybridMIND investigates how to decide how to reason. Contemporary practice tends to favor either detailed natural-language reasoning (CoT-style) or executable symbolic reasoning (code/FOL with external tools). The two paradigms are complementary: NL reasoning captures semantic richness but suffers from ambiguity; symbolic representations enforce precision and make deduction and arithmetic checkable, but can miss linguistic nuance.

HybridMIND proposes a meta-selection framework that dynamically chooses NL, symbolic, or mixed solutions according to the problem's characteristics, and it trains this selector with supervised and iterative (STaR) fine-tuning. Empirically, the hybrid approach improves consistency on logic-heavy tasks and maintains flexibility for language-heavy ones; analytically, it helps explain when and why code-style solutions differ from FOL-style solutions and how those differences affect selection. To assess whether such structured, adaptive reasoning can bear the weight of real-world adjudication, we turn to CourtReasoner.

CourtReasoner tests whether structured, adaptive reasoning can sustain legal argumentation, where there is often no single correct answer and where evaluation must go beyond binary accuracy. Built from real U.S. appellate opinions, the benchmark asks models to produce full judicial-style reasoning—identifying issues, articulating doctrinal elements, bringing in precedent for each element, integrating partial holdings, distinguishing outliers, and closing with a logically connected conclusion. To reduce memorization and copying, it adds adversarial variants by removing or altering facts. Reasoning is evaluated by legal professionals along three axes central to practice: citation relevance (are authorities correctly cited and applied?), constraint extraction (are the conditions of the cited rules correctly identified?), and argument validity (do the conclusions follow from the law and facts?). The study also explores LLM-as-evaluator meta-assessment under long outputs, highlighting both promise and fragility in automated scoring

of legal reasoning. With these pieces in place, we can synthesize the trajectory from logic to law.

Chapter 2

Natural Language Reasoning with First-Order Logic

2.1 Introduction

Logical reasoning is a central component for intelligent systems and should be sufficiently and independently evaluated (Russell and Norvig, 2010). However, existing natural language tasks are inadequate in measuring the complex logical reasoning capability of a model (Saparov and He, 2023; Srivastava et al., 2023; Tian et al., 2021).

Several datasets related to logical reasoning have recently been proposed. However, existing benchmarks often exhibit limited complexity in reasoning or lack language naturalness. Some of these common benchmarks do not

Dataset	Size	Reasoning	Text Source	Real-World Resources	# Reasoning Depth	Vocab	# Distinct AST
CLUTTER (2019)	6k	Inductive	Synthetic	×	×	-	×
RECLOR (2020)	6k	Mixed forms	GMAT, LSAT exams	✓	×	-	×
LogiQA (2021)	8.6k	Mixed forms	NCSE exams	✓	×	-	×
RuleTaker (2020)	500k	Deductive	Synthetic	×	$0 \sim 5$	101	48
ProofWriter (2021)	500k	Deductive	Synthetic	×	$0 \sim 5$	101	48
LogicNLI (2021)	20k	FOL	Synthetic	×	$1 \sim 5$	1077	30
BigBench (2022)	1300	Mixed forms	Human-Written	Partially	×	-	-
ProntoQA (2023)	200	Deductive	Synthetic	✓	1, 3, 5	-	-
FOLIO (ours)	1,435	FOL	**Expert-written**	✓	**0 ∼ 7**	**4351**	**76**

Table 2.1: Comparison of FOLIO with other datasets related to logical reasoning. #Distinct AST stands for the number of distinct abstract syntax trees, representing the number of distinct sentence-level logic structures in the corpus. FOLIO is the first expert-written dataset for FOL reasoning equipped with parallel FOL formulas. The examples are mostly aligned with real-world knowledge and use highly natural wordings. It also has a greater variety than the previous datasets in terms of reasoning depths with a larger number of distinct logic patterns and a large vocabulary.

A FOLIO example based on the Wild Turkey Wikipedia page: `https://en.wikipedia.org/wiki/Wild_turkey`	
NL premises	**NL Conclusions -> Labels**
1. There are six types of wild turkeys: Eastern wild turkey, Osceola wild turkey, Gould's wild turkey, Merriam's wild turkey, Rio Grande wild turkey, and the Ocellated wild turkey. 2. Tom is not an Eastern wild turkey. 3. Tom is not an Osceola wild turkey. 4. Tom is also not a Gould's wild turkey. 5. Tom is neither a Merriam's wild turkey, nor a Rio Grande wild turkey. 6. Tom is a wild turkey.	A. Tom is an Ocellated wild turkey. -> True B. Tom is an Eastern wild turkey. -> False C. Joey is a wild turkey. -> Unknown
FOL Premises	**FOL conclusions -> Labels**
1. $\forall x$(WildTurkey(x) $\rightarrow$ (EasternWildTurkey(x) $\vee$ OsceolaWildTurkey(x) $\vee$ GouldsWildTurkey(x) $\vee$ MerriamsWildTurkey(x) $\vee$ RiograndeWildTurkey(x) $\vee$ OcellatedWildTurkey(x))) 2. $\neg$EasternWildTurkey(tom) 3. $\neg$OsceolaWildTurkey(tom)) 4. $\neg$GouldsWildTurkey(tom) 5. $\neg$MerriamsWildTurkey(tom) $\wedge$ $\neg$RiograndeWildTurkey(tom) 6. WildTurkey(tom)	A. OcellatedWildTurkey(tom) -> True B. EasternWildTurkey(tom) -> False C. WildTurkey($joey$) -> Unknown

Table 2.2: An example story in FOLIO based on the knowledge from the Wikipedia page on wild turkeys. The story consists of five premises and three conclusions with their corresponding FOL formulas and labels for the conclusions. All five premises are needed to infer the conclusions. The model needs to reason under logic patterns with universal quantification ($\forall$), negation ($\neg$), conjunction ($\wedge$), and disjunction ($\vee$).

specifically evaluate logical reasoning independently of other forms of reasoning (Liu et al., 2021; Yu et al., 2020). Those specifically designed for measuring logical reasoning are insufficient in terms of logical reasoning complexity and natural language variety. As shown in Table 2.1, examples in RuleTaker (Clark

et al., 2020) and LogicNLI (Tian et al., 2021) need at most five depths of reasoning. The entire corpus of RuleTaker or LogicNLI has fewer than 50 distinct abstract syntax trees. RuleTaker has only 101 words in its vocabulary and LogicNLI has 1077 words in the vocabulary. Moreover, none of them are written by humans with information drawn from real-world knowledge, making them less applicable to real-world reasoning scenarios. The logical deduction portion in BigBench (Srivastava et al., 2023) requires commonsense reasoning besides logical deduction. ProntoQA (Saparov and He, 2023) only contains logical reasoning questions that are answerable with repeated applications of the Modus Ponens inference rule.

In this chapter, we present a natural language reasoning dataset, *FOLIO*, with first-order logic reasoning problems which require the models to decide the correctness of conclusions given a *world* defined by the premises. In FOLIO, we aim to ensure high language naturalness and complexity, an abundant vocabulary, and factuality while also maintaining high reasoning complexity. FOLIO is a high-quality and manually curated dataset, written by CS undergraduate and graduate students and researchers in academia and industry. To ensure the conclusions of our examples follow the premises logically, we annotated all reasoning examples with first-order logic (FOL) formulas. An example of FOLIO is shown in Table 3.2. Based on our annotations, we propose a new NL-FOL translation task where an NL reasoning example is translated into

its FOL counterpart. Finally, we benchmark the performance of strong LMs in both fully supervised and few-shot settings to understand their capabilities in logical reasoning (*i.e.,* deriving the truth value of a logical conclusion from NL premises). Under the few-shot setting, the most capable publicly available LLM so far achieves only 53.1% on the stories written in a hybrid manner, which is slightly better than random.

To sum up, the contributions of this chapter are threefold. 1) We release a natural language reasoning dataset written by expert annotators, FOLIO, with first-order logical reasoning problems. 2) We use formal logic, i.e., FOL to ensure the logical validity of the examples written in NL and propose a new NL-FOL translation task. 3) We benchmark the performance of LMs by fine-tuning models and prompting LLMs with few-shot examples, on the FOLIO reasoning task. We hope that FOLIO, as a challenging logical reasoning dataset, will be used to facilitate measuring progress in the logical reasoning capabilities of language models.

2.2 Related Work

2.2.1 Datasets for reasoning from text

Developing models capable of reasoning over text has been a central goal in natural language processing (NLP) since the field's inception (Cooper et al., 1996). Over the years, substantial progress has been made toward this goal, accompanied by the introduction of diverse benchmarks targeting different facets of textual reasoning. These include tasks on natural language inference (NLI) (Bowman et al., 2015; Wang et al., 2019) and commonsense reasoning (He et al., 2021; Talmor et al., 2019). These also include multi-hop reasoning that requires integrating information across multiple pieces of text through semantic matching or retrieval (Chen et al., 2020; Yang et al., 2018). Among these reasoning abilities, logical reasoning has recently attracted an increasing amount of study. ReClor (Yu et al., 2020) and LogiQA (Liu et al., 2021) both collected multiple-choice questions from standardized graduate admission examinations, answering which requires various types of logical reasoning. However, these datasets cover mixed forms of reasoning and are not intended to test logical reasoning in isolation.

Meanwhile, testing logical reasoning in isolation without involving other forms of reasoning has also attracted researchers in recent years. CLUTRR (Sinha et al., 2019) covers inductive reasoning, which is beyond the scope of first-

order logic. To measure the deductive reasoning skills of pretrained LMs, several works have developed synthetic corpora targeting purely deductive reasoning (Clark et al., 2021; Saeed et al., 2021; Tian et al., 2021). These datasets do not contain highly natural sentences and often cover limited forms of logic while FOL is much more expressive. Kazemi et al. (2023) created a dataset for reasoning with contradictory information. Kawabata and Sugawara (2023) crowdsourced rationales for over 3000 examples based on ReClor (Yu et al., 2020). ProntoQA (Saparov and He, 2023) is comprised solely of logical reasoning queries that can be resolved through applying the Modus Ponens inference rule while FOLIO questions require applications of multiple types of inference rules. As shown in Table 2.1, FOLIO is the first large-scale first-order logic (FOL) reasoning dataset with formal logic annotations in FOL. FOLIO is logically diverse and complex with complex natural language sentences and a rich vocabulary.

2.2.2 Reasoning using large language models

Reasoning has been demonstrated as one of the emergent abilities of LLMs of sufficient scale recently (Chowdhery et al., 2022; Talmor et al., 2020; Wei et al., 2022a). One such emergent behavior, Chain-of-Thought prompting (Wei et al., 2022b), consists of a series of intermediate reasoning steps output by an LLM. This improves the performance on arithmetic, commonsense, and

symbolic reasoning benchmarks significantly. There has been a line of research continuing on from Chain-of-Thought (Kojima et al., 2022; Li et al., 2022; Yao et al., 2023) to elicit reasoning behavior from LLMs. Building on Chain-of-Thought prompting, many techniques used on top of LLMs to improve downstream performance have been formalized into control flows and programs. These are called language model cascades (Dohan et al., 2022), subsuming techniques such as Chain-of-Thought prompting, STaR (Zelikman et al., 2022b), and Selection-Inference (Creswell et al., 2022) for reasoning. Dasgupta et al. (2022) studied the reasoning ability of LLMs but only used a small set of 48 syllogisms with only two premises each. Saparov and He (2023) created a synthetic dataset that and showed that LLMs are capable of making correct individual deduction steps.

With FOLIO, we aim to set a high standard, ensuring that achieving high performance through superficial strategies and shallow heuristics is prevented, allowing a robust evaluation of the first-order logic reasoning capabilities of LLMs. We show that many LLMs fall short on complex first-order logic reasoning, and that significant room for improvement in this area remains.

2.3 FOLIO Corpus Construction

We collected FOLIO through a carefully designed manual annotation process to achieve high-quality examples that necessitate complex logical reasoning. Writing natural language reasoning stories with FOL requires sufficient knowledge in both semantic parsing and first-order logic, as well as strong analytical skills. Given the complexities of such annotations, we selected annotators based on a few important criteria to ensures that our dataset is annotated with the highest level of precision and expertise, reflecting the complexity and nuance required for first-order logical reasoning. 1). Our annotators are either college or graduate students who are native English speakers or possess near-native proficiency in English.4 2). They possess formal education in first-order logic, having either completed relevant coursework or undertaken self-directed studies in first-order logic or semantic parsing. In addition, the quality-check stages involve highly specialized annotators who also meet the above criteria. At the NL quality check stage, only annotators who are experts in natural language processing or computational linguistics are involved. For the FOL quality check, only annotators who are experts in first-order logic are involved. We also give the annotators several training sessions on how to write a story, by providing them with detailed annotation guidelines. All stories and FOL annotations in FOLIO are written and reviewed by expert annotators, including CS undergraduate and graduate students, and senior researchers, who met the aforementioned

criteria.

We develop our dataset in six stages: WikiLogic collection, HybLogic collection, NL quality control, FOL quality control, NL-FOL alignment and FOL verification, spending 980 man-hours in total.

2.3.1 Example collection

We collected our dataset using two different methods in order to obtain examples that are both logically diverse and complex and have abundant abstract syntax tree (AST) variations. The annotators are free to write stories based on any topic they want while writing the stories.

2.3.1.0.1 WIKILOGIC: ANNOTATION FROM SCRATCH USING WIKIPEDIA ARTICLES AS SEEDS.

At this annotation stage, the annotators are asked to select random Wikipedia pages by repeatedly using the Wikipedia Special Random link.[1] The Wikipedia articles are used to develop ideas for topics to write new stories. We ask the annotators to create new stories *from scratch without using templates* based on real-world knowledge, which should be plausible in general. Each of the stories is composed of several premises and conclusions with truth values of True, False, or Unknown (see Table 3.2 for an example). We also ask the annotators to write

[1] https://en.wikipedia.org/wiki/Special:Random

parallel FOL sentences for both the premises and conclusions. This results in a wide range of topics, abundant AST variations, and a wide vocabulary for FOLIO. Table 2.1 shows a comparison of FOLIO with other reasoning datasets that purely evaluate first-order logic or deductive reasoning.

2.3.1.0.2 HybLogic: hybrid annotation

The task of generating logically sound stories from scratch for a set of facts is very time-consuming for human writers, where the main challenge is to create complex and varied logical patterns to arrive at a conclusion. To address the problems of solely using manual annotation, we also consider a hybrid approach to facilitate the process. Our hybrid method is based on a common form of logical stories: *syllogisms*. A syllogism consists of two premises and a single conclusion, and the conclusion states some facts about the entities and categories in the premises.

In this approach, we first generate logically valid stories, which are templates containing abstract categories and entities, by combining multiple syllogisms into a single story template: the conclusion of one syllogism is used as a premise for the next syllogism. There are 256 logically distinct types of syllogisms and 24 of them are valid (Lehman, 1973). We use various combinations of 24 valid syllogisms. We also add in conjunction, disjunction, and implication. We then ask human annotators to assign nouns, phrases, or clauses to the abstract

entities or categories that reflect real-life scenarios to each template and write logically-valid stories in natural language. The usage of the template is to ensure that we have a set of varied and complex logical stories with multiple conclusions. An example of the resulting templates is as follows:

```
Premises:
All M are P.   All S are M.
Either S or A.  All A are B.
All D are B.  No C are B.
a is either a C or a P.

Conclusions:
[Unknown] a is an S.
[True] If a is either a C or a D,
then a is not either an A or a B.
```

There are many ways of expressing the same logic template in natural language, and so the generated templates augment, rather than limit, the creativity of humans.

2.3.2 Quality control for NL sentences

To ensure the highest quality of the dataset, we dedicated considerable attention to the following key aspects of the natural language sentences during the quality control process.

2.3.2.0.1 FACTUALITY AND BIAS

Our dataset prioritizes realism and factual accuracy, steering clear of biases and stereotypes linked to identity markers like race, ethnicity, gender, sexuality, nationality, class, and religion. Toward these objectives, we manually screened all stories and found that 39.2% of the stories suffer from at least one of these issues. We implemented a detailed protocol to rewrite these stories. Specifically, we rewrote those that are not reflective of well-established scientific, historical, or legal facts. We took out stories that had strongly opinionated language and contained gender, racial, and classist biases. We accept certain classes of "psychologically fundamental generalizations" (Leslie, 2008), however, such as "Covid is transmitted through the air" or "Tigers eat other animals," that may not be factually invariant but add logical and semantic nuances to the stories. For stories that pertain to generalization, such as "All As are Bs," we have added specifiers like "all Dan knows" to give a degree of reasonable factuality. For example, "All science fiction that Dan knows comes from an imaginative process" has a more reasonable degree of factuality than "All science fiction comes from an imaginative process."

2.3.2.0.2 LANGUAGE QUALITY

Apart from grammar, we make sure the sentences in our dataset are highly natural. All the sentences are first checked with a grammar checking tool,

Grammarly. Our annotators who have graduated from or are senior students studying English Literature conducted a thorough round of review for grammatical correctness and language naturalness. We also eliminate natural language ambiguity when it is possible. We include rules below on eliminating ambiguity.

• We always use “either-or” to express exclusive disjunction. We use either “A or B” or “A or B, or both” to express inclusive disjunction. In English “or” itself can be interpreted as either inclusive disjunction or exclusive disjunction. Adding “or both” cancels the exclusive disjunction distinctly. However, it is less common in the wild than just using “or”. we could add “or both” if it is important to emphasize the inclusive part semantically or contextually or for factuality; and do not add “or both” if it is not. We rely on the language model to figure out if it should be inclusive or exclusive, therefore not sacrificing naturalness.

• It is more natural to say "Some A is B" rather than "there exists an A such that A is B." "All A are B" can be more natural than "If A then B".

• Writing NL sentences that express negation over exclusive-or ("either both or neither") can be cumbersome but we found one natural ways of expressing these situations: "Each morning, John either works out and stretches, or he does neither".

Employing these rules effectively reduces the ambiguity of natural language in this reasoning dataset, but incurs the tradeoff of limiting variations in some

usage of language. However, we note that there is still sufficient variation in terms of sentence structures and logical structures as shown in Table 2.1. Other common issues in NL quality include singular/plural issues, especially in statements that deal with both categories and individual members of those categories; as well as ambiguities resulting from improper introduction of, or failure to introduce, proper nouns.

2.3.3 Quality control for FOL formulas

We adopt the FOL definitions and syntax most widely used in the AI community (Russell and Norvig, 2010). We include the following operators: negation $\neg$, conjunction $\wedge$, disjunction $\vee$, implication $\rightarrow$, universal quantifier $\forall$, existential quantifier $\exists$, equal $=$. Following Russell and Norvig (2010), we consider temporal logic and modal logic as special-purpose logics. Consequently, they are beyond the scope of the definition of first-order logic used in our dataset.

2.3.4 FOL modeling conventions

We use n-place predicates when applicable for the expressivity of the FOL formulas. However, we do not use the Davidsonian (Davidson, 2001) or neo-Davidsonian semantics (Parsons, 1990) because translating the majority of the FOL formulas in our dataset only requires one-place and two-place predicates.

Therefore the Davidsonian or neo-Davidsonian semantics are not necessary for the expressivity of the FOL formulas. For example, "Enjoy dressing up in old-fashioned clothing" is rendered as "Enjoy(x, dressingUp, oldFashionedClothing)".

In preliminary investigations, we found that the human-written FOL formulas suffer from FOL consistency issues, which necessitates an additional round of quality control for FOL formulas.

2.3.4.0.1 FOL CONSISTENCY

One NL sentence can be translated into FOL through multiple non-equivalent ways. For example, sometimes additional information inferred from a sentence can be represented in FOL, leading to multiple representations. We therefore design an annotation protocol for FOL translation in order to ensure that our FOL translations are as consistent as possible across all examples in our dataset. We highlight a few important strategies used in the annotation protocol. a). First-order logic formulas need to preserve as much as possible the semantics of natural language sentences. b). First-order logic formulas should stay as faithful to the structure of the original NL sentence as possible. c). Semantic decomposition is not needed unless necessary for maintaining the NL expressivity. This means that "John is a bachelor" can be translated into FOL simply as "`Bachelor(John)`". d). In terms of abstraction, we neglect tense and remove all the plural forms of verbs.

2.3.5 NL-FOL alignment review

Apart from checking whether NL and FOL express equivalent meanings, we also add necessary commonsense knowledge in both the NL and FOL premises. Sometimes humans do not write certain commonsense knowledge in the premises that is required in the FOL reasoning process, which is based solely on the premises given. We add such knowledge as additional premises at this stage. In particular, intrinsic properties of some predicates are required in the FOL reasoning process. For example, "`LocatedIn(x,y)`" should be transitive and "`BeFamily(x,y)`" should be symmetric.

2.3.6 FOL verification

Recognizing that the FOL formula annotations can be error-prone, we verify the syntactic validity and label consistency of FOL formula annotations with an FOL inference engine.

Although there are many provers widely used in the community McCune (2005–2010); Nipkow et al. (2002); Sutcliffe (2017) , we adopt the inference engine provided in the Stanford CS221 course page[2], which is a compact module designed specifically for procesing first-order logic statements. The inference engine does not support input in the FOL syntax adopted by standard education

[2] `https://stanford-cs221.github.io/spring2022/assignments/logic/index.html`

Source	#Stories	#Premises	#Conclusions	NL			Logic	
				Vocab	#Words	Complexity	#Depth	AST
WikiLogic	304	1353	753	3250	8.50	0 - 14 grade	1 - 5	51
HybLogic	183	1054	682	1902	11.52	0 - 14 grade	5 - 8	25
Total	487	2407	1435	4351	9.86	0 - 14 grade	5-8	76

Table 2.3: Statistics based on different data collection methods of FOLIO. #*Words* is the average number of words per NL sentence.

material Russell and Norvig (2010), which is used in our dataset. We therefore developed a FOL parser in order to convert the FOL formulas written by humans to the input format of the inference engine. The converter is a semantic parser tool written in Python. Although LLMs such as GPT-4 can be utilized to conduct the conversion, it is hard to ensure the GPT-4 outputs are always correct.

Proving a story requires three steps. First, the FOL statements of the premises and conclusions of a story annotated by humans are converted to Python code. Then, the code snippets are used as input to the theorem prover. Finally, the theorem prover outputs whether the conclusions are True / False / Unknown, based on the premises.

2.3.7 Dataset statistics

We show basic statistics of FOLIO and demonstrate the abundant vocabulary and logical complexity of FOLIO: Tables 2.1, 2.3 and Figure 2.1.

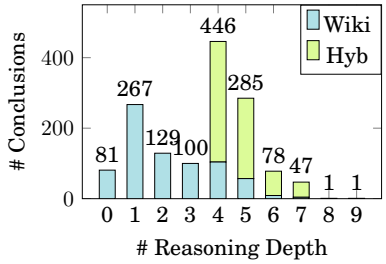


Figure 2.1: Distribution of reasoning depths

2.3.7.0.1 BASIC STATISTICS

Table 2.3 shows that examples based on Wikipedia make up the largest portion of FOLIO, with 304 stories, 1,353 NL and FOL premise pairs, and 753 NL and FOL conclusion pairs. Hybrid annotations consist of 183 stories with 1,054 NL and FOL premise pairs, and 682 NL and FOL conclusion pairs in total.

2.3.7.0.2 NATURAL LANGUAGE COMPLEXITY

We use the Dale–Chall Readability Formula (Dale and Chall, 1948, 1995) to measure the text complexity of FOLIO, following prior work in evaluating linguistic difficulty in reasoning datasets (Arps et al., 2022; Singh et al., 2023; Wei et al., 2021). Figure 2.2 shows the distribution of readability scores across the dataset. The examples span a broad range of grade levels, from early elementary school to college-level text (with 14th grade corresponding to college). While most instances fall between grades 4–8—indicating accessibility to middle-school readers—the distribution also includes a substantial tail of higher-complexity passages. This range reflects the linguistic diversity and

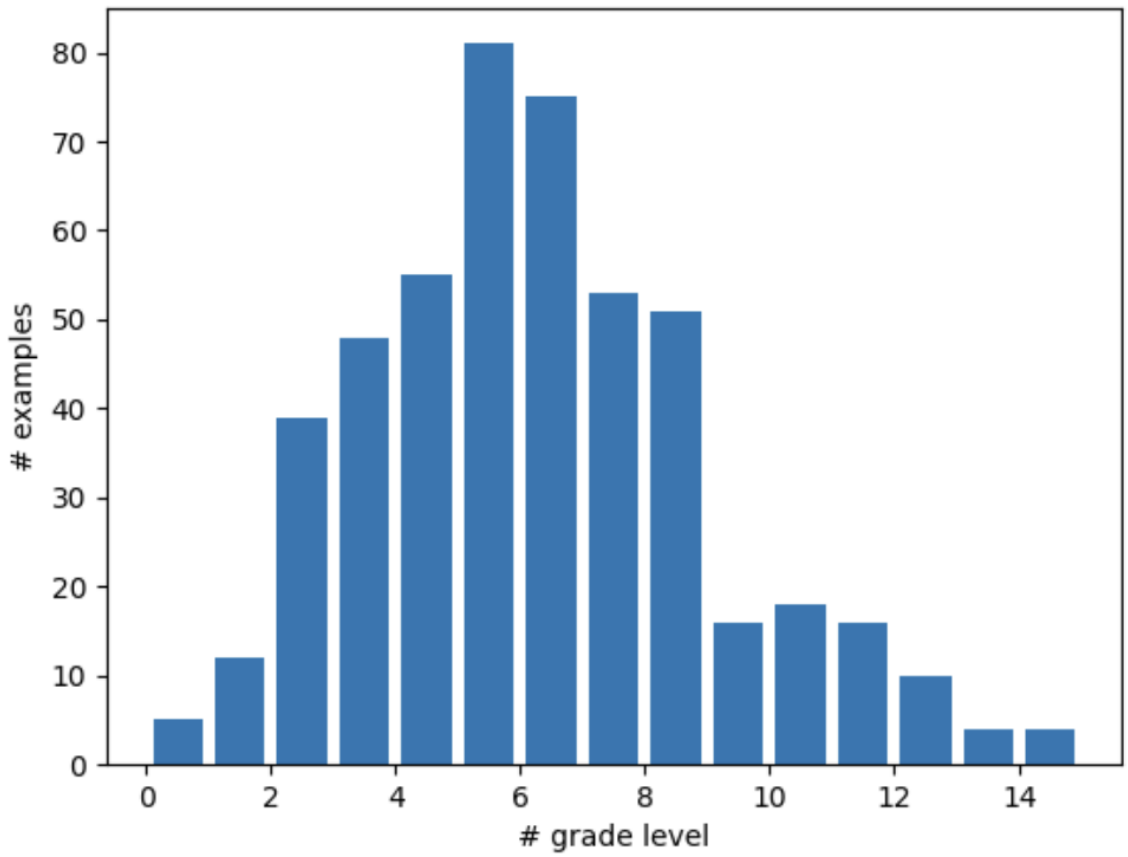


Figure 2.2: Dale-Chall Readability Distribution.

varying cognitive demands of FOLIO, requiring models to handle both simple and highly sophisticated natural-language reasoning tasks.

2.3.7.0.3 Logical complexity and diversity statistics

As shown in Figure 2.1, the mode of reasoning depths is four in FOLIO. 28.7% of the examples need five or more depths of reasoning to infer the conclusions, while the previous datasets needed at most five reasoning depths as shown in Table 2.1. This illustrates the logical complexity of FOLIO. Table 2.1 shows

that FOLIO also has a much larger number of distinct ASTs than the previous datasets, indicating that FOLIO is much more logically diverse. Figure 2.1 demonstrates the distribution of the number of examples in the WikiLogic and HybLogic sets versus the number of premises needed to arrive at a conclusion, showing that most of the conclusions from WikiLogic require one to five premises while those from HybLogic require five to eight premises.

2.3.7.0.4 VOCABULARY AND TOPICS

Table 2.3 shows that our dataset has a vocabulary of 4,351 words, and the examples based on Wikipedia account for 74% of the total vocabulary even though the WikiLogic stories take up only 63% of the total number of stories. The vocabulary of FOLIO is also significantly larger than the previous synthetically constructed datasets for logical reasoning.

2.4 Task Definition

We define two new tasks based on FOLIO, natural language reasoning with first-order logic and NL-FOL translation.

2.4.1 Natural language reasoning with first-order logic

Each natural language (NL) story S in FOLIO consists of n premises: $P = \{p_1, p_2, ..., p_n\}$ and m conclusions: $H = \{h_1, h_2, ..., h_m\}$. All NL stories are annotated with parallel FOL stories SF, which are sets of FOL formulas consisting of n premises $PF = \{pf_1, pf_2, ..., pf_n\}$ and m conclusions $HF = \{hf_1, hf_2, ..., hf_m\}$. pf_i and hf_i are logically and semantically similar to p_i and h_i, respectively. Given P and H, the goal is to determine the truth values of the conclusions: "True", "False" or "Unknown", based on FOL reasoning.

2.4.2 NL-FOL translation

We propose a new natural language to first-order logic translation (NL-FOL translation) task alongside our reasoning dataset. The goal of this task is to translate an NL story S to an FOL story FS. In particular, each of the NL sentence p_i or h_i and the parallel FOL formula pf_i or hf_i should be logically and semantically equivalent. Moreover, the truth values for the conclusions should be the same based on the NL story S and the parallel FOL story FS. In our dataset, the premises and conclusions are set up in such a way to ensure that the inference engine always returns an answer given enough resources such as time and memory. Unlike previous work Singh et al. (2020) which

translates problems with a single premise and a single hypothesis, our task is on translating examples of various lengths with a focus on stories with multiple premises. Thus, it also requires the models to consider *discourse-level* consistencies as opposed to translation at the sentence level.

2.4.2.0.1 NL-FOL EVALUATION METRICS

Two metrics are adopted to evaluate NL-FOL translation to capture different aspects of the generation results: 1). **Syntactic validity (SynV)**. The Syntactic Validity score measures whether the FOL formulas are syntactically valid. The score will be 1 if all FOL formulas of an example can pass the syntactic check and 0 otherwise 2). **Inference Engine execution accuracy (ExcAcc)**. The group of translated FOL for premises and conclusions in one story is fed into our inference engine to output the truth value for each conclusion. We define the accuracy of the output labels as the execution accuracy. We leave for future work the design of a more reliable metric of NL-FOL translation.

2.5 Experiments

In this section, we describe our experiments and main results.

2.5.1 Experimental setup

2.5.1.0.1 Tasks

We conduct experiments on the two tasks in §2.4: *NL reasoning with first-order logic (logical reasoning)* and *NL-FOL translation (NL-FOL).*

2.5.1.0.2 Dataset split

We split FOLIO by 70%/15%/15% split for the train/validation/test sets with 1,001/203/226 examples respectively. We split by story so that models are evaluated on unseen stories.

2.5.1.0.3 Evaluation metrics

We use accuracy for evaluating logical reasoning performance. For NL-FOL translation, we use the metrics in Section 2.4.2.0.1.

2.5.2 Models

We test the logical reasoning capabilities of LMs using fully supervised fine-tuning and few-shot prompting. We also test NL-FOL translation with few-shot prompting.

2.5.2.0.1 FULLY SUPERVISED FINE-TUNING

As fine-tuning baselines, we experiment with BERT (Devlin et al., 2019), and RoBERTa (Liu et al., 2020). We fine-tune the base and large versions of both BERT and RoBERTa, with an additional two-layer classification layer to predict the truth values. For the second task, i.e., NL-FOL translation, we only report few-shot prompting methods.

2.5.2.0.2 FEW-SHOT PROMPTING

We conduct zero-shot and few-shot prompting experiments on larger LMs with few-shot capabilities. For open-source models, we test LLaMA-13B and LLaMA-70B (Touvron et al., 2023), GPT-NeoX-20B (Black et al., 2022); for proprietary models we test GPT-3 (Brown et al., 2020), GPT-3.5-Turbo and GPT-4 (OpenAI et al., 2023) using prompts with 8 examples.[3]

2.5.2.0.3 PROMPTING STRATEGIES

We experiment with incorporating recent prompting strategies into GPT-4 as they have shown improvements in the general reasoning performance of LLMs. The prompting strategies include chain-of-thought (CoT) prompting (Wei et al., 2022b), chain-of-thought prompting with self-consistency (Wang et al., 2023c) and tree-of-thought prompting (Yao et al., 2023).

[3]In experimenting with different prompts, we found 8 shot examples to perform slightly better. It is also the maximum number of examples that fits in the text-davinci-002 context.

2.5.2.0.4 Logical reasoning methods

We also test recent methods specifically designed for logical reasoning: Logic-LM (2023), LINC (Olausson et al., 2023) and DetermLR(Sun et al., 2023), using GPT-4 as the base model. For the second task (NL-FOL translation), we use the same examples as in the Few-Shot NL experiments except that all the conclusions are included in each example.

We run experiments on five randomly sampled sets of examples from the training set and report the average accuracy.

2.5.3 Main results

2.5.3.0.1 Logical reasoning

The majority baseline of our dataset is 38.5% since in our test set, there are 87, 78 and 61 examples with labels of true, false and unknown respectively. As shown in Table 2.4, BERT-base and RoBERTa-base have similar performance on FOLIO with 56.83% accuracy. BERT-large has a 2.2% improvement over BERT-base. RoBERTa-large improves 3.1% over BERT-large. Flan-T5-Large achieves the highest performance in the fine-tuning setting and the accuracy is 65.7%.

We show that zero-shot prompting GPT-3.5 achieves better results than few-shot prompting text-davinci-002. Under few-shot NL prompting setting,

Model	**Size**	**Acc (%)**
majority baseline	-	38.5%
random probability	-	33.3 %
Fully supervised fine-tune		
BERT-base	110M	56.8
BERT-large	340M	59.0
RoBERTa-base	110M	56.8
RoBERTa-large	340M	62.1
Flan-T5-Large	783M	**65.9**
0-shot NL Prompt		
GPT-3.5-Turbo	-	53.1
GPT-4	-	61.3
8-shot NL Prompt		
LLama-13B	13B	33.6
LLama-70B	70B	44.0
LLama-70B - CoT	70B	47.8
LLama-70B - ToT	70B	48.4
text-davinci-002	-	49.5
GPT-3.5-Turbo	-	58.3
GPT-4	-	64.2
GPT-4 - CoT (2022b)	-	68.9
GPT-4 - CoT with SC (2023c)	-	69.5
GPT-4 ToT (2023)	-	70.0
LR-specific Methods		
Logic-LM (2023)	-	78.1
LINC (2023)	-	73.1
DetermLR (2023)	-	77.5

Table 2.4: Logical reasoning results of fully supervised fine-tuning and few-shot prompting on FOLIO test set. The model sizes of text-davinci-002, GPT-3.5-Turbo and GPT-4 are hidden from public[4]. CoT stands for chain-of-thought prompting (Wei et al., 2022b). SC stands for self-consistency (Wang et al., 2023c). ToT stands for tree-of-thought prompting (Yao et al., 2023).

LLama-13B achieves 33.63%, which is only slightly better than chance (33%).

LLama-70B achieves 43.97%, around 10% better than LLaMA-13B and obtains

Model	Zero-Shot		Few-Shot	
	Synv	ExcAcc	Sync	ExcAcc
GPT-3.5-Turbo	68.4	50.4	93.3	56.0
GPT-4	86.1	51.7	93.9	63.8

Table 2.5: NL-FOL translation results on FOLIO. SynV measures syntactic validity and ExcAcc measures the inference engine execution accuracy.

improvements of around 4% with Chain-of-thought prompting and Tree of Thought prompting. Text-davinci-002 achieves 49.53% and GPT-3.5 achieves 58.34%. GPT-4 achieves the best results among GPT series models.

Incorporating recent prompting strategies increases the performance of vanilla few-shot prompting. Chain-of-thought prompting achieves more than a 4% increase over GPT-4. Self-consistency (SC) improves chain-of-thought prompting by 0.6% percent. Tree-of-thought prompting achieves slightly better result than self-consistency with chain-of-thought prompting. For the results of recent methods developed for logical reasoning, LINC (Olausson et al., 2023) achieves around a 9% increase over few-shot prompting GPT-4. Both Logic-LM (GPT-4)(2023) and DetermLR (2023) achieves more than a 13% increase over few-shot prompting GPT-4, showing the superiority of the methods on logical reasoning.

[4]Hereafter, "GPT-3.5" refers to GPT-3.5-Turbo.

NL Premises	**NL Conclusions**
1. A moth is not a butterfly.	A. Cerura vinula emerges
2. Butterflies have thin antennae.	from cocoons.
3. Moths emerge from cocoons.	B. Cerura vinula does not
4. Some moths are pests.	have thin antennae.
5. Cerura vinula is a moth.	C. Cerura vinula is a pest.

Labels	**GPT-4**	**Fine-tune**
A. True	True	Unknown
B. Unknown	True	True
C. Unknown	Unknown	True

Table 2.6: A WikiLogic story and model predictions.

2.5.3.0.2 NL-FOL TRANSLATION

Table 2.5 shows the results of NL-FOL translation. The syntactic validity scores are around 93% with both GPT-3.5-Turbo and GPT-4. This indicates that language models with sufficient scales are good at picking up the patterns for FOL formulas and generating syntactically valid FOL formulas. However, GPT-3.5-Turbo and GPT-4 are not yet good at translating an NL story to a logically or semantically similar FOL counterpart, as indicated by the low inference engine execution accuracy score.

2.5.4 Case Studies

Table 2.6 shows a story from WikiLogic along with the GPT-4 and RoBERTa-Large predictions. Conclusion A is True given premises 5 and 3. From the premises, it cannot be determined if Cerura vinula has thin antennae or if it is a pest. Thus conclusions B and C are Unknown. GPT-4 predictions are correct for conclusions A and C while RoBERTa predictions are wrong for all conclusions.

NL Premises		
1. Some employees good at time management do not exercise every week.		
2. All employees good at time management are efficient in dealing with daily work.		
3. All employees efficient in dealing with daily work perform better than others.		
4. All employees who perform better than others have more opportunities to get a promotion.		
5. James does not have more opportunities to get a promotion.		
NL Conclusions		
A. James exercises every week.		
B. James exercises every week and is good at time management.		
C. If James does not perform better than others, then he exercises every week and is good at time management.		
Labels	**GPT-4**	**Fine-tune**
A. Unknown	Unknown	Unknown
B. False	Unknown	False
C. False	True	True

Table 2.7: A HybLogic story and model predictions.

Table 2.7 shows a story from HybLogic with a more complex FOL reasoning process. Inferred from premises 4 and 5, James does not perform better than others. With premises 3, 2 and 1, we know that James is not good at time management. Therefore, conclusion B is False. It cannot be determined if James exercises every week, thus the first conclusion is Unknown. The truth value of $p \rightarrow q$ is the same as $\neg p \vee q$. It is not true that James does not perform better than others. It is also false that James exercises every week and is good at time management. Thus conclusion C is False. For this example, GPT-4 predicted the correct truth value only for conclusion A and RoBERTa made correct predictions for conclusions A and B.

2.6 Error Analysis

Below we provide analysis of our results and key findings, providing additional insights into our dataset FOLIO and the current capabilities of LLMs in logical reasoning.

Models have higher accuracy on examples with fewer reasoning depths than on those with higher number of reasoing depths We show the accuracy categorized by reasoning depths in Figure 2.3. With few-shot prompting, GPT-3.5 and GPT-4 both perform much better on examples with a $0 \sim 3$ reasoning depth, indicating that examples with a $4 \sim 7$ reasoning depth pose a challenge to the SoTA LMs. With fine-tuning, RoBERTa has slightly higher performance on test examples with $0 \sim 3$ reasoning depth than on those with $4 \sim 7$ reasoning depth, but the difference is much smaller. This indicates that fine-tuning on longer and more difficult reasoning chains in the training set can improve model performance on equally-long test example chains. We note that the presence and prevalence of these difficult examples are unique to FOLIO. FOLIO's unique complexity reveals that current LMs are limited in their ability to extrapolate to longer and more complex reasoning chains, and suggests an avenue for further study.

Models have higher accuracy on WikiLogic than on HybLogic As shown in Table 2.8, in logical reasoning, GPT-3.5 and GPT-4 achieve substantially lower results on HybLogic than on WikiLogic and the result is slightly higher

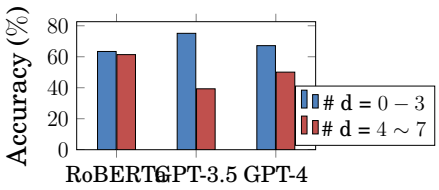


Figure 2.3: Accuracies of different models categorized into examples with different reasoning depths.

Method	**Model**	Wiki	Hyb
Fine-tuning	RoBERTa-large	60.71	63.48
NL Prompting	GPT-3.5-Turbo	68.88	47.70
	GPT-4	75.43	53.10
NL-FOL ExcAcc	GPT-3.5-Turbo	45.17	61.82
	GPT-4	59.12	67.93

Table 2.8: Performance differences on the WikiLogic and HybLogic subset of FOLIO. WikiLogic has more diverse logical structures while HybLogic stories have higher reasoning depths.

than chance. We hypothesize that this is because HybLogic has high logical complexity that the SoTA LLMs like GPT-4 cannot solve yet while WikiLogic examples require shorter reasoning chains which the model is already capable of solving. Moreover, since the examples in WikiLogic are created from scratch by humans, it is possible that LLMs have seen similar texts with similar logical patterns in the training data. However, fine-tuning RoBERTa-large achieves higher performance on HybLogic than on WikiLogic. This is likely because HybLogic is created from templates and some of the logical patterns can be learned during fine-tuning.

In NL-FOL translation, performs 10 points better on HybLogic than WikiLogic. This could be because WikiLogic has more distinct and diverse sentence-level

logical and language patterns and FOL annotations. WikiLogic has 53 ASTs while HybLogic has 33. Despite being more logically complex on a story level, FOL translations for HybLogic stories have simpler logical structures on a statement level. We include case study for one WikiLogic example and one HybLogic example in Appendix 2.5.4 and further analysis on model performance in Appendix 2.6.2.

Faulty path	65%
Wrong derivation	25%
Wrong syntactic comprehension	5%
Spurious shortcut	5%

Table 2.9: Human evaluation on GPT-4 model outputs with incorrect truth value predictions

Human evaluation on model outputs We conduct human evaluation on the GPT-4 model outputs with wrong truth value predictions. As shown in Table 2.9, approximately 65% of the time, the model struggles to construct accurate reasoning chains for complex problems with intricate steps, leading to faulty reasoning paths and indicating a limited ability to solve problems with long reasoning chains. In 25% of cases, erroneous derivations occur within certain reasoning steps, highlighting potential inaccuracies and flaws in logical deductions. 5% of conclusions in FOLIO have a complex syntactic structure, posing comprehension challenges for GPT-4. 5% of outputs show that GPT-4 leverage commonsense reasoning to employ spurious shortcuts that lead to the wrong truth value for the conclusion. We provide a case study for the "Faulty

path" scenario in Table 3.12. In this instance, the model can perform simple derivations from the premises, like "If someone is of age, they can vote and get married." However, because of the problem's complexity, the model struggles to identify the essential intermediate steps and cannot ascertain the truth value of conclusions, such as "Jack is not a child."

2.6.1 Human performance

Expert annotators demonstrate high reliability in evaluating logical reasoning, achieving an accuracy of 95.98%, which establishes a strong upper bound for human performance on FOLIO. In contrast, non-expert annotators achieve 61.82%, showing a substantial performance gap of 34.16% between the two groups. This large disparity indicates that evaluating logical reasoning requires substantial domain knowledge of FOL, beyond general language understanding. Furthermore, the 31.82% gap between expert annotators and GPT-4 highlights that even the best current models still fall significantly short of human expert performance, leaving ample room for improvement.

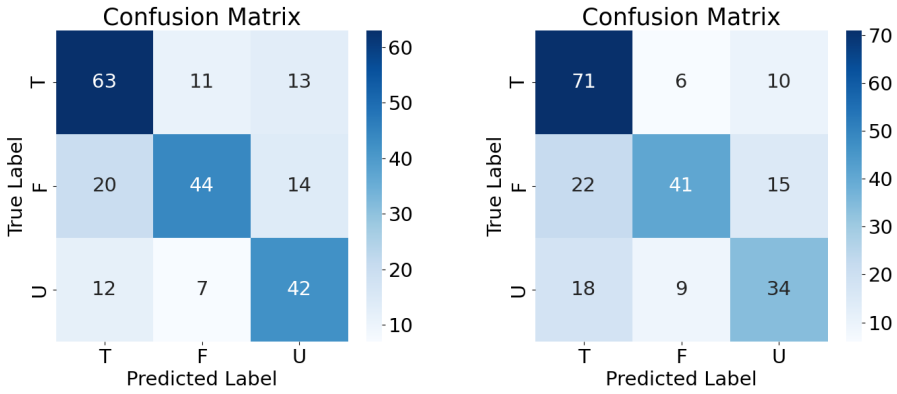


Figure 2.4: Confusion matrices for the results of fine-tuning RoBERTa-Large and few-shot prompting GPT-4.

2.6.2 Further Model Performance Analysis

2.6.2.0.1 Models have more tendency to predict "True" compared with "False" or "Unknown" labels

Confusion matrices in Figure 2.4 for the fine-tuning and 8-shot NL prompt results both show that LLMs are significantly better at making the correct predictions for conclusions with labels of True than the conclusions with labels of False or Unknown. The accuracy on examples with False or Unknown conclusions is 61.9% with fine-tuning and 54.0% with few-shot prompting. They also tend to make more predictions of True than the other labels.

Model	NL	NL-FOL	FOL	NL+FOL
GPT-3.5	58.34	55.96	57.92	57.75
GPT-4	64.16	63.82	64.01	65.21

Table 2.10: Comparison of the results across different input formats with few-shot prompting. NL, NL-FOL, FOL, NL + FOL stands for NL prompting, execution accuracy of NL-FOL translation, using only FOL in the prompt and using concatenated NL and FOL in the prompt respectively.

2.6.2.0.2 Model performance is not affected by the premise ordering

To test if the premise ordering in FOLIO has spurious correlations with the conclusion label which a model can exploit, we shuffle the input premises to evaluate models. We find that accuracy increases or decreases by roughly 1% in most settings compared to our unshuffled premises. This indicates that the ordering of premises in FOLIO examples does not yield significant information about the label, and thus models will not be able to use the premise ordering as a strong heuristic or statistical feature for its predictions.

2.6.2.0.3 Using both NL sentences and FOL formulas in the prompt performs better

FOL formulas have a clearer and more straightforward logical structure than NL sentences. Therefore, we test GPT-3.5 and GPT-4 with another two settings for truth value prediction using few-shot prompting: 1) using only FOL formulas in the prompt; 2) using both NL sentences and FOL formulas by

concatenating each NL sentence and its annotated FOL statement. As shown in Table 2.10, the performance slightly increases in the NL+FOL setting for GPT-4 while GPT-3.5 performs worse in both the NL+FOL and the FOL-only settings. In other words, FOL always serves as additional useful information for GPT-4, but not for GPT-3.5 regardless of whether FOL is concatenated with NL. This observation resonates with the finding that GPT-4 performs much better than GPT-3.5 on code-related tasks (Ni et al., 2023).

2.7 Conclusion

In this chapter, we introduced FOLIO, an expert-written dataset for logical reasoning equipped with FOL formulas. The examples in FOLIO are created based on real-world knowledge with natural language. It exhibits a large number of distinct logic patterns and a large vocabulary. Experiments show that FOLIO presents a challenge for one of the most capable Large Language Model publicly available.

2.8 Limitations

We focus on collecting a very high-quality dataset in evaluating logical reasoning rather than merely a large dataset. Optimizing for quality required

us to adopt a rigorous annotation process with domain experts selected based on a few important criteria as mentioned in Appendix A: Annotator Selection. Significantly scaling up this process would have required resources beyond our current means and we are unable further expand our dataset for investigating how the size of training data affects the performance of fine-tuning experiments. We encourage the community to apply our annotation protocol to expand this realistic and complex FOL reasoning story set.

Dataset	Natural Language		Logic	
	Text Source	Real-World Resource	# Reasoning Step	#Inference rule
ProofWriter (Tafjord et al., 2021)	Synthetic	No	0 - 5	12
ProntoQA-OOD (Saparov et al., 2023)	Synthetic	No	×	6
P-FOLIO	Expert-written	Yes	0 - 20	31

Table 3.1: Comparison of P-FOLIO and other datasets of proofs

Chapter 3

Evaluating and Improving Logical Reasoning with Abundant Human-Written Reasoning Chains

3.1 Introduction

In the previous chapter, we have introduced FOLIO, where a logical reasoning story consists of a series of premises and one or more conclusions (Han et al.,

A FOLIO example	
NL premises	**NL Conclusion -> Label**
1. There are no mansion houses in an urban area. 2. All skyscrapers are in urban areas. 3. Every creepy haunted house is a mansion house. 4. Every terrifying building on Halloween is a creepy haunted house. 5. The LaLaurie House is a creepy haunted house or a terrifying Halloween building.	If the LaLaurie House is either a skyscraper or a creepy haunted house, then it is not a mansion house. -> F

Table 3.2: An example story in FOLIO consisting of five premises and one conclusion with the label for the conclusion.

2022). The goal of logical reasoning is to determine the truth values of the conclusions based on the premises (Russell and Norvig, 2010). However, it cannot be determined whether a machine is indeed equipped with capabilities of logical reasoning even if it is able to output the correct truth value. Generating proofs or reasoning chains that support its decision is essential for explainability. Resolution-based symbolic proofs can be generated automatically with a first-order logic solver given explicit logic structures, but they are hard to read and cannot be written in natural language in a straightforward way in most cases (Russell and Norvig, 2010). Although studies on natural language proof generation for logical reasoning have been conducted before, they focus on synthetically generated logical reasoning datasets (Saha et al., 2020, 2021; Saparov et al., 2023; Tafjord et al., 2021). These logical reasoning chains contains much less natural language variation and challenging reasoning patterns than realistic logic stories (Han et al., 2022).

In this chapter, we present *P-FOLIO*, a new dataset consisting of human-written proofs for an existing popular dataset of logical reasoning, FOLIO (Han et al., 2022) which consists of realistic logical reasoning stories written by

humans. P-FOLIO is collected with an annotation protocol for annotators to write proofs in a step-by-step manner without using proof by contradiction or proof by case. Table 3.2 shows an example story in FOLIO and Table 3.3 shows its human-written reasoning chains in P-FOLIO. P-FOLIO further advances our effort to build a foundational evaluation framework for LLM reasoning by augmenting the outcome-based approach of FOLIO with a process-based perspective. While FOLIO demonstrates that truth-value prediction alone cannot verify genuine logical competence, P-FOLIO introduces structured, human-written reasoning chains that reveal how models arrive at conclusions. This shift enables us not only to evaluate whether a model is correct, but to inspect and improve the intermediate reasoning steps it takes—moving beyond answer accuracy to assess reasoning faithfulness, structure, and soundness.

P-FOLIO proofs are expert-written. They are more logically diverse and challenging than previous logical reasoning datasets equipped with reasoning chains. The number of proof steps in P-FOLIO spans from 0 to 20 and the proofs use diverse inference rules, containing 12 types of widely used and straightforward inference rules and another 20 types of complex inference rules. 26% of a total of **1430** proofs in P-FOLIO consist of more than five reasoning steps and 4% of proofs consist of more than 10 reasoning steps. We show the comparison of P-FOLIO and the previous proof collections for logical reasoning, ProofWriter (Tafjord et al., 2021) and ProntoQA-OOD(Saparov et al., 2023), in

Table 3.1. Both ProofWriter and ProntoQA-OOD are synthetically constructed. ProofWriter contains at most 5 reasoning steps and ProntoQA-OOD only uses 6 types of inference rules.

We use P-FOLIO to provide a more in-depth evaluation and to improve large-language-model reasoning capabilities. We first evaluate LLM reasoning capabilities at a fine-grained granularity via single-step inference rule classification and single-step derivation reasoning. Given that the model-generated reasoning chain could take a completely different path than the human-annotated one, we use the pass@k (Chen et al., 2021b; Kulal et al., 2019), a metric commonly used for evaluating code generation, for evaluating the overlap between them. We show that human-written reasoning chains significantly boost the logical reasoning capabilities of LLMs via many-shot prompting and fine-tuning. Furthermore, fine-tuning Llama3-7B on P-FOLIO improves the model performance by more than 10% on three other out-of-domain logical reasoning datasets. Finally, we conduct detailed analysis to show where most poweful LLMs fall short in reasoning.

The contributions of this chapter are as follow: (1) We collected expert-written reasoning chains for an existing popular logical reasoning dataset. The collected reasoning chains are more logically diverse and challenging than previous logical reasoning datasets equipped with reasoning chains. (2) We evaluate LLM-generated proofs with both automatic metrics and human

evaluation. (3) We show that fine-tuning on our proof collection yields substantial improvements for LLM logical reasoning capabilities on both in-domain and out-of-domain test sets. (4) We identify weaknesses of LLM reasoning based on our findings.

3.2 Related Work

3.2.0.0.1 Natural language reasoning

Existing studies on natural language reasoning (Bang et al., 2023; Creswell et al., 2022; Prystawski et al., 2023; Wei et al., 2022b; Yao et al., 2023) focus on using different prompting strategies on large language models (LLMs) for better results. Despite significant progress on the LLM prediction accuracy of the final answer, no work has been done to evaluate the correctness of the intermediate steps. In our work, we propose to evaluate intermediate steps at different levels of granularity in order to better probe into the reasoning capabilities of LLMs.

3.2.0.0.2 Improving LLM Logical reasoning capabilities

Luo et al. (2024); Ranaldi and Freitas (2024) fine-tuned language models with logical reasoning data to improve logical reasoning capabilities of LLMs. Large language models have also been directly used as soft logic reasoners and a variety of prompting techniques are proposed in order to improve their

performance under this paradigm (Wei et al., 2022b; Yao et al., 2023; Zhou et al., 2024b). Using large language models as semantic parsers has also shown improvement on the reasoning performance (Olausson et al., 2023; Pan et al., 2023) where natural language reasoning problems are first parsed into logical forms before being fed into an inference engine to output the final answer.

3.2.0.0.3 NATURAL LANGUAGE PROOF GENERATION

ProofWriter (Tafjord et al., 2021) and FLD (Morishita et al., 2023) are logical reasoning datasets equipped with natural language proofs, however both of them are synthetically generated dataset which neither contains abundant natural language variation nor encompasses challenging logical reasoning patters. Previous studies on proof generation focus on ProofWriter(Morishita et al., 2023; Saha et al., 2020, 2021; Yang et al., 2022) and ProntoQA (Saparov et al., 2023). LogicBench is a synthetically generated natural language QA dataset and is used for evaluating the logical reasoning ability of LLMs (Parmar et al., 2024). While FOLIO covers first-order logic and one or more inference rules are used in each example, LogicBench focuses on reasoning patterns covering propositional logic, first-order logic, and non-monotonic logic and focuses on the use of a single inference rule for each example.

We collect proofs for FOLIO (Han et al., 2022) instead, a realistic expert-written logical reasoning dataset. Such proofs need to be written from scratch

P No Used	Derivation	D No.	Inference Rule
1	If there is a mansion house in the area, then it is not an urban area.	D1	Transposition
2	Something that is not in an urban area is not a skyscraper.	D2	Transposition
3, D1	A creepy haunted house is not in an urban area.	D3	Hypothetical Syllogism
D3, 2	A creepy haunted house is not a skyscraper.	D4	Hypothetical Syllogism
4, D4	Every terrifying Halloween building is not a skyscraper.	D5	Hypothetical Syllogism
D5, 5	The LaLaurie House is not a skyscraper or is a terrifying Halloween building.	D6	Universal Instantiation
D6, D4	The LaLaurie House is not a skyscraper or is not a skyscraper.	D7	Universal Instantiation
D7	The LaLaurie House is not a skyscraper.	D8	Idempotence
5, 4	The LaLaurie House is a creepy haunted house or a creepy haunted house.	D9	Universal Instantiation
D10	The LaLaurie House is a creepy haunted house.	D11	Idempotence
D11, 3	The LaLaurie House is a mansion house.	D12	Universal Instantiation
D8, D11	The LaLaurie House is a creepy haunted house and not a skyscraper.	D13	Conjunction Introduction
D13, D12	The LaLaurie House is a creepy haunted house and a mansion house and not a skyscraper.	D14	Conjunction Introduction

Table 3.3: Proof written for the example story in FOLIO. “P No” is premise number and “D” stands for derivation.

and are hard and time-consuming to write because humans need to manage both the language and reasoning complexity in the proof-writing process and manually construct many steps of reasoning. The resulting proofs contain more diverse types of inference rules and reasoning patterns in addition to containing more natural language variation and ensured semantic richness.

3.3 P-FOLIO

In this section we describe the construction and properties of our dataset P-FOLIO.

Type	Rules
Widely-used rules	universal instantiation (UI), hypothetical syllogism (HS), Modus Ponens (MP), Modus Tollens (MT), disjunctive syllogism (DS), disjunction introduction (DI), conjunction introduction (CI), conjunction elimination (CE), transposition (TP), material implication (MI), existential introduction (EI)
Boolean Identities	"A or A" is equivalent to A; A and A is equivalent to A
Complex rules	From $a \oplus b$, we know $a \rightarrow \neg b$; from $a \rightarrow b$ and $b \rightarrow a$, we know $\neg(a \oplus b)$. $\neg b \oplus b$ is always true; $a \oplus b$ is equivalent to $\neg a \oplus \neg b$ From $\forall \mathrm{x}(\mathrm{A}(\mathrm{x}) \rightarrow \mathrm{C}(\mathrm{x}))$ and $\mathrm{A}(paul) \wedge B(paul)$, we know $C(paul) \wedge B(paul)$.

Table 3.4: Categories of inference rules used in our annotation protocol.

3.3.1 Inference Rules

We first define a set of inference rules that can be used for derivations of each proof step.

3.3.1.0.1 Widely-used inference rules.

The most widely used logical reasoning inference rules include universal instantiation, hypothetical syllogism, modus ponens, modus tollens, disjunctive syllogism, conjunction introduction, conjunction elimination, transposition, disjunction introduction, material implication and existential introduction.

3.3.1.0.2 Boolean identities.

During the pilot annotation process, we found that boolean identities are needed for certain derivations. For example, if "A or A" is true then "A" is true. We therefore allow the usages of boolean identities

3.3.1.0.3 Complex inference rules.

Some complex inference rules are intuitively correct and can also be proved logically correct with an inference engine. For example, from "A XOR B", we know that "A implies not B". We include in Table 3.4 the different categories of inference rules used in our protocol.

3.3.2 Dataset Annotation

We offer an annotation protocol that facilitates annotators to annotate well-structured proofs for any logical reasoning problems in a step-by-step manner. Our annotators are selected based on the the following criteria: 1). They are college or graduate students who are either native English speakers or have near-native English proficiency. 2) They have a formal background in first-order logic, gained through either relevant coursework or self-directed study in first-order logic or semantic parsing. 3). we conducted in-person interviews with all the annotators to understand if they are motivated about completing the task before we added them into the annotator pool. All the annotators selected have expressed a keen interest in solving logical puzzles and are guaranteed to have a significant amount of time commitment. We also give the annotators several detailed tutorial sessions on how to write a proof and give detailed annotation guidelines to minimize possible ambiguities that can occur in the annotation process and make sure the annotation protocol is well-understood. A total of six annotators were selected through this process.

3.3.2.0.1 ANNOTATION PROTOCOL.

An annotator is presented with a logical reasoning problem consisting of a series of premises and one conclusion. For each step of a proof, we ask the annotator to write the indices of the premises used, a natural language

derivation deduced from the premises used and the inference rule used for the derivation. The natural language derivation can then be used as a premise for future proof steps. We also specify that only 1 or 2 premises should be used in each step. Table 3.2 shows an example of the FOLIO dataset and Table 3.3 shows the entire proof annotated for the example. The proof consists of 14 steps in total with 14 natural language derivations and the final derivation is the conclusion. This annotation protocol facilitates humans to annotate well-structured proofs for first-order logic reasoning problems in a step-by-step manner and the resulting proofs are easier to read and evaluate than proofs written in pure natural language without being divided into individual steps.

We provide the detailed guidelines on proof annotation in the appendix. The guideline contains detailed instructions on how to write the proofs. Multiple examples were provided to the annotators and we conducted in-person tutorials for the annotators. We promptly ask the annotators for feedback and resolve their questions throughout the entire annotation process. We found that annotators are well-versed in the well-established inference rules and boolean identities. However, sometimes universal instantiation inferences can be wrongly identified as hypothetical syllogism. At the initial stage of annotation, annotators sometimes identified intuitively correct but logically incorrect complex rules. Therefore, we ask them to verify each complex rule with a logic solver before using it.

We asked the annotators to keep a record of the time taken for writing each proof. Most of the proofs requiring 1-5 steps take less than 20 minutes to annotate with five minutes being the average while more complicated proofs requiring six steps or more take 20 minutes to 90 minutes to annotate with 1 hour being the average since to write these proofs requires a significant amount of deliberate thinking and annotators need to go through an intermediate deduction process before writing down the proof steps.

For first-order logical reasoning questions, multiple reasoning paths can be undertaken to derive the same conclusion. Due to the time-consuming nature of the task, only one annotator annotates each sample with one reasoning path in our annotation procedure although in our pilot annotation study we identified different annotators sometimes provide different proofs for the same example.

3.3.2.0.2 QUALITY CONTROL VIA CROSS CHECKING.

We conducted a comprehensive cross-checking process for all the proofs written by dividing the annotators into 3 groups, each comprised of 2 people. We asked the two annotators in each group to review and validate each other's work and address any identified errors. We also provided annotators with cross checking guidelines to make the process more efficient. This approach greatly reduces errors for P-FOLIO.

3.3.3 Dataset statistics

3.3.3.0.1 Number of proof steps.

We show the distribution of the number of proof steps in the entire P-FOLIO dataset in Figure 3.2. we have carefully validated that each of the splits follows a similar distribution with less than a 2% difference in each split. The majority of written proofs contain five steps or less while written proofs comprised of 6-10 steps also take up a significant portion. There are 55 proofs which are comprised of 11 steps or more. This represents the portion of the dataset with the highest complexity.

3.3.3.0.2 Inference rule distribution.

The widely-used inference rule and boolean identities distribution is shown in Figure 3.3. Among the widely-used inference rules, universal instantiation and hypothetical syllogism are the most common inference rules in P-FOLIO while disjunctive syllogism, conjunction introduction, conjunction elimination and transposition also appeared a large number of times. Although Modus ponens, Modus tollens, existential introduction, disjunction introduction, material implication and idempotent are the least common, their are 50 - 150 number of them in P-FOLIO. The occurrences of complex inference rules all range from 1 to 29. The complex inference rules are essential for making certain derivations although their occurrences are less frequent.

3.4 Reasoning Evaluation

We propose three tasks to evaluating LLM reasoning capabilities with P-FOLIO at different levels of granularities.

3.4.1 Single-step inference-rule classification

Given a single step of inference in a human-annotated proof which consists of the premises used $P = \{P_1, P_2\}$ and a derivation d_1, single-step inference-rule classification aims to identify which inference rule has been used to arrive at d_1 from P. This task aims to evaluate whether a model knows the type of inference required for a single inference step.

3.4.2 Single-step derivation reasoning

Given a single step of inference in a human annotated proof, the goal of single-step derivation reasoning is to ask the model to generate the truth value of a derivation based on the corresponding premises. The truth value label is always True.

3.4.3 Proof generation

Given a series of premises $P = \{p_1, p_2, ..., p_n\}$ and a conclusion c_1, the goal of proof generation is ask the model to generate the proof and the final truth value. This task aims to evaluate the overall proof generation capability.

3.4.4 Proof generation evaluation

3.4.4.0.1 PASS@K.

Considering that a model-generated reasoning chain can diverge significantly from a human-annotated one in logical reasoning, we sample multiple reasoning chains from GPT-3.5 and GPT-4 and employ pass@k metrics (Chen et al., 2021a) to assess the quality of these model-generated chains. GPT-4 is utilized to evaluate whether two reasoning chains follow a similar path to reach their conclusions. We provide the prompts used for this in the Appendix. If we sample k proofs from a model, we define pass@k to be the percentage of instances where at least one proof matches the human-written proof.

3.4.5 Task comparison

Single-step inference-rule classification and single-step derivation allow for a more granular assessment of how well the model can follow logical rules and derive conclusions by breaking down complex reasoning chains

into individual steps. By analyzing each step independently, identify specific strengths and weaknesses can be identified in the model's reasoning processes, facilitating a better understanding of the model's overall performance and targeted improvements. The entire proof generation evaluates the reasoning capabilities of LLMs by challenging them with complex reasoning problems. This approach requires the model to generate a complete proof from start to finish, demonstrating its ability to handle intricate logical structures and multi-step derivations. By assessing the model's performance on entire proofs, we can gauge its proficiency in maintaining logical coherence, applying relevant rules, and effectively managing the complexity inherent in comprehensive reasoning tasks. This evaluation method provides a holistic view of the model's reasoning abilities.

3.5 Experimental Results

We conduct experiments using P-FOLIO. The train/dev/test split is 70%/15%/15%, which is the same as the split for FOLIO (Han et al., 2022). For all experiments involved with GPT-4 (OpenAI et al., 2023), we use gpt-4-0125-preview and the temperature was set to 0.

	Inference Rule		Truth Value	
	w/o Examples	**w Examples**	**w/o Explanation**	**w Explanation**
Established Rules	54.72	55.08	85.31	87.13
Complex Rules	40.8	43.62	70.67	80.43

Table 3.5: Single-step inference rule classification and single-step derivation reasoning results.

3.5.1 Single-step evaluation

We conduct single-step inference rule classification and single-step derivation reasoning on individual proof steps in P-FOLIO. For single-step derivation reasoning, we prompt GPT-4 to output the classification results in zero-shot setting and 5-shot prompting. The results of inference rule classification are shown in the first two columns of Table 3.5. Giving GPT-4 a few examples leads to better results for complex inference rules while the improvement is minimal for widely used inference rules. For single-step inference rule classification, we test two settings. We prompt GPT-4 to output the truth value directly in one setting. In the other setting, we prompt GPT-4 to output the explanation first before generating the final truth value. The results of single-step inference rule classification are shown in the last two columns of Table 3.5. Prompting GPT-4 to output the explanation before generating the final truth value leads to better results for both widely-used rules and complex rules.

Table 3.6: Instruction used for prompting GPT-4 for zero-shot proof generation.

Using deductive reasoning, find out the truth values of the conclusions based on the premises. The truth value can be True, False or Uncertain. First show the reasoning process, and then output the truth value in the format of "Truth value: ".

3.5.2 Proof generation

3.5.2.0.1 Zero-shot Prompting LLM

We use GPT-4 (OpenAI et al., 2023) to generate proofs and truth value for all the examples. By explicitly prompting GPT-4 to generate the reasoning process before the final answer increase the performance by 5% over prompting GPT-4 to generate only the final answer.

3.5.2.0.2 Instruction used in the GPT-4 prompt

Table 3.6 shows the instruction used in the GPT-4 prompt.

#shot	R-1	R-2	R-L	Acc (%)
0-shot	60.2	52.7	61.5	65.2
5-shot	63.9	54.2	64.1	68.1
10-shot	65.8	54.9	65.9	70.7
20-shot	70.6	59.5	69.6	75.3
40-shot	73.7	62.1	72.7	77.4
60-shot	78.2	66.3	78.1	80.6

Table 3.7: Result of many-shot prompting with GPT-4.

We manually inspect all the generated proofs. Among the examples with correct truth values, 10.7% exhibited incorrect reasoning processes. In these

Method	Answer Acc	Proof Acc
w/o proofs.	60.2	-
w/ proofs.	65.2	58.2

Table 3.8: Comparison of final answer accuracy and proof accuracy (%).

	FOLIO (2022)		BirdElectricity (2021)		ProntoQA (2023)		bAbi-deductive (2015)	
	w/o Proofs	w/ Proofs	w/o Proofs	w/ Proofs	w/o Proofs	w/ Proofs	w/o Proofs	w/ Proofs
Zero-Shot Flan-T5-Large	5.3%	5.3%	42.1%	40.9%	6.5%	5.0%	38.9%	40.4%
Fine-Tune Flan-T5-Large	65.9%	67.5%	53.4%	60.8%	33%	41%	44.70%	50.30%
Zero-Shot Llama3-8B	52.1%	54.1%	54.8%	57.5%	63.1%	64.9%	72.1%	75.7%
Fine-tune Llama3-8B	**70.2%**	**76.3%**	**60.8%**	**66.7%**	**76.1%**	**80.6%**	**85.1%**	**87.9%**

Table 3.9: Fine-tuning results and out-of-domain generalization capabilities of LLMs trained on FOLIO and P-FOLIO. "w/o Proofs" indicates the model is trained with FOLIO and "w/Proofs" indicates the model is trained with P-FOLIO.

cases, the model either struggled to form the correct reasoning chain for complex problems involving many reasoning steps or used commonsense reasoning as a shortcut. Among the examples with incorrect truth values, 70% failed to form the correct reasoning chain for complex problems. Additionally, 10% contained wrong derivations at a certain steps in the reasoning process. About 15% of the examples presented conclusions with language or logical structures too intricate for the model to fully comprehend. Another 5% incorrectly applied commonsense reasoning as a shortcut, leading to errors in the final conclusions. We provide more analysis in section 3.6.

Model	Pass @1	Pass @5	Pass @10	Pass @20
GPT-3.5	49.5	51.6	53.9	55.7
GPT-4	55.3	59.2	62.1	66.9

Table 3.10: Pass@k results for GPT-3.5 and GPT-4

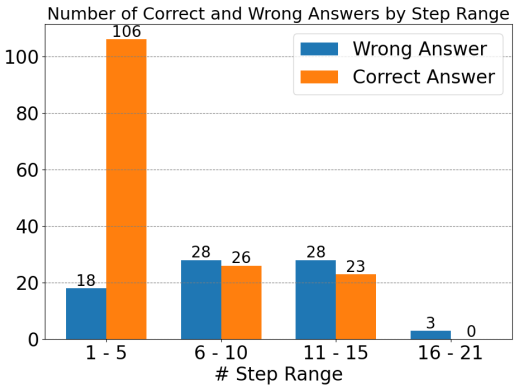


Figure 3.1: Distribution of the number of correct & wrong answers vs the number of human-written proof steps

Added Rule Type	**Accuracy (%)**
Complex rules	70.60
Widely-used rules	63.20

Table 3.11: Final answer accuracy of GPT-4 with human-written derivations

3.5.2.0.3 MANY-SHOT PROMPTING LLMS

We conduct many-shot prompting experiments with GPT-4 using 5, 10, 20, 40, and 60 examples in in-context learning. Sixty is the maximum number of example and proof pairs we can fit into the context window of GPT-4. We present the results using ngram-overlap-based metrics, including ROUGE-1, ROUGE-2, and ROUGE-L (Lin, 2004), comparing the model-generated proofs with human-written proofs. Additionally, we report the final answer prediction accuracy. Table 3.7 demonstrates that adding 5 examples to the prompt improves GPT-4's performance over zero-shot prompting. Increasing the number of examples for in-context learning from 5 to 20 results in a further performance improvement

Prompt Using deductive reasoning, find out the truth values of the conclusions based on the premises. The truth value can be True, False or Uncertain. First show the reasoning process, and then output the truth value in the format of "Truth value: ". Premises: 1. If someone has never left the United States, then they have not been to Europe. 2. If someone has traveled to the Iberian Peninsula, then they have been to Europe. 3. Everyone who has visited Barcelona has traveled to the Iberian Peninsula. 4. Everyone who has taken a selfie at the Basilica Familia Sagrada has visited Madrid. 5. If Kate has either taken a selfie at the Basilica Familia Sagrada or has never left the United States, then Kate has not either visited Barcelona or never left the United States. Conclusion: Kate has not taken a selfie at the Basilica Familia Sagrada.
GPT-4 Output Reasoning process: Let's analyze the premises and see if we can deduce the truth value of the conclusion. Premise 1: If someone has never left the United States, then they have not been to Europe. Premise 2: If someone has traveled to the Iberian Peninsula, then they have been to Europe. Premise 3: Everyone who has visited Barcelona has traveled to the Iberian Peninsula. Premise 4: Everyone who has taken a selfie at the Basilica Familia Sagrada has visited Madrid. Premise 5: If Kate has either taken a selfie at the Basilica Familia Sagrada or has never left the United States, then Kate has not either visited Barcelona or never left the United States. ... *Verbatim repetition of the original premises...* Conclusion: Kate has not taken a selfie at the Basilica Familia Sagrada. From Premise 5, we know that if Kate has taken a selfie at the Basilica Familia Sagrada, then she has visited Barcelona or left the United States. However, we don't have any information about whether Kate has actually taken a selfie at the Basilica Familia Sagrada or not. Therefore, we cannot deduce the truth value of the conclusion based on the given premises. Truth value: Uncertain

Table 3.12: Case Study

of approximately 7%. Using 60 examples in the prompt significantly enhances performance compared to zero-shot prompting.

3.5.2.0.4 PASS@K EVALUATION

Table 3.10 presents the pass@k results for GPT-3.5 and GPT-4 on the P-FOLIO dataset, illustrating the comparative performance of these models

in generating reasoning chains. Pass@k metrics indicate the percentage of instances where at least one out of k sampled reasoning chains matches the human-annotated proof. The results show a clear performance improvement from GPT-3.5 to GPT-4 across all values of k. Specifically, GPT-4 achieves a pass@1 of 55.3%, outperforming GPT-3.5's 49.5%. This trend continues with GPT-4 attaining pass@5, pass@10, and pass@20 scores of 59.2%, 62.1%, and 66.9% respectively, compared to GPT-3.5's 51.6%, 53.9%, and 55.7%. These findings suggest that GPT-4 is more effective in generating reasoning chains that align with human-annotated proofs, highlighting its enhanced capability in logical reasoning tasks.

3.5.2.0.5 Prompt used for evaluating reasoning path

We provide an example in Table 3.13 including our prompt used and GPT-4 input and output for how GPT-4 is used to evaluate whether two reasoning chains follow a similar path to reach their conclusions.

3.5.2.0.6 Fine-tuning and Out-of-Domain Generalization

We test the performance of fine-tuning with Flan-T5-Large (Chung et al., 2022) and Llama3-8B (Dubey et al., 2024) and test the out-of-domain generalization capabilities of LLMs fine-tuned on P-FOLIO. We compares two models, Flan-T5-Large and Llama3-7B (Dubey et al., 2024), in both zero-shot and fine-tuned

Prompt
Given a deductive reasoning question, demonstrate whether the two reasoning chains are semantically similar and follow the same reasoning path to derive the final answer. After your explanations, output your decision in the format of "Decision: ". Your decision should be either Yes or No.
Premises: 1. Northwich Victoria FC is a football club that plays at Wincham Park. 2. Simon Hackney is a member of Northwich Victoria FC. 3. Northwich Victoria FC is part of the Northern Premier League. 4. If someone is a member of an organization that is part of a group, then the person is a member of the group. 5. If someone is a member of an organization and the organization plays at a certain place, then the person plays at the place.
Conclusion: Simon Hackney is a member of the Northern Premier League.
Reasoning chain A: Reasoning process: 1. Simon Hackney is a member of Northwich Victoria FC (Premise 2). 2. Northwich Victoria FC is part of the Northern Premier League (Premise 3). 3. If someone is a member of an organization that is part of a group, then the person is a member of the group (Premise 4). 4. Therefore, Simon Hackney is a member of the Northern Premier League (from 1, 2, and 3).
Reasoning chain B: Reasoning process: From premise 2 and premise 3, we know that Simon Hackney is a member of Northwich Victoria FC, which is a part of the Northern Premier League. From this and premise 4, we know that Simon Hackney is a member in the Northern Premier League.

GPT-4 Output
Analyzing Reasoning Chain A: Step 1: It asserts that Simon Hackney is a member of Northwich Victoria FC, which directly uses Premise 2. Step 2: It states that Northwich Victoria FC is part of the Northern Premier League, utilizing Premise 3. Step 3: It invokes the rule from Premise 4, which is a conditional statement about membership extension from an organization to a group. Step 4: This is the conclusion step that synthesizes the information from steps 1, 2, and 3 to conclude that Simon Hackney is a member of the Northern Premier League. This follows a logical and direct application of the premises. **Analyzing Reasoning Chain B**: Initial Statement: Acknowledges that Simon Hackney is a member of Northwich Victoria FC (from Premise 2) and that the club is part of the Northern Premier League (from Premise 3). Subsequent Logic: Directly applies Premise 4, which relates individual membership in an organization to membership in a broader group if the organization is part of that group. Conclusion: Concludes that Simon Hackney is a member of the Northern Premier League, which aligns with the conclusion from the initial statement and the application of Premise 4. Comparison and Decision: **Both reasoning chains follow the same logical structure:** They acknowledge Simon Hackney's membership in Northwich Victoria FC. They recognize the club's affiliation with the Northern Premier League. They apply the same general rule from Premise 4 to extend Simon's membership from the club to the league. The primary difference lies in the explicitness and breakdown of steps: Chain A is more structured and delineated, breaking down each logical step explicitly. Chain B is more concise, combining some steps but still following the same logical pathway. **Decision: Yes — Both reasoning chains are semantically similar and follow the same reasoning path to derive the final answer.**

Table 3.13: Example including prompt, GPT-4 model input and output for evaluating whether two model outputs following the same reasoning path.

configurations. We provide the hyper-parameter settings in the Appendix. Table 3.9 highlights that fine-tuning on FOLIO (w/o Proofs) and P-FOLIO (w/Proofs) significantly enhances the performance of both models across all tasks. Llama3-7B consistently outperforms Flan-T5-Large.

Table 3.9 also shows that fine-tuning without proofs using the FOLIO dataset and fine-tuning with proofs boost the performance on the three out-of-domain logical reasoning datasets tested, BirdElectricity (Tafjord et al., 2021), ProntoQA

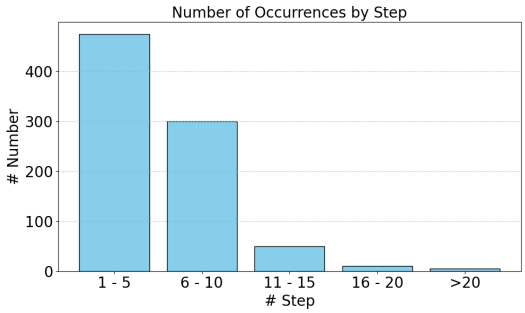


Figure 3.2: Distribution of the number of proof steps

(Saparov and He, 2023) and bAbi-deductive (Weston et al., 2015). The inclusion of human-written proofs consistently improves the accuracy for fine-tuned models, underscoring the importance of proofs in bolstering the generalization capabilities of LLMs in out-of-domain datasets. These results suggest that LLMs fine-tuned with human-written rationales are able to generalize learned reasoning strategies to new, unseen tasks, demonstrating robust out-of-domain generalization.

3.6 Analysis

3.6.0.0.1 Most powerful LLMs fail on examples with 10 steps or more in human-written reasoning chains.

Figure 3.1 illustrates the performance of GPT-4 on logical reasoning tasks of varying complexity. While GPT-4 accurately answers most questions requiring 1-5 human-written reasoning steps, its accuracy drops significantly for more

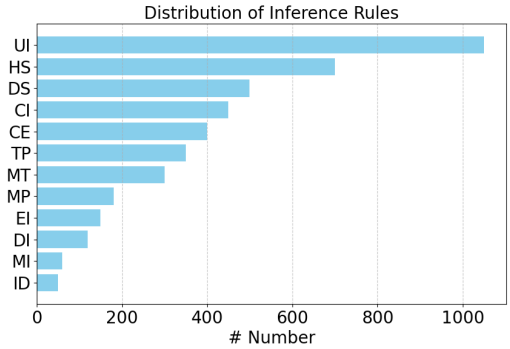


Figure 3.3: Distribution of widely used inference rules

complex tasks. For questions involving 6-10 reasoning steps, GPT-4 answers correctly only about half the time. Its performance further declines with examples requiring 11-15 steps, where it fails more than half the time. For tasks with 16 or more reasoning steps, GPT-4 is unable to provide correct answers at all.

3.6.0.0.2 MAKING NEW DERIVATIONS WITH COMPLEX RULES COULD BE ONE OF THE BOTTLENECKS FOR LLM REASONING

To gain a more detailed understanding of LLM reasoning capabilities, we conducted an experiment by augmenting the original logical reasoning examples with additional derivations. First, we created a set of examples by adding derivations derived exclusively from widely-used rules, ensuring that no complex or less common rules were included. Next, we created another set of examples by incorporating derivations that were based solely on well-established rules, explicitly excluding any derivations that originated from widely-used rules. This

approach allowed us to isolate the impact of different types of inference rules on the reasoning performance of LLMs.

Table 3.11 reports the zero-shot prompting performance of GPT-4 on the two new set of augmented examples we created. The results reveal that GPT-4 achieves much higher accuracy on the set augmented with complex inference rule derivations, at 70.60%, compared to an accuracy of 63.20% on the set with widely-used rule derivations. This indicates that while GPT-4 performs better when leveraging complex inference rules, the creation of new derivations using these rules could pose a significant challenge, potentially acting as a bottleneck for large language model (LLM) reasoning. This suggests that further advancements in handling complex rule derivations may be necessary to improve the overall reasoning capabilities of LLMs.

3.6.0.0.3 FAULTY REASONING

We show a case study where GPT-4 makes a mistake in writing the proof and producing the final truth value. The logical reasoning example and the GPT-4 output are shown in Table 3.12. This mistake happens due to the model's inability to form the correct reasoning chain for a complex problem involving lots of steps, which is also the main cause of most model failures. Notably, in this scenario, the model indeed follows formal rules in each step perfectly and can make correct derivations based on the premises provided. However, it is unable

to reach a conclusion because it is unable to correctly derive certain crucial intermediate steps in the reasoning process. We observe a similar pattern across different examples that require 10 reasoning steps or more. As we can see from the GPT-4 output, the model fails to find the correct reasoning chain, thereby not finding the final conclusion and producing "Uncertain" for the truth value although the correct answer should be "True". In particular, the model fails to arrive at the derivation that "No one both has visited Iberian Peninsula and never left the United States.", which can be derived from "If someone has visited Iberian Peninsula, then they have left the United States." using one of the complex rules: "from $a \rightarrow \neg b$, we know $\neg(a \wedge b)$".

3.7 Conclusion

We presented P-FOLIO, a new dataset consisting of human-written proofs for FOLIO, a set of realistic logical reasoning stories written by humans. 26% of a total of 1430 proofs in P-FOLIO consist of more than five reasoning steps and 4% of proofs consist of more than 10 reasoning steps. We further use P-FOLIO to evaluate and improve large-language-model reasoning capabilities. We propose tasks with different levels of granularities and use various metrics to evaluate the performance of LLM reasoning capabilities. We show that human-written reasoning chains significantly boost the logical reasoning capabilities of LLMs

via many-shot prompting and fine-tuning. Fine-tuning on P-FOLIO improves the model performance by more than 10% on three other out-of-domain logical reasoning datasets. We conduct detailed analysis to show where most poweful LLMs fall short in reasoning.

Chapter 4

Meta Selection of Natural Language and Symbolic Language for Enhanced LLM Reasoning

4.1 Introduction

As introduced in chapter 2, reasoning with LLMs is typically approached through two paradigms. The first paradigm focuses on designing various prompting strategies to elicit detailed and natural language (NL) reasoning processes. This line of research continues from Chain-of-Thought prompting (Wei et al., 2023; Zheng et al., 2024a; Zhou et al., 2023). The second one leverages LLMs to generate symbolic language, which can then be executed with external

tools to derive the final answer (Chen et al., 2023; Gao et al., 2023; Gou et al., 2024; Gu et al., 2024; Olausson et al., 2023; Ye et al., 2023; Zhou et al., 2024a).

Reasoning with symbolic language (SL) incorporates external tools to perform logical deduction and arithmetic calculation, where existing LLMs suffer at when reasoning with purely natural language (Zhou et al., 2024c). Natural language reasoning is highly flexible and broadly accessible to humans, but it often suffers from ambiguities and imprecision (Gao et al., 2023; Liu et al., 2023). In contrast, SL reasoning, being grounded in formal rules and representations, can be executed by machines and offers precise, unambiguous inferences (Chen et al., 2023; Zhou et al., 2024a).

The strengths of each paradigm are complementary: NL reasoning handles semantic richness better, while symbolic representations excel at precise, unambiguous inference and error-checking (Chen et al., 2023; Olausson et al., 2023). Relying exclusively on one approach can thus lead to suboptimal performance. Pure NL reasoning may overlook logical subtleties, whereas purely symbolic methods risk overlooking the complexity and nuances in real-world language (Han et al., 2024; Olausson et al., 2023).

In this chapter, we propose HybridMind, which leverages the best of both worlds. By dynamically meta-selecting whether to reason in natural language, symbolic language, or a combination of the two, HybridMind tailors the solution style to the characteristics of each problem. Conceptual or explanatory tasks

benefit from the clarity of step-by-step NL reasoning, while problems requiring exact heavy computation are more effectively handled in symbolic form (Gao et al., 2023; Zhou et al., 2024c). In other words, rather than forcing a single method to fit all problem types, HybridMind adaptively selects the right choice for a problem.

Through extensive experimentation, we demonstrate that fine-tuning or prompting with this meta-selector boosts performance over using only a single reasoning style. For fine-tuning, we adopt Low-rank Adaptation (LoRA) (Hu et al., 2021) for both fully-supervised finetuning and STaR, an iterative fine-tuning method (Zelikman et al., 2022a). We propose a systematic process for producing labeled training data for our meta-selector. Specifically, we employ GPT-4o, a state-of-the-art large language model (LLM) to solve logical or mathematical reasoning problems using four approaches (NL reasoning only, SL reasoning only, two approaches of mixing NL and SL reasoning). The meta-selector's label is assigned to whichever approach successfully solves the problem; if multiple approaches are correct, we randomly select one.

The contributions of this chapter are threefold. 1) We propose HybridMind, a method that dynamically chooses between natural language reasoning and symbolic (Python code or first-order logic formulas) reasoning based on a specific reasoning problem. This "meta-selection" strategy ensures that the language model uses the most effective approach, natural language or symbolic reasoning,

depending on each problem's characteristics. 2) STaR finetuning over LLaMA-3.1-8B as a meta-selector yields a 4.4% accuracy gain on the FOLIO dataset, a logical reasoning dataset measuring complex logical reasoning capabilities of LLMs (Han et al., 2024). Using GPT-3.5-Turbo as a meta-selector yields a 10% improvement on the more challenging subset of FOLIO over GPT-4o. 3) We also identify key statistics beyond final performance and perform case study and prompt ablation study. We also provide reasons for the performance difference of FOLIO with MATH to interpret the performance improvement of our method, discussing how the difference between Python code and first-order logic formulas could affect the meta-selection results.

4.1.1 Related Work

4.1.1.0.1 Reasoning with natural language and symbolic language

Recent research has substantially advanced the logical and mathematical reasoning capabilities of LLMs by incorporating step-by-step reasoning (Kojima et al., 2023; Wang et al., 2023b; Wei et al., 2023; Zheng et al., 2024a; Zhou et al., 2023). These methods encourage models to produce explicit intermediate reasoning steps in natural language. While these methods have proven successful across various tasks, they depend entirely on language representations and can yield incoherent or unreliable solutions when confronted with logically complex problems (Han et al., 2024; Olausson et al., 2023).

Symbolic-based approaches take questions in natural language form and generate symbolic forms alone such as Python code or First-Order Logic or generate symbolic forms together with natural language reasoning steps(Chen et al., 2023; Gao et al., 2023; Gou et al., 2024; Imani et al., 2023; Zhou et al., 2024a). External code executors or symbolic engines are then leveraged for verification or deriving the final answer These methods guide the model to generate executable symbolic language forms, enabling direct checks of intermediate or final outputs. Meanwhile, SymbCoT (Xu et al., 2024) converts premises into a symbolic format and uses an LLM-based solver and verifier to validate each step. By grounding reasoning in a checkable programmatic or symbolic form, these methods can offer stronger verifiability than solely natural language–based solutions due to incorporation of an external execution engine.

4.1.1.0.2 META SELECTION WITH LLMS

Gao et al. (2024); Zhou et al. (2024c) use meta prompts to enable LLMs to choose multiple fundamental reasoning modules, such as critical thinking and step-by-step reasoning, and organize them into a clear reasoning structure that guides the model during decoding. However, these methods only adopt natural language reasoning while HybridMind exploits the of natural language reasoning and symbolic language reasoning, taking into consideration the characteristics of logical and mathematical reasoning problems

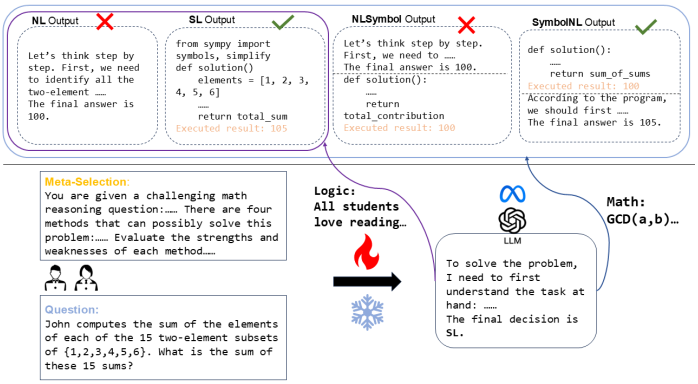


Figure 4.1: Illustration of HybridMind. HybridMind enables the model to analyze the problem and choose the most suitable approach among NL, SL, NLSymbol, and SymbolNL. We have different reasoning paths for logic and mathematical reasoning problems. In this example, HybridMind selects SL for a mathematical reasoning problem, which leads to the right solution (105).

4.2 Method

4.2.1 Approaches to reasoning

We consider four fundamental approaches involving reasoning with natural language and symbolic language. Natural Language (NL), Symbolic Language (SL), SymbolNL, and NLSymbol. NL represents that the LLM tackles the problem by generating a step-by-step breakdown of the reasoning process in natural language, guiding toward the solution. SL represents that the LLM is instructed to generate a symbolic language solution or symbolic form of the problem and it will be executed to arrive at the final solution. SymbolNL is a two-stage method. In the first stage, the LLM is prompted to write a solution in symbolic language or converts the problem into symbolic language. In the second stage, the LLM step-by-step analyzes the problem in natural language

based on the symbolic form to obtain the final answer. NLSymbol is also a two-stage method. The LLM first generates a natural language solution or hints for solving the problem. Then the output will be used to generate symbolic language for solving the problem.

The SymbolNL approach emphasizes the importance of analyzing the code's execution through natural language reasoning, allowing for corrections and insights even if the initial code has errors. In contrast, NLSymbol starts with a natural language outline to clarify the problem before writing the code, which can enhances the likelihood of successful implementation. These two formats leverage both reasoning forms to improve problem-solving effectiveness. The optimal choice among these approaches varies significantly across problem types — some problems benefit from the precision of symbolic reasoning, while others require the flexibility of natural language decomposition (Chen et al., 2023; Olausson et al., 2023; Zhou et al., 2024a).

This observation motivates HybridMind, a meta-selection framework that analyzes each problem's characteristics to determine the most suitable reasoning approach. As illustrated in Figure 4.1, before using a specific approach to derive the final answer, HybridMind uses a meta-selection module to first analyze the problem and decide which reasoning approach to apply among NL, SL, SymbolNL and NLSymbol, before using the selected approach to generate a solution.

The core intuition behind HybridMind is that not all problems benefit equally from the same reasoning approach (Zhou et al., 2024d). Some problems may require step-by-step natural language reasoning, while others are better suited for symbolic solutions. HybridMind empowers the model to dynamically adapt its strategy by selecting the most appropriate method, ensuring flexibility and maximizing performance across different problem types.

4.2.2 Choice of symbolic language

In mathematical reasoning and logical reasoning, the choice between symbolic languages such as Python and formal logical systems like first-order logic (FOL) arises from their fundamental differences in execution and purpose (Russell and Norvig, 2010). Python is a procedural language designed for computation, making it well-suited for mathematical reasoning tasks that involve numerical calculations, symbolic algebra, and algorithmic problem-solving. We include examples where NL reasoning is better and where SL reasoning is better in the appendix. With built-in support for arithmetic operations, iterative processes, and specialized libraries such as NumPy and SymPy, Python efficiently performs both exact and approximate computations (Gao et al., 2023; Zhou et al., 2024a). In contrast, FOL provides a declarative framework for logical reasoning, allowing for the formal specification of knowledge, relationships, and inferential rules (Han et al., 2024; Lyu et al., 2023; Olausson

et al., 2023). Unlike Python, which executes a sequence of computational steps, FOL is primarily used for theorem proving and rule-based inference, relying on formal logical operators, quantifiers, and axioms to establish truths within a domain. While Python can simulate logical reasoning through symbolic computation, its execution semantics is step-wise and does not allow general deduction, which may result in unnecessary steps to reach the final value (Russell and Norvig, 2010). Consequently, we select Python to be the symbolic language for math reasoning, and FOL to be the symbolic language for logical reasoning.

4.2.3 Training data generation

We present a systematic procedure for generating labeled training data for our meta-selector. We use a state-of-the-art LLM, GPT-4o to solve a logical or math reasoning problem based on NL, SL, NLSymbol, and SymbolNL. Each problem is solved in four aforementioned ways using the same LLM. Labels are automatically created by measuring answer correctness with exact match: whichever approach yields the correct solution is treated as the "label" for that problem. If multiple approaches work, one is chosen at random[1]. These "correctness-labeled" examples are then used to train a meta-selector model that can predict which solution strategy is most likely to be correct when given a

[1]Notably, having 4 labels for whether each of the method solves the problem correctly produces a more rigorous setting, but it leads to severe label imbalance which is hard to be mitigated in our initial experiments

new problem.

This training data generation method has two major advantages. 1) The data generation is fully automatic with the label directly generated with the original logical or math reasoning dataset answer labels. 2) Different prompting strategies can complement one another; the meta-selector can learn to pick the more suitable strategy for a given problem.

4.3 Experiments

Our experiments are designed to test whether large language models can reason more effectively by dynamically choosing between natural language reasoning and symbolic language reasoning (or mixing both) rather than relying on a single approach. We explore whether the model can analyze each problem first and then pick the most suitable method — natural language, symbolic language, or a combination to arrive at a solution. By doing so, we aim to see if different types of problems in math and logical reasoning benefits from different styles of reasoning, and whether a "meta-selection" strategy can yield higher accuracy. Implementation details can be found in Appendix A.3.

4.3.1 Datasets

In our experiments, we consider two challenging datasets: MATH (Hendrycks et al., 2021b) for evaluating mathematical reasoning and FOLIO (Han et al., 2024) for evaluating logical reasoning. These two datasets are widely adopted (OpenAI et al., 2024; Xu et al., 2025) and offer a range of categories and difficulty levels, enabling us to conduct various analyses. The MATH dataset comprises 7,498 training samples and 5,000 testing samples in total across 7 categories and 5 difficulty levels; and the FOLIO dataset contains 1,001 training samples and 226 testing samples spanning 2 categories. Our finetuning methods are trained on the entire training set and evaluated based on the entire testing set. Since we reported model performance by category, our experimental setting consists of a total of 9 subsets. Detailed sample counts for each category are provided in Appendix A.4.

4.3.2 Model choice

We selected several advanced LLMs as the base models for our meta-selector. For *open-source* LLMs, we employ Llama-3.1-8B-Instruct; for *proprietary* LLMs, we utilize GPT-3.5-turbo, GPT-4o, and o3-mini. [2] Finetuning with our training data reduces the bias of keeping selecting NL reasoning, but hurts the final

[2]In our initial experiments, we have also tested other models with 3-shot and 8-shot prompting, such as Qwen2.5-14B-Instruct, Qwen-32B-Coder-Instruct, and CodeLlama for meta-selection, however, these models only selected reasoning with NL for all of the tested examples, even when they are given multiple shots, which rendered them a non-optimal choice for testing with our method.

performance. We then transform these base models into meta-selectors by applying either finetuning or prompting. To ensure a fair evaluation, we use 8 shots for both NL and SL, whereas SymbolNL and NLSymbol both use 4 shots at each stage. Details of the complete prompt texts are in the Appendix.

For open-source models, we consider both zero-shot prompting approach and fine-tuning approaches including Supervised Fine-Tuning (SFT) (Liu et al., 2024; Zhao et al., 2023; Zheng et al., 2024b) and iterative finetuning strategy, STaR (Zelikman et al., 2022a), to train a meta-selector. Our fine-tuning experiments are based on LLaMA-factory (Zheng et al., 2024b), which is widely employed in LLM development (Liu et al., 2024; Zhao et al., 2023). Proprietary models are based on both the zero-shot prompting and few-shots prompting approaches to act as a meta-selector.

4.3.3 NL, SL, NLSymbol and SymbolNL

We perform meta selection over NL, SL, NLSymbol, and SymbolNL for MATH since all of the four methods achieve high performance on MATH. Furthermore, the oracle accuracy of at least one of the methods is at least 10.3% higher than that of only one other methods, showing the potential of these methods to be complementary. However, NLSymbol and SymbolNL both achieve very low performance on FOLIO. The first-order logic formulas generated by NLSymbol in the second stage tend to be unexecutable and SymbolNL and

the output generated by SymbolNL in the first stage would distract the model from generating a reasoning path. Therefore, for FOLIO we adopt two-class meta-selection over NL and SL without considering NLSymbol and SymbolNL.

4.3.4 Experiments on FOLIO

Method	`WikiLogic`	`HybLogic`	**Avg.**
Baselines			
Random	75.68	60.00	67.70
NL	80.18	60.87	70.35
SL	72.07	69.57	70.80
NL (8 MV)	<u>81.98</u>	66.09	73.89
Finetune			
2-class (base)	81.08	63.48	72.12
2-class SFT	81.08	60.87	70.35
STaR	**82.88**	66.69	74.78
Prompting (0-shot)			
GPT-3.5-Turbo	80.18	59.13	69.74
GPT-4o-mini	75.68	60.00	67.70
GPT-4o	80.18	66.96	73.45
o3-mini	80.18	62.61	71.24
Prompting (2-shot)			
GPT-4o-mini	81.08	66.09	73.45
GPT-4o	81.08	67.83	74.34
o3-mini	80.18	<u>70.43</u>	<u>75.22</u>
FOLIO (GPT-3.5-Turbo)	79.28	**73.91**	**76.55**
Upperbound			
Best-of-two (NL/SL)	92.79	83.48	88.05

Table 4.1: Model performance on FOLIO *test* set. **Bold** and <u>underlined</u> numbers indicate the best and second-best performance in each category, respectively. MV: Majority Vote. Best-of-2 (NL/SL): At least one of NL/SL is correct.

For FOLIO, NL and SL denote solving the questions based on Chain-of-Thought (Wei et al., 2023) and LINC (Olausson et al., 2023) using GPT-4o. We only sample once for both methods for efficiency.

4.3.4.0.1 Baseline and upperbound.

Although state-of-the-art LLMs achieve better performance at reasoning in natural language compared to symbolic language on both of the two subsets we tested (Table 4.1), the accuracy of at least one of the methods being correct (best-of-2) is around 17% higher than NL or SL reasoning. This score is higher than performing reasoning solely in natural language or symbolic language, demonstrating the complementary strengths of these different reasoning approaches and the potential benefits of selecting between one of them effectively.

4.3.4.0.2 Finetuning results.

Regarding the analysis of finetuning methods, STaR achieves the best performance, demonstrating the importance of finetuning strategy for enhancing models' capabilities on logical reasoning tasks. Different from STaR, we experiment with applying SFT directly to Llama-3.1-8B to select between NL and SL. These settings cover all cases where a question can be solved using either NL or SL. As shown in Table 4.1, direct SFT performs poorly on logic reasoning questions, failing to improve the base models' accuracy and predominantly selecting NL as the final solution. This result suggests that the unique design of STaR, characterized by filtering out incorrectly predicted data during training and generating rationale information, might be crucial for effectively addressing logical reasoning tasks.

4.3.4.0.3 PROMPTING RESULTS.

In addition to the above methods, we also try prompting advanced LLMs to act as a meta-selector. We generate few shot examples by prompting o3-mini with the premises and conclusion. The rationale generated effectively captures the characteristics of reasoning with NL versus SL. Notably, prompting GPT-3.5-Turbo with 2-shot examples generated by o3-mini achieves the best performance on `HybLogic`, outperforming the second-best method o3-mini by 4.9%. Furthermore, when evaluating the models by difficulty levels, HybridMind improves upon NL by 17.64% on `HybLogic`, highlighting its strong capacity for addressing complex logical reasoning scenarios. Overall, the best-performing model is GPT-3.5-Turbo, configured as a meta-selector using a few-shot prompting strategy. Moreover, HybridMind also surpassed NL/SL based on GPT-4o by 8.8% and 8.1%, respectively. This improvement is statistically significant, as validated by the Wilcoxon Rank Sum test (Virtanen et al., 2020) ($p < 0.05$) shown in Appendix A.5. A detailed analysis for these improvements is provided in section 4.4.2. Our method reduces the model's bias toward NL and utilizes the rationale generated during prompting to select the correct reasoning path for output generation.

Overall, the results demonstrate that HybridMind functions as an effective meta-selector in choosing the most suitable strategy for a given logic reasoning problem.

Method	Algebra	Counting & Probability	Geometry	Number & Theory	Intermediate & Algebra	Precalculus	Prealgebra	Avg.
				Baselines				
NL	**92.67**	78.69	**61.38**	81.48	58.91	60.07	**89.78**	76.98
SL	72.37	78.48	46.56	71.48	44.30	28.94	80.60	62.00
SymbolNL	79.87	74.05	56.99	81.48	58.69	57.51	81.63	71.43
NLSymbol	64.95	82.07	50.31	81.11	51.72	39.38	85.07	65.24
MV (4 methods)	88.29	**84.60**	57.62	85.56	59.36	53.85	**89.78**	75.98
				Finetune				
2-class (base)	72.37	78.48	46.56	71.78	44.30	28.94	80.60	62.00
2-class	**92.67**	78.69	**61.38**	81.48	58.91	60.07	**89.78**	76.98
4-class (base)	72.37	78.48	46.56	71.78	44.30	28.94	80.60	62.00
STaR (fine-tuning)	90.65	79.32	61.17	77.04	58.03	58.24	86.91	75.20
FOLIO (4-class)	92.42	79.54	**61.38**	**82.78**	60.35	**60.99**	**89.78**	**77.50**
				Prompting (0-shot)				
GPT-3.5-Turbo	84.58	78.27	58.04	75.93	55.26	47.44	85.30	71.28
GPT-4o	91.58	81.65	60.96	79.26	**60.69**	57.69	88.98	76.64
o3-mini	87.87	79.75	58.87	76.30	56.81	50.18	87.26	73.24
				Upperbound				
Best-of-2 (NL/SL)	95.79	87.55	66.81	89.63	68.77	64.10	93.57	82.84
Best-of-4	97.47	92.41	72.86	94.07	76.63	72.71	94.60	87.30

Table 4.2: Model performance on MATH *test* set. **Bold** and underlined numbers indicate the best and second-best performance in each category, respectively. MV: Majority Vote. Best-of-2 (NL/SL): At least one of NL/SL is correct. Best-of-4: At least of one of NL/SL/SymbolNL/NLSymbol is correct.

4.3.5 Experiments on MATH

4.3.5.0.1 BASELINE AND UPPERBOUND.

The results of experiments on MATH are in Table 4.2. NL baseline achieves the highest performance, while SL performs the worst among the four methods [3]. NLSymbol shows apparently better results, indicating that outlining a reasoning path prior to generating code can improve reasoning with code, particularly in models with limited code capabilities. Each of these methods falls considerably short of the best-of-4 performance, highlighting the potential of dynamically selecting different methods for different problems (Section 4.4.2).

[3]While SL achieves better performance than CoT in Gao et al. (2023), our adopted dataset and base model are both different from the ones used in their study.

4.3.5.0.2 PROMPTING RESULTS.

In the experiment of zero-shot prompting Llama-3.1-8B-Instruct model, the model performance falls closer to the average of the four approaches. This suggests that more advanced models, like o3-mini, are better at selecting the optimal approach based on the problem. Moreover, prompting the Llama-3.1-8B model to act as a meta-selector cannot surpass NL reasoning with GPT-4o, which shows the limitation of the model's capacity for selecting the correct strategy for solving mathematical reasoning problems. On the other hand, prompting advanced models as a meta selector cannot lead to better performance either, which also demonstrates the constraint of prompting in this type of question.

4.3.5.0.3 FINETUNING RESULTS.

Methods incorporating NL reasoning (NL, NLSymbol, and SymbolNL) outperform SL, emphasizing the importance of NL in mathematical reasoning. Overall, HybridMind demonstrates higher selection accuracy, effectively identifying the most appropriate approach for each problem. HybridMind also surpasses selectors with the same scale or larger scales in different categories by 0.3% at least and 2.0% at most. Our improvement is statistically significant, as validated by Wilcoxon Rank Sum test (Virtanen et al., 2020) ($p = 0.002$) shown in Appendix A.5.

HybridMind particularly works well in questions belonging to geometry,

number theory, precalculus, and prealgebra, and thus it shows the strong capacity of a fine-tuned model for solving complicated geometric or numerical problems. Questions with these types are discussed to be more challenging than other categories for general LLMs (Ahn et al., 2024; Zhang et al., 2024) to solve. A prompt and paired rationale generated by HybridMind can be found in Table 4.4b, which matches well with regular reasoning paths as a meta selector. Moreover, for finetuning experiments, we have tried different types of meta-selectors, which are summarized in Appendix A.3. The best performer for MATH is a meta-selector trained based on labels created with four methods. Our final choice also outperforms STaR with fewer shots. This could be because that training the meta-selector for math reasoning requires more examples.

Finally, while majority voting performs well on problems with higher accuracy, it struggles with more challenging problems, such as those in the precalculus category. We note that both majority voting (MV) [4] and random choosing cannot surpass our method as well as NL, especially in challenging problems in the precalculus category. This observation demonstrates the necessity of fine-tuning for improving LLMs' mathematical reasoning.

Overall, in this section, we demonstrate that a smaller-scale model can be finetuned as a meta-selector in selecting the most suitable strategy for a given mathematical reasoning problem, leading to better performance than

[4]When comparing majority voting, it's important to consider the trade-off between effectiveness and cost. Majority voting requires calling the language model to execute each of the four methods, whereas HybridMind only necessitates executing a single selected method.

state-of-the-art baselines.

4.4 Analysis

4.4.1 Analysis on FOLIO

The FOLIO dataset consists of two subsets: `WikiLogic` and `HybLogic`. `WikiLogic` presents simpler logical reasoning tasks (1 – 5 reasoning steps), whereas `HybLogic` includes more logically complex problems (5 – 8 reasoning steps). HybridMind improves model performance on `HybLogic` set by 13.04% (Table 4.1).

4.4.1.0.1 META-SELECTOR STATISTICS.

Table 4.3: Analysis on FOLIO

Strategy	WikiLogic	HybLogic	Avg.
NL	80.18	60.87	70.35
Vanilla	80.18	62.61	71.24
+ wn info	80.18	59.13	69.47
+ wn info + 2-shots	79.28	73.91	76.55

(a) Comparison of different prompting strategies on the FOLIO *test* set using GPT-3.5-Turbo.

	w/o wn info		w/ wn info	
Subset	**NL**	**SL**	**NL**	**SL**
WikiLogic	111	0	102	9
HybLogic	110	5	31	84

(b) Distribution of NL vs. SL selectedby FOLIO on `WikiLogic` and `HybLogic` w/o weakness info and w/ weakness info.

Initially, the meta selector utilized only a simple instruction prompt, providing limited performance improvement (Vanilla in Table 4.3a). Subsequent prompt engineering informs the model of the weaknesses of NL: NL reasoning can be be

prone to errors if the problem is especially logically challenging and its chain-of-though becomes long or convoluted. It also informs the model of the weakness of SL reasoning (weakness info): SL is not ideal if the premises and conclusion rely heavily on nuanced interpretations of language, or involve concepts that are hard to formalize in FOL. As shown in Table 4.3b, by informing LLM the weaknesses of both method, HybridMind now employs SL more frequently for more logically challenging `HybLogic` problems and NL for `WikiLogic` examples with more complex natural language patterns. In particular, within the logically challenging `HybLogic` set, there are 26 problems that can be solved by SL but not by NL. Of these 26 problems, HybridMind correctly routes 23 to SL, illustrating its effectiveness in detecting and handling harder reasoning questions.

4.4.1.0.2 CASE STUDY.

Table 4.4a provides an example where HybridMind accurately discovers complex logical relationships between entities in different premises and identifies "the need for precise logical deductions to reach the conclusion." It successfully selects SL. This underscores its capacity to dynamically select the most suitable reasoning approach.

Table 4.4: FOLIO output example

Input
Premises All runners who ranked in the top 1000 in the Boston Marathon finished the marathon within 3 hours. All runners from the University Running Club ranked in the top 1000 in the Boston Marathon. Some of the runners wearing T-shirts with the bulldog logo are majoring in Computer Science. Tim is a runner and if Tim finishes the Boston Marathon within 3 hours, then he is not both from University Running Club and ranked in the top 1000 in the Boston Marathon. Tim is a runner and if Tim majors in computer science, then he is either wearing a T-shirt with the bulldog logo or is a runner from the University Running Club. **Conclusion** Tim majors in computer science.
Output
... While the premises are not overly complex, the relationships between the entities involved are intertwined and require careful consideration. Given the structured nature of the premises and the need for precise logical deductions ... **Selected method: SL (Symbolic Language).**

(a) FOLIO output on FOLIO.

Input
Question
Six-digit integers will be written using each of the digits 1 through 6 exactly once per six-digit integer. How many different positive integers can be written such that all pairs of consecutive digits of each integer are relatively prime? (Note: 1 is relatively prime to all integers.)
Rationale
... given the limited number of permutations and the straightforward nature of checking the pairwise condition—the SL (Symbolic Language) method is the most effective. ...
Outputs
Selected method: SL (Symbolic Language).

(b) FOLIO output on MATH.

4.4.2 Analysis on MATH

4.4.2.0.1 HybridMind can mitigate the bias of choosing correct methods.

The bias of LLMs is shown to have negative effects towards generating correct outputs in reasoning tasks (Gupta et al., 2023; Reif and Schwartz, 2024). We noticed that small-scale models before fine-tuning, tended always to choose NL as their final solution, and thus, it is important to investigate the output distribution by choices under both mathematical reasoning and logical reasoning questions. Therefore, we investigated the frequency of choice to solve problems in the MATH and FOLIO datasets, shown in Table S3. Considering the case

of random choice, the proportion of each choice will be 25% for MATH and 50 % for FOLIO, while the outputs from our method do not follow this proportion, and our model did not choose the solution randomly but performs rigorous inference. Moreover, base models before fine-tuning had 100% NL choices in the outputs, where our generated outputs only contained 94.10% NL choices for MATH and 58.84% NL choices for FOLIO. Considering the high performance of NL in solving mathematical reasoning problems, the outputs from our method should have a higher proportion of NL choices while it can also select correct choices for other questions that NL cannot resolve, and this improvement also happened in our testing results based on the FOLIO dataset. We also tried to downsample NL in our training data and increase the proportion of SL for both problems as a preliminary attempt to mitigate existing label bias, but the final performance was reduced by 5.3% in this setting. Therefore, the improvement of HybridMind can also be interpreted by the contribution of reducing selection bias compared with other baselines.

4.4.2.0.2 CASE STUDY

Table 4.4b provides an example from the number theory category, where HybridMind selects the correct reasoning path in the problem that NL cannot address. HybridMind makes highest percentage of improvement in the problems from this category, and HybridMind will utilize the shots containing the description of problem statistics (e.g., length) as well as the most effective solution and then

make a final decision. This specific property allows us to handle complicated metathetical reasoning problems with a stronger meta-selector. We include more examples where NL reasoning is better and where SL reasoning is better in the appendix.

4.4.3 Comparing FOLIO with MATH

HybridMind produces more substantial improvement on FOLIO than on MATH, as manifested by the smaller p-value. We hypothesize this could be because that FOL formulas are designed to capture the original FOL reasoning question. In contrast, reasoning with Python code like SL for MATH follows a procedural approach that more closely resembles natural language (NL) reasoning than generating FOL formulas for FOLIO. The smaller difference between best-of-2 performance and the NL or SL approach alone indicates that these two classes are more alike, which ultimately makes the classifier more difficult to train because distinguishing between more similar classes is more challenging. For future work, we will explore auto-formalization using Isabelle-form formal specifications as SL for MATH reasoning (Yang et al., 2024b; Zhou et al., 2024a).

4.5 Conclusion and Future Work

In this chapter, we explored the integration of natural language and code-based reasoning in mathematical problem-solving using state-of-the-art LLMs. Through a series of experiments on GSM8k, MATH and AIME, we demonstrated the complementary nature of natural language and code-based reasoning. We observed that natural language remains a critical component of successful mathematical problem-solving, even when combined with code. Additionally, our proposed approach, HybridMind, dynamically selects the optimal form of reasoning from natural language, code, or a combination of both based on the problem type. HybridMind outperformed baseline methods by effectively leveraging the strengths of both natural language and code reasoning.

Chapter 5

Evaluating LLM Agents' Capabilities of Reasoning like Judges

5.1 Introduction

As discussed in chapter 2, chapter 3 and chapter 4, large language models (LLMs) have achieved impressive results across mathematical and logical reasoning tasks (Anthropic, 2024; DeepMind, 2025; OpenAI, 2025a,b). However, legal reasoning under the U.S. legal system poses distinct challenges. Unlike math or factual question answering (QA), legal problems often lack a single correct answer. Expert disagreement is common, particularly in cases that

reach litigation or appellate review (Post, 2024). Binary accuracy metrics used in many legal QA benchmarks (Guha et al., 2023; Hendrycks et al., 2021a; Koreeda and Manning, 2021) fail to capture this complexity.

Logical reasoning is foundational to legal reasoning: doctrines often resemble structured rules, judicial opinions frequently apply conditional logic to facts, and lawyers must ensure that conclusions follow coherently from premises. The work in chapter 2 and P-FOLIO 3 therefore provides a natural starting point for studying legal reasoning, as it establishes the importance of explicit structure and transparent inference processes. However, legal reasoning extends beyond formal logic. Legal reasoning requires constructing coherent arguments grounded in precedent, applying doctrine to nuanced facts, and often addressing competing values or interpretations (Schauer, 2009). Evaluating such reasoning involves more than labeling the "correct" party: it must assess how well a model engages with case law, distinguishes counterarguments, and weaves a persuasive narrative (Dworkin, 1986; Hanks et al., 1994).

This task is further complicated by ultra-long context requirements: inputs may span tens of thousands of tokens across statutes, precedents, and factual records, while outputs are also often long-form, exceeding 5,000 tokens. Unlike in math or logic tasks, increasing token limits at test time is computationally prohibitive and insufficient (Zheng et al., 2025). Consequently, strategies such as test-time scaling, which have proven effective in mathematical and logical

reasoning tasks by increasing the number of input or output tokens (Muennighoff et al., 2025), may not be directly applicable. In the legal domain, simply scaling token length is computationally prohibitive and can lead to exponential growth in both the size of the input context and the complexity of the output.

To address this gap, we introduce CourtReasoner, a benchmark built from real U.S. court opinions drawn from `https://case.law/caselaw/`, a database of U.S. court opinions. Rather than focusing on multiple-choice answers or bar exams, we evaluate full-length legal reasoning. Each document is manually segmented into background and facts and reasoning sections, and models must construct the entire judicial reasoning section. A judicial reasoning section includes breaking the issue into doctrinal building blocks, bringing in one precedent per element, synthesizing the sub-conclusions into the holding, showing why an outlier precedent does not control and finally closing with a logically connected conclusion. Each of the components can be repeated and iterated through multiple times to draw the final conclusion. We chose the U.S. appellate system because it offers a suitable and well-documented environment for this specific task, supported by a large, publicly accessible corpus of judicial opinions necessary for building a high-fidelity benchmark.

LLM agents have access to online documents. Since all the cases we can access are online, LLMs can simply search for the exact case based on the background/facts section we feed to it in the input. Therefore, we design

three adversarial settings in order to evaluate whether an LLM is capable of formulating novel arguments rather than copying online texts. In the adversarial settings, we ask annotators to remove facts from original documents to present an alternative scenario unseen by models to assess their ability to reason without recalling information seen in training or internet searching. Because complex disputes often involve several independent doctrines (e.g., standing, immunity, merits), these steps may be iterated: the opinion cycles through the sequence for each doctrine, then integrates those partial holdings into a final judgment.

We work with experienced legal annotators, who are well-experienced lawyers and paralegals practicing law in the United States, to evaluate reasoning quality in three key dimensions listed below that are deemed critical by legal professionals.

1) *citation relevance*, i.e., is precedent correctly cited and applied?
2) *constraint extraction*, i.e., are the conditions needed for the cited rule correctly identified?
3) *argument validity*, i.e., do the conclusions logically follow from the cited law and facts?

Our human annotators carefully evaluate the outputs of several proprietary agents, namely `GPT-4o Deep Research` (OpenAI, 2025b), `OpenAI o3` with search (OpenAI, 2025a), `Gemini Pro Deep Research` (Google, 2024). In

addition, we included an open-source baseline, namely `Open Deep Research`[1], our custom legal `Open-RAG` pipeline.

Our findings shed light on both the impressive capabilities of modern LLM agents which can mimic complex reasoning chains to a degree, but still falls short in citing relevant cases, extract correct constraints from precedent cases, and producing valid arguments for why constraints are satisfied.

Although we were able to annotate process-level reasoning outputs for FOLIO in P-FOLIO, extending this methodology to the legal domain presents a far greater challenge. Annotating legal reasoning processes requires the expertise of trained legal professionals—lawyers and paralegals who can interpret doctrines, precedents, and factual nuances with precision. This work is significantly more time-consuming and cost-intensive than annotating logical proofs, both because of the domain's inherent complexity and because legal reasoning often involves competing interpretations rather than a single ground truth. As a result, while P-FOLIO demonstrates the feasibility of large-scale, process-level supervision for logical reasoning, applying the same approach to law necessitates new strategies, motivating the design of CourtReasoner as a meta-evalution benchmark focused on evaluating reasoning quality without requiring exhaustive manual proof annotation. CourtReasoner is intended to provide insights into the capababilities of LLMs as evaluators of legal reasoning. We

[1]https://github.com/langchain-ai/open_deep_research

found that producing relevant citations, constraints and valid arguments for the side ruled against is significantly harder than for the side supported by the original court opinion for LLM agents. While LLM is capable of producing mostly relevant citations for 60% of the time when arguing for the same side taken by an online case, they can only produce mostly relevant citations for 30% of the time when arguing for a different side although the different side is completely arguable from a lawyer's perspective.

We observe that of the LLM-based evaluators (e.g., o3, Claude-3.7, Gemini-Pro) achieve consistently strong correlation with human annotators across all perturbation conditions. We interpret this as a critical finding rather than a shortcoming of our study: it highlights the current fragility and inconsistency of using LLMs to evaluate legal reasoning, especially under stress-testing of evaluating 4k - 10k output tokens.

We also observe that `o3` achieves high or moderate correlation with human scoring under a few settings. `Claude-3.7` (Anthropic, 2025) and `OpenAI o3` evaluator achieve moderate correlation with human scoring under few settings although none of the LLM evaluators we evaluated achieve moderate or stonger correlation with human annotation across all task settings for all the three meta-evaluation aspects, including `OpenAI o3`, `Claude-3.7`, `Gemini-Pro` (Google DeepMind, 2025), `Gemini-Flash` (DeepMind, 2025) and `Qwen-2.5-72B` (Yang et al., 2024a).

Our contributions are as follows.

1. We propose CourtReasoner, a novel benchmark for measuring complex realistic legal reasoning. The benchmark consists of both question answer pairs and meta-evaluations. Complex legal reasoning is distinct from previous complex LLM reasoning tasks as both sides can be true and the model output tokens are extremely long. The inter-annotator agreements of the meta-evaluations are 75% or higher among all five task settings.
2. We identify three quantifiable aspects that are indispensable for judicial reasoning quality, citation relevance, constraint extraction and argument validity. We construct a scoring rubric or automatically scoring LLM agents' outputs in three aspects and measure state-of-the-art LLM agents' capability to evaluate these three aspects respectively.
3. We perform qualitative analysis to shed light on how LLMs perform on moral/emotional reasoning these three aspects with selected cases where these aspects are particularly relevant.

5.2 Related Work

5.2.1 LLM reasoning benchmarks in the legal domain

GPT-4 excelled at the Uniform Bar Exam (OpenAI, 2023), demonstrating LLMs' growing legal knowledge. GPT-4 could reason through statutory problems, though imperfectly (Blair-Stanek et al., 2023). Similarly, it has been shown that LLMs can reach high accuracy on tax law problems with proper prompting, though still below expert levels (Nay et al., 2024).

Our benchmark differs from prior work by focusing on court opinion documents rather than question-answer accuracy (Guha et al., 2023; Han et al., 2025a; Hendrycks et al., 2021a; Holzenberger and Van Durme, 2021; Koreeda and Manning, 2021; Lippi et al., 2019; Ravichander et al., 2019; Wang et al., 2023a; Wilson et al., 2016; Zheng et al., 2021; Zimmeck et al., 2019) or bar exam performance (Zheng et al., 2025). This allows us to better capture the practical reasoning found in legal writing. Evaluating practical reasoning is essential because expert consensus is rare on issues contentious enough to reach litigation. Split opinions are common among Supreme Court justices (Post, 2024), making benchmarks focused solely on widely-agreed legal tasks an incomplete measure of LLM reasoning. Instead, we assess how well LLMs can produce reasoning

that supports opposing legal conclusions.

Thus, evaluation must go beyond accuracy to assess citation relevance, constraint extraction, and argument validity.

5.2.2 Legal reasoning with LLMs

Previous work identified that judicial reasons generated by LLMs contain legal "rhetoric" rather than legal "reasons", allowing judgments to become works of persuasion rather than of deduction (Re, 2023). Furthermore, while LLMs hold considerable promise in legal analysis, their responses are highly susceptible to changes in prompts and the framing of counterarguments. Several scholars have also noted that LLM judicial reasons may hallucinate both law and facts (Schwarcz et al., 2025). More broadly, previous literature identified that LLMs provide sufficient reasons in response to simple legal questions, but have organizational problems, overlook legal issues, and ignore exceptions and rule variations in more complex legal questions (Choi and Schwarcz, 2025).Taken together, these concerns suggest that while LLMs can mimic the surface structure of reasoning, they may fall short of the coherence, depth, and precision required for judicial analysis in complex cases. In our work, we show that the most advanced LLMs, such as `OpenAI o3` and `Gemini Flash` are very good at generating a coherent and easy-to-follow logical flow of reasoning and the premises are built up coherently. In some cases, LLM produces very

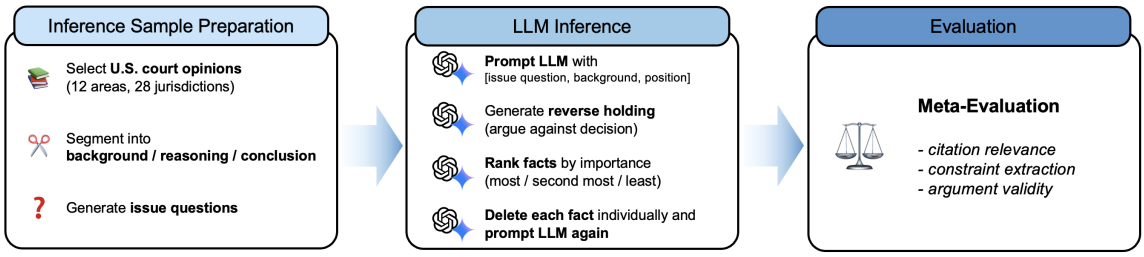


Figure 5.1: Overview of the CourtReasoner pipeline for judicial reasoning evaluation of LLM agents.

thorough explanations that can help a lawyer originally not an expert in a certain domain of law although in some cases, it can be a lot more thorough than needed to be. Previous LLM agents may fall short in cases that involve not just applying the facts and rules but also addressing ethical and interpretive subtleties (Dawson, 2024).

5.3 CourtReasoner construction

Here we describe the CourtReasoner workflow, which is also illustrated in Figure 5.1.

5.3.1 Overall input and output annotation

Annotators begin by selecting U.S. case-law opinions from `https://case.law` of diverse areas of law, jurisdictions with preference for opinions whose analytical sections vary in length and structural complexity.

5.3.1.0.1 Issue construction

In appellate case law, the initial paragraph of each subsection generally introduces the legal question at issue, while the subsequent paragraphs elaborate on the court's reasoning by drawing upon precedent, applying statutory interpretation, and evaluating the specific facts of the case. Leveraging this structure, we prompt a large language model (LLM) to generate a legal question from the initial paragraph of each subsection. The remaining paragraphs in that subsection are treated as the corresponding reasoning or "solution". This formulation naturally yields a dataset of question–answer pairs grounded in authentic judicial reasoning.

From the chosen opinion, annotators copy only the first paragraph (or at most the first two if both state the court's position) of one analytical sub-section. Using `GPT-4o` they convert this narrative into a single well-formed legal issue whose answer is supplied by the remainder of the analysis section. The issue is manually reviewed and, if necessary, manually edited for correctness.

5.3.1.0.2 Model output generation

The input to be model will then be the concatenation of (1) the generated issue questions, (2) the introduction, background and facts sections, and (3) the position the model should take.

We use `Gemini Pro Deep Research`, `GPT-4o` with Deep Research enabled,

Table 5.1: Distribution of question lengths.

Dataset setting	LLM input length	
	word count	sentence count
Default	1758 ± 947	71 ± 48
Deleting most	1565 ± 1261	71 ± 60
Deleting 2nd most	1625 ± 1345	72 ± 61
Deleting least	1699 ± 1315	71 ± 64

Table 5.2: Distribution of model output lengths.

Model	LLM output length	
	word count	sentence count
`Gemini Deep Research`	5023 ± 2164	184 ± 76
`GPT-4o Deep Research`	2614 ± 1411	91 ± 44
`o3-search`	1060 ± 179	46 ± 9
`open-deep-research`	1635 ± 240	60 ± 17
`Open-RAG`	2094 ± 1466	96 ± 72

`o3` with search enabled, and `open-deep-research` from langchain[2] as the base models for generating outputs. An example of complete `GPT-4o Deep Research` model input and output is included in Appendix A.9.

We also developed `Legal OpenRAG`, a legal retrieval-augmented generation (RAG) framework comprising three key components: (1) a retriever built using the `E5-base` embedding (Wang et al., 2022) to index a corpus of 6.7 million case law documents that we crawled from CaseLaw, (2) a reranker powered by `GPT-4o` to refine the top retrieved results, and (3) a generator based on the `o3` model, which produces the final output using the top 10 reranked precedent cases and the user's query.

5.3.1.0.3 STRUCTURED ERROR ANALYSIS

Annotators then assess the model's capability of generating court's side or alternate-side answer, paragraph-by-paragraph and citation-by-citation. "Alternate-side" refers to the opposite party, i.e., the side not favored in the court's decision. For each cited authority they record:

A. relevance (high, remote, or irrelevant),

[2]https://github.com/langchain-ai/open_deep_research

B. whether all legally operative constraints (typically larger than 3) are correctly extracted,

C. whether the reasoning convincingly shows those constraints are satisfied.

If a citation fails at level A, we skip levels B and C for that citation; similarly, if any constraint fails at level B, we skip level C for that constraint. Additional flags are assigned for faulty logical steps, off-point answers, or weak "hand-wavy" reasoning. Citations that markedly streamline the argumentation are tagged as "good" to highlight persuasive precedent. The full scoring rubric is listed in Appendix A.8.

We collected feedback from the annotators that language models are able to generate a coherent overall analysis from preliminary inspection of the evaluated LLM agents' output. We therefore did not include this aspect in the evaluation rubrics. This was further confirmed by annotators during post-annotation discussions.

5.3.2 Adversarial complex legal reasoning task construction

The annotation workflow is designed to probe whether a tool-augmented `GPT-4o` can truly reason about law rather than retrieve and paraphrase existing opinions. We therefore begin with a fixed set of court opinions whose introductory

material, background facts, and questions have already been studied. Anchoring the task on pre-selected, question-answerable cases eliminates noise from case selection and lets us focus on the model's analytic competence.

5.3.2.0.1 RANKING FACT IMPORTANCE

In Step 2 the annotator manually ranks every fact in the introduction and background according to its influence on the question at issue, as revealed by the court's original answer paragraphs. The two most pivotal facts and the single least important fact are flagged. This ranking phase is critical: by explicitly identifying what matters most and what barely matters, we can later remove those facts one at a time to stress-test the model's doctrinal chain. The idea is simple but powerful: if a model truly grasps why the court ruled as it did, deleting a key premise should force it to construct a different (yet still valid) line of arguments, whereas deleting a trivial premise should leave its reasoning largely intact.

5.3.2.0.2 MODEL OUTPUT GENERATION

Step 3 operationalizes that test. For each of the three targeted facts (most important, second most important, and least important), the annotator deletes only that fact with all other facts and background information untouched and then prompts `GPT-4o Deep Research` to draft a fresh analysis that reaches

Table 5.3: Performance of LLMs evaluated by human annotators, reported in mean.

	Gemini Deep Research			GPT-4o Deep Research					o3-search		Open DR		Open-RAG		
	~~most~~	~~2nd~~	~~least~~	Def.	~~most~~	~~2nd~~	~~least~~	Opp.	~~most~~	~~2nd~~	~~most~~	~~2nd~~	Def.	~~most~~	~~2nd~~
CT	2.07	2.28	2.80	3.33	3.00	3.06	3.13	2.42	2.92	2.89	2.10	2.70	3.00	2.80	2.90
CS	2.30	2.55	2.40	2.96	2.65	2.69	2.75	2.17	2.85	2.58	2.60	3.20	3.10	2.60	3.00
AG	2.17	2.17	2.60	3.08	2.65	2.63	2.69	2.25	2.58	2.46	2.40	2.70	3.00	2.50	2.50

the same side as the original court while explicitly avoiding the deleted premise and any verbatim reuse of the court's methodology. The prompt instructs the model to rely solely on the facts that remain, forbids citation to the source case or to Wikipedia, and demands in-text citations to precedents that pre-date the opinion.

5.3.2.0.3 ANNOTATION AND LEGAL EXPERTISE

The evaluation of model outputs was conducted by trained legal professionals, including attorneys and paralegals, to ensure accurate and domain-appropriate assessments. Annotators were hired through the Upwork platform[3] and fairly compensated for their time and expertise ($50 per hr for lawyers and $25 - $40 per hour for paralegals). While inter-annotator agreement was high, we acknowledge the inherently interpretive nature of legal analysis and include transparency regarding evaluation protocols and scoring rubrics to mitigate subjectivity.

Table 5.4 reports inter-annotator agreement (Spearman correlation) between

[3]https://www.upwork.com

Table 5.4: Spearman correlation (ρ) between two annotators across evaluation conditions. `o3-search` shows the strongest correlation, followed by `open-deep-research`.

	Gemini Deep Research			GPT-4o Deep Research					o3-search		Open DR		Open-RAG		
	~~most~~	~~2nd~~	~~least~~	Def.	~~most~~	~~2nd~~	~~least~~	Opp.	~~most~~	~~2nd~~	~~most~~	~~2nd~~	Def.	~~most~~	~~2nd~~
CT	67	59	65	67	64	59	64	66	74	74	72	72	64	64	64
CS	65	62	67	66	64	59	64	66	74	74	70	70	62	62	62
AG	59	65	57	61	59	53	59	60	71	71	69	69	58	58	58

two annotators across systems and evaluation categories. inter-annotator agreement varies across both systems and evaluation categories. Agreement on constraint extraction is generally highest, since constraints in prompts are usually explicit (e.g., specific requirements, conditions, or limits), making them relatively straightforward for annotators to identify consistently. Citation relevance shows more moderate agreement, as judgments about whether a source is sufficiently relevant can be somewhat subjective: one annotator may view a citation as adequately supportive while another may find it tangential. The lowest agreement appears for argument validity, which requires assessing the soundness of reasoning in model outputs. Because such judgments are more interpretive and dependent on annotators' background knowledge, disagreements are more frequent in this category.

Table 5.5: Performance of LLMs evaluated by human annotators, reported in percentage. Higher values are colored in darker shades of red. Overall, the removal of key facts results in a noticeable decline in performance.

	Gemini Deep Research			GPT-4o Deep Research					o3-search		Open-DR		Open-RAG		
	~~most~~	~~2nd~~	~~least~~	Def.	~~most~~	~~2nd~~	~~least~~	Opp.	~~most~~	~~2nd~~	~~most~~	~~2nd~~	Def.	~~most~~	~~2nd~~
CT = 0	13	10	20	0	0	0	0	17	0	0	0	0	0	0	0
CT = 1	13	7	0	0	6	0	6	8	4	4	30	0	0	0	0
CT = 2	30	41	20	4	24	25	12	17	12	19	30	40	10	20	20
CT = 3	40	28	0	58	35	44	44	33	73	62	40	50	80	80	70
CT = 4	3	14	60	38	35	31	38	25	12	15	0	10	10	0	10
CS = 0	7	3	40	0	0	0	0	17	0	0	0	0	0	0	0
CS = 1	10	14	0	4	6	0	6	17	0	4	20	0	0	0	0
CS = 2	47	28	0	8	47	56	44	0	42	46	30	20	30	60	30
CS = 3	20	34	0	75	24	19	19	67	31	38	20	40	40	20	40
CS = 4	17	21	60	12	24	25	31	0	27	12	30	40	40	20	30
AG = 0	3	3	20	0	0	0	0	17	0	0	0	0	0	0	0
AG = 1	27	34	20	0	0	12	19	8	4	4	20	0	0	0	0
AG = 2	37	21	0	12	53	44	25	8	42	46	30	40	30	60	60
AG = 3	17	24	0	67	29	12	25	67	46	50	40	50	40	30	30
AG = 4	17	17	60	21	18	31	31	0	8	0	10	10	30	10	10

5.3.3 Dataset statistics

5.3.3.0.1 TASK SETTINGS AND MODELS

We have collected 292 expert-annotated meta-evaluation examples in total. The same 50 seed cases are used to generate all the model outputs throughout the annotations. We include five distinct model variants in our evaluation:

1. **`Gemini Deep Research`** was evaluated under three adversarial conditions: deleting the most important fact or the second most important, with 30 examples each, and 5 examples with deleting the least important fact.
2. **`GPT-4o Deep Research`** was tested under five conditions: original same-side reasoning (50 examples), asking the model to argue from a different side (50 examples), and three citation-deletion adversaries (15 examples each), totaling 145 examples,
3. **`o3-search`** was tested under two adversarial deletion conditions, with 26

Table 5.6: Average per-citation relevance score (%) for Gemini Deep Research (Gemini DR) and Open-Deep-Research (Open DR).

	Gemini DR		Open DR	
	~~most~~	~~2nd most~~	~~most~~	~~2nd most~~
Per-Citation	2.30	2.80	2.41	3.30

examples each,

4. **`open-deep-research`** (LangChain-based) underwent evaluation on the two adversarial variants, with 10 examples per condition,
5. **`Open-RAG`** underwent evaluation on the original case and the two adversarial variants, with 10 examples per condition.

The total dataset comprises 292 examples spanning diverse reasoning conditions and model variants. This evaluation assesses the robustness of legal reasoning under perturbation and the consistency of output quality across LLM agents with different retrieval and generation strategies.

5.3.3.0.2 INPUT/OUTPUT LENGTH

Table 5.1 summarizes the distribution of input lengths, measured in both word count and sentence count. In the default setting, questions average 1,758 words and 71 sentences. When a fact is deleted, the input length decreases slightly.

Table 5.2 presents the distribution of output lengths generated by each model, measured in both word count and sentence count. Among the models evaluated, `Gemini Deep Research` produces the longest outputs by a significant margin,

averaging 5,023 words and 184 sentences, indicative of its long-form extensive reasoning style. `GPT-4o` Deep Research outputs are substantially shorter, averaging 2,614 words and 91 sentences, suggesting a more concise but still elaborate reasoning process. `Open-RAG` and `open-deep-research` fall in a middle range, with outputs averaging 2,094 and 1,635 words respectively. These outputs tend to be less verbose than `GPT-4o`'s but still maintain substantive argumentation structure. `o3-search`, by contrast, produces the shortest outputs on average, with only 1,060 words and 46 sentences, reflecting either a minimalistic style or possible limitations in compositional depth. This variation in output length underscores significant differences in the number reasoning tokens across models.

5.3.4 Meta-evaluation

Annotating legal reasoning processes by legal professionals is expensive, and therefore we provide a meta-evaluation benchmark to provide insights into the capabilities of LLMs in legal reasoning.

Table 5.3 reports the mean human evaluation scores of each model's legal reasoning performance across three key dimensions: citation relevance (CT), constraint extraction (CS), and argument validity (AG). Each model was tested under a range of perturbation settings, including deletion of the most, second most, or least important fact, as well as an "opposite side" framing and a default

unperturbed condition.

`GPT-4o Deep Research` demonstrates the strongest overall performance in the default setting, with average scores of 3.33 for citation relevance, 2.96 for constraint extraction, and 3.08 for argument validity. Its relative robustness across perturbed settings suggests that `GPT-4o` benefits from both strong citation grounding and stable logical reasoning. By contrast, `Gemini Deep Research`, despite producing substantially longer outputs, shows weaker performance particularly in citation relevance (as low as 2.07) and greater variability across perturbations. This may indicate a verbosity bias, where the model generates superficially rich but less precise content, leading to degraded grounding in precedent and fuzzier constraint extraction.

`Open-RAG` performs competitively in the default condition, particularly on constraint extraction (3.10) and argument validity (3.00). This suggests that reasoning could be more effective when it is disentangled from the retrieval and reranking process and that retrieving from a database consisting solely of case law is effective for a legal research framework. However, performance slightly degrades under fact-deletion perturbations, possibly due to potential memorization of the default setting. Both `open-deep-research` and `o3-search` show moderate to low performance across all categories. These models may suffer from limited retrieval precision or insufficient output supervision, which leads to incomplete or loosely connected legal reasoning steps.

Table 5.5 displays the distribution of human evaluation scores for each model across three dimensions: citation relevance, constraint extraction, and argument validity. These values are reported as percentages over discrete rating levels (0 to 4). Darker shades represent higher values, making trends visually salient. The table reveals clear patterns: performance generally degrades when key facts are removed from the input, particularly when the most- or second-most-important facts facts are omitted. We also found that producing relevant citations, constraints, and valid arguments for the side ruled against are significantly harder than for the side supported by the original court opinion for LLM agents.

The degradation when critical facts are missed is most evident for `Gemini Deep Research`, which shows a substantial increase in low (0-1) scores and a drop in high (3-4) scores across all dimensions under deletion conditions. For example, when the most important fact is deleted, 13% of `Gemini` outputs are rated 0 in citation relevance, and only 3% receive the top rating of 4. `GPT-4o Deep Research` demonstrates greater resilience under perturbation. Even when key facts are removed, a majority of its outputs maintain high ratings (scores of 3 or 4), suggesting more robust internal reasoning and better generalization. This may be attributed to its superior retrieval reranking mechanism and generation strategies and it remains capable of generating reasonable legal reasoning under the adversarial scenarios. Nevertheless,

performance still drops in the "opposite-side" condition, especially in citation relevance and constraint extraction, indicating its capability to reason from the other side of the case is still limited. `Open-RAG` performs competitively in the default condition, with 80% of outputs rated 3 or 4 in citation relevance. Notably, `Open-RAG` has no scores of 0 or 1 in both the argument validity and constraint extraction evaluations. This suggests that restricting the retrieval space to be entirely based on case law and disentangling the retrieval and reasoning processes can potentially help an LLM agent perform more robustly on complex legal reasoning. However, like other models, its scores decline as critical facts are removed. `o3-search` and `open-deep-research` exhibit lower robustness, with relatively flat distributions across scores and a tendency to cluster around mid-range ratings, potentially reflecting limited citation grounding or weaker constraint modeling.

Table 5.6 presents the average per-citation relevance scores for `Gemini Deep Research` and `Open Deep Research` under conditions where either the most-important or second-most-important fact was removed. Relevance is assessed on a $0-4$ scale, with higher values indicating stronger citation support for the claim. `Gemini Deep Research` obtained mean scores of 2.30 (most) and 2.80 (second-most), whereas Open Deep Research achieved 2.41 and 3.30, respectively. These results align with the overall citation relevance statistics reported in Table 5.3.

These patterns suggest that while larger or better-tuned models can partially robustly conduct complex legal reasoning even under adversarial scenarios, the deletion of high-salience facts introduces brittleness across systems, highlighting the limitations of existing LLM agents.

5.4 Experiments with LLM Evaluators

5.4.1 LLM as an evaluator

To understand how well different large language models can grade complex judicial reasoning chains, we instantiate four state-of-the-art models as evaluators: `OpenAI o3` (OpenAI, 2025a), `Claude-3.7` (Anthropic, 2025), `Gemini-Pro` (Google DeepMind, 2025), `Gemini-Flash` (DeepMind, 2025) and `Qwen2.5-72B` (Yang et al., 2024a). Each evaluator receives (1) the question containing the position the model is asked to take and the background and introduction sections of the case, (2) the candidate analysis, and (3) the three-layer rubric for *citation relevance*, *constraint extraction*, and *argument validity* that human annotators already follow.

5.4.1.0.1 Correlation with human evaluation

We computed the Spearman correlations (Table 5.7) between LLM-based graders and human annotations across citation relevance (CT), constraint

Table 5.7: Meta-evaluation on citation relevance (CT), constraint extraction (CS), and argument validity (AG) across various perturbation settings. **Spearman correlations** are reported, in percentage. Values are colored by the blue-to-red `coolwarm` colormap.

	o3			**Claude**			**G-Pro**			**G-Flash**			**Qwen2.5**		
	CT	CS	AG	CT	CS	AG	CT	CS	AG	CT	CS	AG	CT	CS	AG
`GPT-4o Deep Research`															
default	30	21	20	30	16	0	10	21	5	14	0	4	6	23	29
~~most important~~	40	51	0	48	54	-3	4	-14	-5	3	5	-10	43	19	22
~~2nd most important~~	30	10	41	19	6	5	44	0	23	3	32	13	0	41	28
~~least important~~	20	-30	0	17	6	-12	17	13	34	-14	-9	20	13	-9	4
opposite	35	15	15	56	75	81	5	37	28	38	38	28	45	52	52
`Gemini Deep Research`															
~~most important~~	52	54	49	22	36	33	49	49	6	42	47	35	10	25	19
~~2nd most important~~	18	10	13	29	27	28	44	35	12	7	30	32	6	-22	9
~~2nd most important~~	97	0	-32	29	27	28	44	35	12	7	30	32	6	-22	9
`o3-search`															
~~most important~~	32	25	-7	36	29	14	-2	-2	-10	0	0	0	0	21	0
~~2nd most important~~	6	47	6	6	-8	-4	3	7	13	-3	-12	-28	19	-35	-1
`open-deep-research`															
~~most important~~	-5	0	10	34	26	47	26	42	74	8	18	57	38	26	37
~~2nd most important~~	0	21	34	47	14	11	6	18	2	6	18	52	-11	0	22
`OpenRAG`															
default	0	85	0	-56	39	6	0	64	28	0	5	29	0	0	0
~~most important~~	0	0	27	-25	-16	-27	0	50	13	-17	0	-26	0	0	0
~~2nd most important~~	0	38	0	42	6	0	0	43	9	0	24	-7	-43	-60	0

extraction (CS), and argument validity (AG). Results are reported under various perturbation settings to test evaluation robustness. Claude consistently achieves the strongest alignment with human ratings, particularly under adversarial conditions. For `GPT-4o`'s "opposite side" setting, Claude reaches Spearman correlations as high as 75 (CS) and 81 (AG), indicating strong sensitivity to reasoning flips and constraint violations. `Qwen2.5-72B` also performs reliably across multiple models and conditions, especially on `GPT-4o` and `Gemini`

outputs. However, it struggles with `OpenRAG` and `open-deep-research` outputs, where correlations often drop to zero or turn negative, highlighting challenges in grading retrieval-augmented generations. `Gemini Pro` and `Gemini Flash` show inconsistent performance: they perform well on their own outputs but degrade across others, suggesting limited generalization in evaluation logic. `o3`, while occasionally correlated with its own or `Gemini` outputs, performs poorly on cross-model evaluation, especially under fact-deletion perturbations. Several negative correlations indicate misalignment with human judgment in degraded reasoning scenarios. Overall, these findings show that LLM-based evaluators vary significantly in reliability across models and settings. While `Claude` and `Qwen2.5` show promise, robust meta-evaluation in legal reasoning likely requires grader ensembles or model-aware scoring mechanisms to maintain consistency under perturbation.

5.5 Conclusion

We introduce CourtReasoner, a benchmark for evaluating LLMs' ability to generate full judicial reasoning in U.S. court cases. Unlike prior legal QA datasets, our benchmark emphasizes structured, precedent-based analysis and tests models under adversarial conditions. The results show that even advanced LLMs struggle with citation relevance, constraint extraction, and argument

validity, especially when key facts are removed or when reasoning from the opposite side. We also find that LLMs are inconsistent as evaluators of legal reasoning. Our benchmark highlights the challenges of legal reasoning and provides resources for advancing reliable legal AI.

5.6 Limitations

While CourtReasoner offers a comprehensive benchmark for evaluating LLMs' ability to conduct structured legal reasoning, several limitations merit discussion.

First, the benchmark is built on U.S. appellate court opinions, which may limit the generalizability of findings to trial court decisions, international legal systems, or non-adversarial legal contexts. Judicial reasoning practices differ across legal traditions, and the reasoning structure assessed here, such as doctrinal decomposition, precedent synthesis, and final holding, may not fully reflect legal reasoning in other jurisdictions. Our 319 annotated examples, while deeply analyzed, may not represent the full range of legal doctrines or complexities. Due to the high cost of expert legal annotation (minimum $40-50$ / hr and a total of 300 hrs), we prioritized depth over breadth in this release, ensuring each example spans multiple adversarial settings and reasoning dimensions. Nonetheless, larger-scale annotation or cross-dataset validation

would strengthen the generality and robustness of the findings. We plan to release detailed annotation guidelines to enable community-driven expansion, and we welcome future collaborations to scale the benchmark.

Second, although we simulate adversarial scenarios by deleting critical facts and reversing positions, these perturbations are designed and constrained manually. They do not exhaustively capture the space of real-world fact patterns or the full adversarial complexity of litigation. Furthermore, LLMs might still memorize or approximate aspects of the original cases despite our efforts to mask them, especially if such cases exist in their training corpora. Our adversarial setup deliberately removes key facts, prohibits citation of the source case, and restricts precedents to those predating the decision. The original case remains publicly available, so partial reuse or adaptation is still possible. These constraints are intended to stress-test models' reasoning rather than to fully eliminate contamination, and retrieval leakage cannot be entirely ruled out.

Finally, our evaluation depends on expert annotations, which, while necessary for legal reasoning, are costly and thus limit scale. The 319-instance meta-evaluation set, while detailed, may not cover the full spectrum of legal doctrines, jurisdictions, or case complexities.

5.7 Ethical considerations

This work raises several important ethical considerations related to the responsible use and evaluation of large language models (LLMs) in legal contexts.

5.7.0.0.1 Use of public legal data

All case law used in the construction of the CourtReasoner benchmark is sourced from publicly available U.S. court opinions from https://case.law/caselaw/, a database of U.S. court opinions. These legal documents are in the public domain and do not contain personal identifying information beyond what is already made public in judicial decisions. No additional private or confidential legal materials were accessed.

5.7.0.0.2 Avoiding harm in legal automation

Our results demonstrate that even frontier LLMs often generate legally invalid arguments, hallucinate citations, or misrepresent case facts. If adopted uncritically in legal practice, such behavior could lead to serious harms, including misinformation, unjust outcomes, and erosion of public trust in the legal system. These risks illustrate the broader dangers of overreliance on opaque or hallucinating systems in high-stakes domains and **reinforce the need to keep human legal professionals firmly in the loop**. Against this

backdrop, COURTREASONER is introduced not as a tool for deployment, but as a research and evaluation framework—intended to advance understanding of LLM reasoning in legal contexts while safeguarding against misuse.

5.7.0.0.3 Responsible open release

We plan to publicly release all benchmark data, prompts, annotations, and evaluation code to promote transparency and reproducibility. We also include clear usage disclaimers that the benchmark is not intended to certify any LLM for legal use and should be used solely for research and evaluation purposes.

By foregrounding these concerns, we aim to foster responsible AI research in the legal domain and promote careful consideration of the limitations and risks of automated legal reasoning.

Chapter 6

Conclusion

6.1 Summary of Findings

This thesis transforms structured reasoning — a key component of human intelligence — into something that machines can learn, be evaluated on, and apply to high-stakes domains like law. Through four major studies—FOLIO, P-FOLIO, HYBRIDMIND, and CourtReasoner—the work systematically examines how reasoning manifests across different levels of abstraction, from formal logic to legal argumentation. Each project advances a distinct dimension of the same underlying goal: to understand how models can reason systematically, explain their inferential processes, and apply reasoning principles to real-world domains.

FOLIO and P-FOLIO together lay the foundation for this dissertation's

study of structured reasoning. FOLIO introduces a large-scale benchmark that aligns natural language with first-order logic, allowing reasoning to be evaluated on its formal validity rather than linguistic plausibility. It reveals that even advanced language models can produce fluent and seemingly coherent answers while failing to preserve logical consistency—exposing the gap between surface understanding and true inferential competence. Building on this, P-FOLIO reframes reasoning as a process rather than a mere outcome. By providing human-written reasoning chains that trace each inferential step, it enables models to learn and be evaluated on how they reach conclusions, not just whether those conclusions are correct. This shift from outcome-based to process-based evaluation reveals that the quality of reasoning hinges on the structure and transparency of the intermediate steps connecting premises to conclusions. By constraining inference through formal structure and guiding it through human-written reasoning paths, these works transform reasoning from an incidental by-product of language modeling into a systematic, teachable, and verifiable capability—laying the groundwork for evaluating not just what models conclude, but how and why they reach those conclusions.

Findings from FOLIO reveal that even when using the same model, reasoning performance diverges sharply depending on the representational nature of the problem. Models handle natural language reasoning tasks relatively well when linguistic cues align with surface semantics, but they struggle when inference

depends on deeper logical form. Conversely, on symbolic language reasoning tasks, models can manipulate explicit logical operators but fail to generalize when the same structures are expressed in natural language. These contrasting results show that reasoning competence in LLMs is fragmented—strong in linguistic association but weak in structural abstraction. This asymmetry directly motivates HYBRIDMIND, which bridges this gap by integrating both reasoning modes within a unified framework. Findings from HYBRIDMIND show that dynamically selecting between symbolic and natural-language reasoning pathways substantially improves reasoning consistency across problem types. The hybrid approach enhances overall performance by enabling more reasonable selection between the symbolic language mode and the natural language mode for reasoning, suggesting that robust reasoning requires adaptive coordination rather than commitment to a single mode of inference. In doing so, HYBRIDMIND transforms FOLIO's empirical insight—the context dependence of reasoning performance—into an architectural principle for flexible and generalizable reasoning.

The final study, CourtReasoner, extends this research from the controlled formalism of logic to the open realism of law. Legal reasoning, unlike pure logic, operates in a world of interpretation, precedent, and contestation. CourtReasoner evaluates whether LLM agents can "reason like judges"—constructing coherent, citation-based arguments grounded in factual background and legal holdings.

It introduces a human-AI in-the-loop dataset construction method to capture diverse reasoning outputs and a meta-evaluation procedure to score model reasoning quality. Findings reveal that frontier LLM agents can retrieve relevant precedents and produce full-length judicial opinions, but their reasoning remains inconsistent: more than half of outputs include invalid arguments or incorrect citations. Errors such as misstating procedural histories or misidentifying controlling rules demonstrate that current models still lack the structural discipline required for legally accountable reasoning. These results indicate that domain-specific fine-tuning, rather than web-scale pretraining, is essential for developing models capable of trustworthy legal reasoning.

Taken together, these studies trace reasoning's evolution from formalism to realism. FOLIO represents the formalist phase, where reasoning is defined by logical closure under rules. CourtReasoner, at the opposite end, embodies realism, where reasoning involves context-dependent interpretation and value-laden judgment. Between them, P-FOLIO and HYBRIDMIND serve as bridges, teaching reasoning as a process and enabling adaptive control over reasoning modes. This progression illustrates that while formal reasoning offers clarity and rigor, real-world reasoning demands flexibility and contextual awareness. The synthesis of both traditions points toward reasoning systems that are not only logically sound but also interpretively robust.

6.2 Overarching Lessons and Retropsection

Retrospectively, this dissertation moves from diagnosis to design to application. FOLIO diagnoses the structural weaknesses in LLM reasoning. P-FOLIO designs a method for supervising reasoning processes. HYBRIDMIND engineers a mechanism for adaptive reasoning control. CourtReasoner applies these ideas to a high-stakes domain where reasoning must be both logical and legitimate. Across these stages, the research trajectory mirrors the intellectual growth of the field itself: from viewing reasoning as an isolated computational task to understanding it as a holistic cognitive and ethical enterprise.

The common learning across all chapters converges on four interrelated principles that together define what it means for machines to reason. First, reasoning requires structure: without explicit logical constraints, models cannot sustain internal consistency or distinguish valid inference from surface-level correlation. Second, reasoning requires process transparency: interpretable intermediate steps are essential for teaching, evaluating, and refining reasoning, ensuring that models can not only produce correct conclusions but also demonstrate how they are reached. Third, reasoning requires adaptability: because no single reasoning mode—whether symbolic or linguistic—suffices across all problem types, robust reasoning depends on the capacity to flexibly select and coordinate different inferential strategies. Finally, reasoning requires accountability: in socially consequential domains such as law, reasoning must

be traceable to justifiable principles, allowing its conclusions to be explained, challenged, and improved. Together, these principles redefine reasoning not merely as a cognitive skill but as a disciplinary framework that integrates logical precision, procedural transparency, contextual flexibility, and social responsibility—anchoring intelligence in both rigor and integrity.

6.3 Limitations and Future Directions

Looking ahead, several avenues emerge from this thesis for advancing LLM reasoning.

6.3.1 Disentangling Memorized Knowledge from Higher-Level Problem-Solving

Building on the central insights of this thesis—that correct outputs do not guarantee genuine reasoning, that reasoning benefits from explicit structure and supervision, and that models must dynamically adapt their reasoning approaches—an important future direction is to **systematically disentangle which reasoning behaviors arise from memorization versus those requiring higher-level problem-solving**. This direction asks a question that has been implicit across FOLIO, P-FOLIO, HYBRIDMIND, and CourtReasoner: when do

models “know,” when do they “think,” and how can we measure the difference in a principled way?

A first step is to investigate which abilities can be reproduced simply through exposure to curated datasets during pretraining or midtraining, and which ones fail to emerge without deliberate cognitive strategies such as subgoal-setting, abstraction, verification, and creative problem decomposition. For instance, recent frontier models like o3 outperform similarly sized models such as LLaMA-3.3-70B not merely by recalling arithmetic patterns, but by breaking down large-number multiplication into sequential subtasks and applying heuristics like the “mod 9” check for self-verification. This suggests that certain domain-specific heuristics may be naturally learned through midtraining on math-rich corpora such as OpenWebMath, while reinforcement learning and trajectory-level feedback may be more capable of cultivating deeper behaviors—such as strategic search, long-horizon reasoning, and creative subgoal formation—that are unlikely to arise from parametric recall alone.

Executing this research agenda calls for constructing controlled reasoning environments where tasks vary systematically in their dependence on memorized knowledge versus strategic reasoning. Rather than relying on static benchmark distinctions, these environments should explicitly stress-test whether a model can generalize reasoning skills when surface forms or domain heuristics are deliberately perturbed. Performance should be evaluated not only through final

answers, but through reasoning trace quality, including decomposition steps, verification behaviors, calibration, justification, and the diversity of solution paths. In doing so, we can map which capabilities emerge through scale and data exposure and which require explicit incentive structures or curriculum design.

A complementary line of inquiry draws from cognitive psychology, examining whether models exhibit behaviors associated with true problem-solving, including overcoming initial fixation, generating novel strategies, and experiencing analogs to human "insight." Benchmarking models on puzzles that require structural re-representation, rather than direct recall, can help differentiate memorized heuristics from genuinely productive reasoning. Likewise, probing susceptibility to cognitive illusions—such as belief bias or overconfidence compounding—or analyzing when models default to over-learned templates can reveal the boundary between pattern completion and deliberate thinking.

Ultimately, this research aims to produce a principled reasoning taxonomy and training curriculum grounded in the lessons of this dissertation: explicit logic supports fidelity, reasoning traces support verifiability, adaptive strategy selection supports robustness, and contextual accountability supports real-world deployment. By rigorously separating what models absorb through data from what they derive through structured reasoning, we can design more efficient training pipelines, clearer evaluation standards, and developmental pathways

for models that move beyond memorization toward structured, flexible, and reflective reasoning. This line of inquiry has the potential to shape the next phase of LLM research, bridging the gap between powerful pattern learners and responsible, strategic reasoners.

6.3.2 Constructing a Reasoning Taxonomy for Improving Large Reasoning Models

Having worked on diverse reasoning problems—from fundamental logical inference, to mathematically structured problem-solving, to legally grounded reasoning where argumentation and retrieval intertwine—I am motivated to build a comprehensive taxonomy of reasoning that can inform future language-model design. The goal is to systematically characterize reasoning tasks not only by their surface domains (e.g., logic, math, code, law) but by the cognitive demands and representational structures they require. A taxonomy must be both bottom-up, grounded in benchmark characteristics (e.g., purely deductive tasks like first-order logic; knowledge-intensive math such as AIME and MATH; intuition-driven, low-knowledge puzzles such as BRAINTEASERS (Han et al., 2025b)), and top-down, motivated by principles from cognitive science and AI (e.g., deductive, inductive, abductive, and analogical reasoning; search and planning; verification and subgoal formation; bias and cognitive illusions;

deterministic vs. probabilistic settings; resource-bounded heuristics). Such a framework would help distinguish knowledge from rules, and heuristics from systematic decomposition, reflecting that many real-world problems are hybrid knowledge-reasoning tasks where retrieval and inference constantly interact. It would also explicitly incorporate tractability and bounded rationality, recognizing that some domains (e.g., brain-teaser-style insight problems or open-domain legal analysis) require domain-specific heuristics and fast, frugal decision strategies rather than pure algorithmic reasoning. Ultimately, the taxonomy would serve as a principled guide for identifying gaps in existing benchmarks, articulating reasoning subskills, and shaping model architectures and training methods that reflect the full spectrum of human-like reasoning—from formal deduction to heuristic insight, from search-based planning to normative judgment.

To execute this taxonomy-building agenda concretely, the next step is to construct a systematic experimental framework that bridges empirical benchmarking with cognitive-theoretical synthesis. The work should begin with a bottom-up empirical mapping of reasoning tasks across existing datasets—spanning logic, mathematics, programming, commonsense, and law—and quantifying the representational and inferential structures they involve. This would include defining dimensions such as reasoning depth, abstraction level, dependence on external knowledge, modality (symbolic vs. natural language), determinism, and

the extent of strategic decomposition required. By clustering benchmarks along these measurable dimensions, we can empirically identify where reasoning capabilities diverge, where transfer occurs, and where gaps remain in model competence.

Parallel to this empirical mapping, a top-down theoretical modeling phase should integrate insights from cognitive science and computational rationality. Here, reasoning types—deductive, inductive, abductive, and analogical—can be positioned within a unified space of problem characteristics, such as uncertainty, tractability, and resource constraints. Drawing from the theory of bounded rationality, this framework would model reasoning not as perfect logical inference, but as the optimal use of limited computational resources. Cognitive signatures like belief bias, heuristic search, and insight generation can be operationalized as measurable model behaviors. This top-down modeling, when aligned with bottom-up task characterization, will yield a multidimensional taxonomy that is both empirically grounded and theoretically principled. The third stage involves building benchmark suites and diagnostic probes designed explicitly to test hypotheses from the taxonomy. For example, a "reasoning stress test" could systematically vary problem tractability, knowledge availability, or cognitive load, observing whether models transition from algorithmic reasoning to heuristic shortcuts. Similarly, insight-style puzzles could measure whether models display productive re-representation—a hallmark of human reasoning not easily

captured by standard accuracy metrics. These benchmarks would be used not merely for leaderboard comparisons, but for diagnostic profiling, allowing researchers to assess the structure, adaptability, and robustness of reasoning strategies within and across model architectures.

Finally, the taxonomy should feed directly into training and evaluation design. By mapping reasoning subskills to model behaviors, we can design curricula and reinforcement learning objectives that explicitly target missing capabilities—such as hypothesis generation, verification, or counterfactual reasoning—rather than relying on incidental emergence during large-scale pretraining. This process would also enable reasoning-aware pretraining, where data selection is guided by the taxonomy's coverage of reasoning types, and reasoning-structured evaluation, where models are assessed on both outcomes and intermediate reasoning trajectories. Together, these efforts would transform the taxonomy from a descriptive tool into a generative framework—one that guides how we build, train, and test future reasoning models capable of both logical rigor and adaptive intelligence.

Bibliography

Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. 2024. Large language models for mathematical reasoning: Progresses and challenges. *arXiv preprint arXiv:2402.00157*.

Anthropic. 2024. Claude 3.5 sonnet. https://www.anthropic.com/news/claude-3-5-sonnet.

Anthropic. 2025. Claude 3.7 sonnet and claude code. Announcement blog post.

David Arps, Jan Kels, Florian Krämer, Yunus Renz, Regina Stodden, and Wiebke Petersen. 2022. HHUplexity at text complexity DE challenge 2022. In *Proceedings of the GermEval 2022 Workshop on Text Complexity Assessment of German Text*, pages 27–32, Potsdam, Germany. Association for Computational Linguistics.

Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, Quyet V. Do, Yan Xu, and Pascale Fung. 2023. A multitask, multilingual, multimodal

evaluation of chatgpt on reasoning, hallucination, and interactivity. *Preprint*, arXiv:2302.04023.

Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, USVSN Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. 2022. Gpt-neox-20b: An open-source autoregressive language model. *arXiv preprint*.

Andrew Blair-Stanek, Nils Holzenberger, and Benjamin Van Durme. 2023. Can gpt-3 perform statutory reasoning? In *Proceedings of the Nineteenth International Conference on Artificial Intelligence and Law*, pages 22–31.

Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 632–642, Lisbon, Portugal. Association for Computational Linguistics.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott

Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021a. Evaluating large language models trained on code. *CoRR*, abs/2107.03374.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman,

Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, Suchir Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021b. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*.

Wenhu Chen, Hanwen Zha, Zhiyu Chen, Wenhan Xiong, Hong Wang, and William Yang Wang. 2020. HybridQA: A dataset of multi-hop question answering over tabular and textual data. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1026–1036, Online. Association for Computational Linguistics.

Jonathan Choi and Daniel Schwarcz. 2025. Ai assistance in legal analysis: An

empirical study. *73 J. Legal Educ. 384*.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. 2022. Scaling instruction-finetuned language models. *arXiv preprint*.

Peter Clark, Oyvind Tafjord, and Kyle Richardson. 2020. Transformers as soft reasoners over language. *CoRR*, abs/2002.05867.

Peter Clark, Oyvind Tafjord, and Kyle Richardson. 2021. Transformers as soft reasoners over language. In *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, pages 3882–3890.

Robin Cooper, Dick Crouch, Jan Van Eijck, Chris Fox, Johan Van Genabith, Jan Jaspars, Hans Kamp, David Milward, Manfred Pinkal, Massimo Poesio, et al.

1996. Using the framework. Technical report, Technical Report LRE 62-051 D-16, The FraCaS Consortium.

Antonia Creswell, Murray Shanahan, and Irina Higgins. 2022. Selection-inference: Exploiting large language models for interpretable logical reasoning. *arXiv preprint arXiv:2205.09712*.

Edgar Dale and Jeanne S. Chall. 1948. A formula for predicting readability. *Educational Research Bulletin*, 27(1):11–28.

Edgar Dale and Jeanne S. Chall. 1995. *Readability Revisited: The New Dale-Chall Readability Formula*. Brookline Books.

Ishita Dasgupta, Andrew K Lampinen, Stephanie CY Chan, Antonia Creswell, Dharshan Kumaran, James L McClelland, and Felix Hill. 2022. Language models show human-like content effects on reasoning. *arXiv preprint arXiv:2207.07051*.

Donald Davidson. 2001. 105The Logical Form of Action Sentences. In *Essays on Actions and Events*. Oxford University Press.

April Dawson. 2024. Algorithmic adjudication and constitutional ai - the promise of a better ai decision making future? *SMU Science & Technology Law Review*, 27:11–. First page 11; full page range not specified.

BIBLIOGRAPHY

Google DeepMind. 2025. Gemini 2.0 flash. https://deepmind.google/technologies/gemini/flash/.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

David Dohan, Winnie Xu, Aitor Lewkowycz, Jacob Austin, David Bieber, Raphael Gontijo Lopes, Yuhuai Wu, Henryk Michalewski, Rif A. Saurous, Jascha Sohl-dickstein, Kevin Murphy, and Charles Sutton. 2022. Language model cascades. *arXiv preprint*.

Abhimanyu Dubey, Abhinav Jauhri, and Others. 2024. The llama 3 herd of models. *Preprint*, arXiv:2407.21783.

Ronald Dworkin. 1986. *Law's Empire*. Harvard University Press. Google-Books-ID: 8TCx_UbbI3AC.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Pal: Program-aided language models. *Preprint*, arXiv:2211.10435.

BIBLIOGRAPHY

Peizhong Gao, Ao Xie, Shaoguang Mao, Wenshan Wu, Yan Xia, Haipeng Mi, and Furu Wei. 2024. Meta reasoning for large language models. *Preprint*, arXiv:2406.11698.

Google. 2024. Gemini deep research. `https://gemini.google/overview/deep-research/?hl=en`. Accessed: 2025-05-20.

Google DeepMind. 2025. Gemini pro. Accessed: 2025-05-20.

Zhibin Gou, Zhihong Shao, Yeyun Gong, yelong shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2024. ToRA: A tool-integrated reasoning agent for mathematical problem solving. In *The Twelfth International Conference on Learning Representations*.

Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. 2024. Cruxeval: A benchmark for code reasoning, understanding and execution. *Preprint*, arXiv:2401.03065.

Neel Guha, Julian Nyarko, Daniel Ho, Christopher Ré, Adam Chilton, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel Rockmore, Diego Zambrano, et al. 2023. Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models. *Advances in neural information processing systems*, 36:44123–44279.

Shashank Gupta, Vaishnavi Shrivastava, Ameet Deshpande, Ashwin Kalyan,

Peter Clark, Ashish Sabharwal, and Tushar Khot. 2023. Bias runs deep: Implicit reasoning biases in persona-assigned llms. *arXiv preprint arXiv:2311.04892*.

Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Luke Benson, Lucy Sun, Ekaterina Zubova, Yujie Qiao, Matthew Burtell, David Peng, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Shafiq Joty, Alexander R. Fabbri, Wojciech Kryscinski, Xi Victoria Lin, Caiming Xiong, and Dragomir Radev. 2022. Folio: Natural language reasoning with first-order logic. *Preprint*, arXiv:2209.00840.

Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alexander Wardle-Solano, Hannah Szabó, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander Fabbri, Wojciech Maciej Kryscinski, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying, Arman Cohan, and Dragomir Radev. 2024. FOLIO: Natural language reasoning with first-order logic. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 22017–22031, Miami, Florida, USA. Association

for Computational Linguistics.

Simeng Han, Stephen Xia, Grant Zhang, Howard Dai, Chen Liu, Lichang Chen, Hoang Huy Nguyen, Hongyuan Mei, Jiayuan Mao, and R Thomas McCoy. 2025a. Creativity or brute force? using brainteasers as a window into the problem-solving abilities of large language models. *arXiv preprint arXiv:2505.10844*.

Simeng Han, Stephen Xia, Grant Zhang, Howard Dai, Chen Liu, Lichang Chen, Hoang Huy Nguyen, Hongyuan Mei, Jiayuan Mao, and R. Thomas McCoy. 2025b. Creativity or brute force? using brainteasers as a window into the problem-solving abilities of large language models. *arXiv preprint arXiv:2505.10844*.

Eva H. Hanks, Michael E. Herz, and Steven S. Nemerson. 1994. *Elements of Law*. Foundation Press, Westbury, NY.

Weinan He, Canming Huang, Yongmei Liu, and Xiaodan Zhu. 2021. WinoLogic: A zero-shot logic-based diagnostic dataset for Winograd Schema Challenge. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3779–3789, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. 2021a. Cuad: An expert-annotated nlp dataset for legal contract review. In *Thirty-*

fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1).

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021b. Measuring mathematical problem solving with the math dataset. *Preprint*, arXiv:2103.03874.

Nils Holzenberger and Benjamin Van Durme. 2021. Factoring statutory reasoning as language understanding challenges. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 2742–2758.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Shima Imani, Liang Du, and Harsh Shrivastava. 2023. Mathprompter: Mathematical reasoning using large language models. *Preprint*, arXiv:2303.05398.

Akira Kawabata and Saku Sugawara. 2023. Evaluating the rationale understanding of critical reasoning in logical reading comprehension. *Preprint*, arXiv:2311.18353.

BIBLIOGRAPHY

Mehran Kazemi, Quan Yuan, Deepti Bhatia, Najoung Kim, Xin Xu, Vaiva Imbrasaite, and Deepak Ramachandran. 2023. Boardgameqa: A dataset for natural language reasoning with contradictory information. *Preprint*, arXiv:2306.07934.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *arXiv preprint arXiv:2205.11916*.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2023. Large language models are zero-shot reasoners. *Preprint*, arXiv:2205.11916.

Yuta Koreeda and Christopher D Manning. 2021. Contractnli: A dataset for document-level natural language inference for contracts. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 1907–1919.

Sumith Kulal, Panupong Pasupat, Kartik Chandra, Mina Lee, Oded Padon, Alex Aiken, and Percy S Liang. 2019. Spoc: Search-based pseudocode to code. *Advances in Neural Information Processing Systems*, 32.

Anne Lehman. 1973. Two sets of perfect syllogisms. *Notre Dame Journal of Formal Logic*, 14(3):425 – 429.

Sarah-Jane Leslie. 2008. Generics: Cognition and Acquisition. *The Philosophical Review*, 117(1):1–47.

Yifei Li, Zeqi Lin, Shizhuo Zhang, Qiang Fu, Bei Chen, Jian-Guang Lou, and Weizhu Chen. 2022. On the advance of making language models better reasoners. *arXiv preprint arXiv:2206.02336*.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain. Association for Computational Linguistics.

Marco Lippi, Przemysław Pałka, Giuseppe Contissa, Francesca Lagioia, Hans-Wolfgang Micklitz, Giovanni Sartor, and Paolo Torroni. 2019. Claudette: an automated detector of potentially unfair clauses in online terms of service. *Artificial Intelligence and Law*, 27:117–139.

Alisa Liu, Zhaofeng Wu, Julian Michael, Alane Suhr, Peter West, Alexander Koller, Swabha Swayamdipta, Noah A. Smith, and Yejin Choi. 2023. We're afraid language models aren't modeling ambiguity. *Preprint*, arXiv:2304.14399.

Jian Liu, Leyang Cui, Hanmeng Liu, Dandan Huang, Yile Wang, and Yue Zhang. 2021. Logiqa: a challenge dataset for machine reading comprehension with logical reasoning. In *Proceedings of the Twenty-Ninth International*

Conference on International Joint Conferences on Artificial Intelligence, pages 3622–3628.

Tianyu Liu, Yijia Xiao, Xiao Luo, Hua Xu, Wenjin Zheng, and Hongyu Zhao. 2024. Geneverse: A collection of open-source multimodal large language models for genomic and proteomic research. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 4819–4836, Miami, Florida, USA. Association for Computational Linguistics.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Ro{bert}a: A robustly optimized {bert} pretraining approach. *arXiv preprint arXiv:1907.11692*.

Man Luo, Shrinidhi Kumbhar, Ming shen, Mihir Parmar, Neeraj Varshney, Pratyay Banerjee, Somak Aditya, and Chitta Baral. 2024. Towards logiglue: A brief survey and a benchmark for analyzing logical reasoning capabilities of language models. *Preprint*, arXiv:2310.00836.

Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. 2023. Faithful chain-of-thought reasoning. *Preprint*, arXiv:2301.13379.

W. McCune. 2005–2010. Prover9 and mace4. `http://www.cs.unm.edu/~mccune/prover9/`.

BIBLIOGRAPHY

Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. 2023. Learning deductive reasoning from synthetic corpus based on formal logic. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 25254–25274. PMLR.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*.

John J Nay, David Karamardian, Sarah B Lawsky, Wenting Tao, Meghana Bhat, Raghav Jain, Aaron Travis Lee, Jonathan H Choi, and Jungo Kasai. 2024. Large language models as tax attorneys: a case study in legal capabilities emergence. *Philosophical Transactions of the Royal Society A*, 382(2270):20230159.

Ansong Ni, Pengcheng Yin, Yilun Zhao, Martin Riddell, Troy Feng, Rui Shen, Stephen Yin, Ye Liu, Semih Yavuz, Caiming Xiong, Shafiq Joty, Yingbo Zhou, Dragomir Radev, and Arman Cohan. 2023. L2ceval: Evaluating language-to-code generation capabilities of large language models. *Preprint*, arXiv:2309.17446.

BIBLIOGRAPHY

Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. 2002. *Isabelle/Hol a Proof Assistant for Higher-Order Logic*. Springer.

Theo Olausson, Alex Gu, Ben Lipkin, Cedegao Zhang, Armando Solar-Lezama, Joshua Tenenbaum, and Roger Levy. 2023. LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5153–5176, Singapore. Association for Computational Linguistics.

OpenAI. 2025a. Introducing openai o3 and o4-mini. `https://openai.com/index/introducing-o3-and-o4-mini/`. Accessed: 2025-05-13.

OpenAI. 2025b. Introducing deep research. Announcement blog post.

OpenAI, Josh Achiam, and Others. 2023. Gpt-4 technical report. *Preprint*, arXiv:2303.08774.

OpenAI, Josh Achiam, and Others. 2024. Gpt-4 technical report. *Preprint*, arXiv:2303.08774.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. 2023. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics:*

EMNLP 2023, pages 3806–3824, Singapore. Association for Computational Linguistics.

Mihir Parmar, Nisarg Patel, Neeraj Varshney, Mutsumi Nakamura, Man Luo, Santosh Mashetty, Arindam Mitra, and Chitta Baral. 2024. LogicBench: Towards systematic evaluation of logical reasoning ability of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13679–13707, Bangkok, Thailand. Association for Computational Linguistics.

Terence Parsons. 1990. *Events in the Semantics of English*. MIT Press, Cambridge, MA, USA.

Robert Post. 2024. The Supreme Court's Crisis of Authority: Law, Politics, and the Judiciary Act of 1925.

Ben Prystawski, Michael Y. Li, and Noah D. Goodman. 2023. Why think step by step? reasoning emerges from the locality of experience. *Preprint*, arXiv:2304.03843.

Leonardo Ranaldi and Andre Freitas. 2024. Aligning large and small language models via chain-of-thought reasoning. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1812–1827, St. Julian's, Malta. Association for Computational Linguistics.

BIBLIOGRAPHY

Abhilasha Ravichander, Alan W Black, Shomir Wilson, Thomas Norton, and Norman Sadeh. 2019. Question answering for privacy policies: Combining computational and legal perspectives. *arXiv preprint arXiv:1911.00841*.

Richard Re. 2023. Artificial authorship and judicial opinions. *George Washington Law Review*, 92:1558–. First page 1558; full page range not specified.

Yuval Reif and Roy Schwartz. 2024. Beyond performance: Quantifying and mitigating label bias in llms. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6784–6798.

Stuart Russell and Peter Norvig. 2010. *Artificial Intelligence: A Modern Approach*, 3 edition. Prentice Hall.

Mohammed Saeed, Naser Ahmadi, Preslav Nakov, and Paolo Papotti. 2021. RuleBERT: Teaching soft rules to pre-trained language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 1460–1476, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Swarnadeep Saha, Sayan Ghosh, Shashank Srivastava, and Mohit Bansal. 2020. PRover: Proof generation for interpretable reasoning over rules. In *Proceedings of the 2020 Conference on Empirical Methods in Natural*

Language Processing (EMNLP), pages 122–136, Online. Association for Computational Linguistics.

Swarnadeep Saha, Prateek Yadav, and Mohit Bansal. 2021. multiPRover: Generating multiple proofs for improved interpretability in rule reasoning. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3662–3677, Online. Association for Computational Linguistics.

Abulhair Saparov and He He. 2023. Language models can (kind of) reason: A systematic formal analysis of chain-of-thought. In *International Conference on Learning Representations*.

Abulhair Saparov, Richard Yuanzhe Pang, Vishakh Padmakumar, Nitish Joshi, Mehran Kazemi, Najoung Kim, and He He. 2023. Testing the general deductive reasoning capacity of large language models using OOD examples. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Frederick Schauer. 2009. *Thinking Like a Lawyer: A New Introduction to Legal Reasoning*. Harvard University Press, Cambridge, MA.

Daniel Schwarcz, Sam Manning, Patrick James Barry, David R. Cleveland, J. J. Prescott, and Beverly Rich. 2025. Ai-powered lawyering: Ai reasoning models, retrieval augmented generation, and the future of legal practice. Technical

Report 25-16, Minnesota Legal Studies Research Paper. Minnesota Legal Studies Research Paper No. 25-16.

Hrituraj Singh, Milan Aggrawal, and Balaji Krishnamurthy. 2020. Exploring neural models for parsing natural language into first-order logic. *arXiv preprint arXiv:2002.06544*.

Pranaydeep Singh, Luna De Bruyne, Orphée De Clercq, and Els Lefever. 2023. Misery loves complexity: Exploring linguistic complexity in the context of emotion detection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 12871–12880, Singapore. Association for Computational Linguistics.

Koustuv Sinha, Shagun Sodhani, Jin Dong, Joelle Pineau, and William L. Hamilton. 2019. CLUTRR: A diagnostic benchmark for inductive reasoning from text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4506–4515, Hong Kong, China. Association for Computational Linguistics.

Aarohi Srivastava, Abhinav Rastogi, and +447 Authors. 2023. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Preprint*, arXiv:2206.04615.

Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb,

BIBLIOGRAPHY

Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. 2022. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*.

Hongda Sun, Weikai Xu, Wei Liu, Jian Luan, Bin Wang, Shuo Shang, Ji-Rong Wen, and Rui Yan. 2023. From indeterminacy to determinacy: Augmenting logical reasoning capabilities with large language models. *Preprint*, arXiv:2310.18659.

G. Sutcliffe. 2017. The TPTP Problem Library and Associated Infrastructure. From CNF to TH0, TPTP v6.4.0. *Journal of Automated Reasoning*, 59(4):483–502.

Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. 2021. ProofWriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3621–3634, Online. Association for Computational Linguistics.

Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language*

Technologies, Volume 1 (Long and Short Papers), pages 4149–4158, Minneapolis, Minnesota. Association for Computational Linguistics.

Alon Talmor, Oyvind Tafjord, Peter Clark, Yoav Goldberg, and Jonathan Berant. 2020. Leap-of-thought: Teaching pre-trained models to systematically reason over implicit knowledge. *Advances in Neural Information Processing Systems*, 33:20227–20237.

Jidong Tian, Yitian Li, Wenqing Chen, Liqiang Xiao, Hao He, and Yaohui Jin. 2021. Diagnosing the first-order logical reasoning ability through LogicNLI. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3738–3747, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *Preprint*, arXiv:2302.13971.

Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. 2020. Scipy 1.0: fundamental algorithms for scientific computing in python. *Nature methods*, 17(3):261–272.

BIBLIOGRAPHY

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2019. Superglue: A stickier benchmark for general-purpose language understanding systems. *Advances in neural information processing systems*, 32.

Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*.

Steven H Wang, Antoine Scardigli, Leonard Tang, Wei Chen, Dimitry Levkin, Anya Chen, Spencer Ball, Thomas Woodside, Oliver Zhang, and Dan Hendrycks. 2023a. Maud: An expert-annotated legal nlp dataset for merger agreement understanding. *arXiv preprint arXiv:2301.00876*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023b. Self-consistency improves chain of thought reasoning in language models. *Preprint*, arXiv:2203.11171.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023c. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*.

Jason Wei, Kelly Finn, Emma Templeton, Thalia Wheatley, and Soroush Vosoughi. 2021. Linguistic complexity loss in text-based therapy. In

Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 4450–4459, Online. Association for Computational Linguistics.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022a. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. 2022b. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-thought prompting elicits reasoning in large language models. *Preprint*, arXiv:2201.11903.

Jason Weston, Antoine Bordes, Sumit Chopra, Alexander M. Rush, Bart van Merriënboer, Armand Joulin, and Tomas Mikolov. 2015. Towards ai-complete question answering: A set of prerequisite toy tasks. *Preprint*, arXiv:1502.05698.

Shomir Wilson, Florian Schaub, Aswarth Abhilash Dara, Frederick Liu, Sushain Cherivirala, Pedro Giovanni Leon, Mads Schaarup Andersen, Sebastian Zimmeck, Kanthashree Mysore Sathyendra, N Cameron Russell, et al. 2016.

The creation and analysis of a website privacy policy corpus. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1330–1340.

Fangzhi Xu, Qika Lin, Jiawei Han, Tianzhe Zhao, Jun Liu, and Erik Cambria. 2025. Are large language models really good logical reasoners? a comprehensive evaluation and beyond. *IEEE Transactions on Knowledge and Data Engineering*.

Jundong Xu, Hao Fei, Liangming Pan, Qian Liu, Mong-Li Lee, and Wynne Hsu. 2024. Faithful logical reasoning via symbolic chain-of-thought. *Preprint*, arXiv:2405.18357.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024a. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*.

Kaiyu Yang, Jia Deng, and Danqi Chen. 2022. Generating natural language proofs with verifier-guided search. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 89–105, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Kaiyu Yang, Gabriel Poesia, Jingxuan He, Wenda Li, Kristin Lauter, Swarat Chaudhuri, and Dawn Song. 2024b. Formal mathematical reasoning: A new frontier in ai. *Preprint*, arXiv:2412.16075.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium. Association for Computational Linguistics.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik R Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. 2023. SatLM: Satisfiability-aided language models using declarative prompting. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Weihao Yu, Zihang Jiang, Yanfei Dong, and Jiashi Feng. 2020. Reclor: A reading comprehension dataset requiring logical reasoning. In *International Conference on Learning Representations*.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. 2022a. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022b. Star: Bootstrapping reasoning with reasoning. *arXiv preprint*.

Jiaxin Zhang, Zhongzhi Li, Mingliang Zhang, Fei Yin, Chenglin Liu, and Yashar Moshfeghi. 2024. Geoeval: benchmark for evaluating llms and multi-modal models on geometry problem-solving. *arXiv preprint arXiv:2402.10104*.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*.

Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H. Chi, Quoc V Le, and Denny Zhou. 2024a. Take a step back: Evoking reasoning via abstraction in large language models. *Preprint*, arXiv:2310.06117.

Lucia Zheng, Neel Guha, Brandon R Anderson, Peter Henderson, and Daniel E Ho. 2021. When does pretraining help? assessing self-supervised learning for law and the casehold dataset of 53,000+ legal holdings. In *Proceedings of the eighteenth international conference on artificial intelligence and law*, pages 159–168.

Lucia Zheng, Neel Guha, Javokhir Arifov, Sarah Zhang, Michal Skreta, Christopher D Manning, Peter Henderson, and Daniel E Ho. 2025. A reasoning-focused legal retrieval benchmark. In *Proceedings of the 2025 Symposium on Computer Science and Law*, pages 169–193.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi

Feng, and Yongqiang Ma. 2024b. Llamafactory: Unified efficient fine-tuning of 100+ language models. *arXiv preprint arXiv:2403.13372*.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V Le, and Ed H. Chi. 2023. Least-to-most prompting enables complex reasoning in large language models. In *The Eleventh International Conference on Learning Representations*.

Jin Peng Zhou, Charles Staats, Wenda Li, Christian Szegedy, Kilian Q. Weinberger, and Yuhuai Wu. 2024a. Don't trust: Verify – grounding llm quantitative reasoning with autoformalization. *Preprint*, arXiv:2403.18120.

Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V. Le, Ed H. Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. 2024b. Self-discover: Large language models self-compose reasoning structures. *Preprint*, arXiv:2402.03620.

Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V. Le, Ed H. Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. 2024c. Self-discover: Large language models self-compose reasoning structures. *Preprint*, arXiv:2402.03620.

Yukai Zhou, Zhijie Huang, Feiyang Lu, Zhan Qin, and Wenjie Wang. 2024d.

Don't say no: Jailbreaking llm by suppressing refusal. *arXiv preprint arXiv:2404.16369*.

Sebastian Zimmeck, Peter Story, Daniel Smullen, Abhilasha Ravichander, Ziqi Wang, Joel R Reidenberg, N Cameron Russell, and Norman Sadeh. 2019. Maps: Scaling privacy compliance analysis to a million apps. *Proc. Priv. Enhancing Tech.*, 2019:66.

Appendix A

Appendix

A.1 P-FOLIO Annotation Guidelines

We provide the detailed annotation guidelines in Table S1.

A.2 P-FOLIO Hyper-parameter settings

For fine-tuning Flan-T5-Large, we use a learning rate of 1e-4, batch size of 8, gradient accumulation of 2, warmup step of 100 and the model was trained for 3 epochs. For fine-tuning LLama-8b, we use a learning rate of 2e-5, batch size of 1 and gradient accumulation of 8. The warmup steps were set to 25 and the model was trained for 3 epochs.

A.3 HybridMind implementation details

A.3.0.0.1 PROMPTING-BASED HYBRIDMIND.

In our experiment, we considered prompting different LLMs with various instructions to work as a meta selector for generating correct solution based on the given question. Regarding the format of prompting, we tried two different zero-shot instruction design and made comparison. We also tried few-shot prompting design, whose number of examples (shots) in the instruction is at most 2. The shot contains question, answer, and the rationale generated by GPT-o3-mini.

A.3.0.0.2 FINE-TUNING-BASED HYBRIDMIND.

We also performed experiments to fine-tune Llama 3.1-8B to perform complicated reasoning tasks. We prepared the training datasets by sampling the choices and paired one correct choice with one question. The implementation of SFT is based on Llama-factory and the hyper-parameter setting is same as the example configuration template. The implementation of STaR is modified from the original code base. STaR utilizes models to generate rationale and then select the corret samples as well as the generated rationales to perform fine-tuning. To fine-tune our models, we utilize both NVIDIA H100 GPU and Together AI API. Details of our training codes can be found in the attached zip file to this

submission.

A.3.0.0.3 Information of baseline models.

In our experiment, we considered prompting closed-source LLMs, including GPT-4o, GPT-3.5-turbo, GPT-o3-mini, and prompting open-source LLM including Llama 3.1-8b. The prompting experiment is performed with different strategies (NL, SL, SymbolNL, NLSymbol) for solving the question directly. MV represents performing majority voting based on the methods' outputs. We also considered fine-tuning the base model with different strategies, including making binary choices (NL or SL), or making 4-class choices (NL, SL, SymbolNL, NLSymbol). The fine-tuning process is for making a meta selector.

A.4 Statistics of testing data in HybridMind

Please check Tables S2, S3, and S4.

Table S2 shows the number of samples in the MATH testing set, broken down by problem category. The dataset includes a diverse set of mathematical domains such as Algebra, Counting & Probability, Geometry, Number Theory, Intermediate Algebra, Precalculus, and Prealgebra. The distribution highlights the breadth of mathematical reasoning required, with category sizes ranging from fewer than 500 problems (e.g., Counting & Probability, Geometry) to

over 1,000 problems (e.g., Algebra), for a total of 5,000 test samples. This categorization is useful for analyzing model performance across distinct mathematical subfields.

Table S3 shows the frequency and proportion of selected choice types produced by HybridMind when applied to the MATH and FOLIO test sets. For MATH, a 4-choice selector was implemented, while for FOLIO, a 2-choice selector was used. The table compares natural language (NL) answers, symbolic logic (SL) outputs, and hybrid symbolic–natural language approaches (SymbolNL and NLSymbol) where applicable. Results show that NL dominates in both datasets (94.10% for MATH and 58.84% for FOLIO), but symbolic forms still play a meaningful role, especially in FOLIO where SL accounts for over 40% of the responses. These proportions illustrate the differences in model selection characteristic across datasets.

Table S4 shows the number of samples in the FOLIO testing set, broken down by logical reasoning category. The dataset distinguishes between WikiLogic and HybLogic cases, with a near-equal distribution of samples (111 vs. 115), for a total of 226 test cases. This balanced division enables evaluation of model performance across different types of logical reasoning tasks, capturing both simpler knowledge-based logic and more complex hybrid logical reasoning structures.

A.5 Significance test for HybridMind

A.6 Analysis between SL and NL with Case Studies for HybridMind

We present case studies for SL and NL reasoning in TablesS7, S8, S9, and S10.

```
nl_system_prompt = '''
You are a helpful assistant who is good at solving math problems.
    You should follow the guidelines below:
- Present the final result in LaTeX using a `\boxed{}` without
   any units.
- Utilize the `pi` symbol, and simplify all fractions and square
   roots without converting them to decimal values.
'''

nl_instruction_prompt = "Please␣think␣step␣by␣step.␣"

nl_math_shots= [
'''Question: Kevin Kangaroo begins hopping on a number line at 0.
    He wants to get to 1, but he can hop only $\frac{1}{3}$ of
   the distance. Each hop tires him out so that he continues to
   hop $\frac{1}{3}$ of the remaining distance. How far has he
   hopped after five hops? Express your answer as a common
   fraction.
Answer: Let's think step by step
Kevin hops $1/3$ of the remaining distance with every hop.
His first hop takes $1/3$ closer.
```

For his second hop, he has $2/3$ left to travel, so he hops forward $(2/3)(1/3)$.

For his third hop, he has $(2/3)^2$ left to travel, so he hops forward $(2/3)^2(1/3)$.

In general, Kevin hops forward $(2/3)^{k-1}(1/3)$ on his kth hop.

We want to find how far he has hopped after five hops.

This is a finite geometric series with first term $1/3$, common ratio $2/3$, and five terms.

Thus, Kevin has hopped $\frac{\frac{1}{3}\left(1-\left(\frac{2}{3}\right)^5\right)}{1-\frac{2}{3}} = \boxed{\frac{211}{243}}$.

The answer is \frac{211}{243}''',

'''Question: What is the area of the region defined by the equation $x^2+y^2 - 7 = 4y-14x+3$?

Answer: Let's think step by step

We rewrite the equation as $x^2 + 14x + y^2 - 4y = 10$ and then complete the square,

resulting in $(x+7)^2-49 + (y-2)^2-4=10$,

or $(x+7)^2+(y-2)^2=63$.

This is the equation of a circle with center $(-7, 2)$ and radius
$\sqrt{63},$
so the area of this region is $\pi r^2 = \boxed{63\pi}$.
The answer is 63\pi'''',

''''Question: If $x^2+y^2=1$, what is the largest possible value
of $|x|+|y|$?
Answer: Let's think step by step
If (x,y) lies on the circle,
so does $(x,-y),$ $(-x,-y),$ and $(-x,-y),$ (which all give the
same value of $|x| + |y|$),
so we can assume that $x \ge 0$ and $y \ge 0.$
Then $|x| + |y| = x + y.$ Squaring, we get
\[(x + y)^2 = x^2 + 2xy + y^2 = 1 + 2xy.\]
Note that $(x - y)^2 \ge 0.$
Expanding, we get $x^2 - 2xy + y^2 \ge 0,$ so $2xy \le x^2 + y^2
= 1.$
Hence,\[1 + 2xy \le 2,\]which means $x + y \le \sqrt{2}.$
Equality occurs when $x = y = \frac{1}{\sqrt{2}},$
so the maximum value of $|x| + |y|$ is $\boxed{\sqrt{2}}.$
The answer is \sqrt{2}''''',

APPENDIX A. APPENDIX

```
'''Question: If $f(x)=\frac{ax+b}{cx+d}, abcd\not=0$ and $f(f(x))
   =x$ for all $x$ in the domain of $f$, what is the value of $a+
   d$?
Answer: Let's think step by step
The condition $f(f(x))$ means that $f$ is the inverse of itself,
so its graph is symmetrical about the line $y = x$.
With a rational function of this form, we will have two
   asymptotes:
a vertical one at $x=-d/c$ if $cx+d$ does not divide $ax+b$,
and a horizontal one at $y=a/c$,
if we take the limit of $f(x)$ as $x$ goes to $\pm\infty$.
In order for $f$ to be its own inverse, the intersection of the
   asymptotes must lie on the line $y=x$
so that it and its asymptotes reflect onto themselves.
This means that $-d/c=a/c$,
and therefore $-d=a$ and $a+d=\boxed{0}$.
The answer is 0''',
```

APPENDIX A. APPENDIX

'''Question: A math teacher requires Noelle to do one homework assignment for each of the first five homework points she wants to earn; for each of the next five homework points, she needs to do two homework assignments; and so on, so that to earn the nth homework point, she has to do $n\div5$ (rounded up) homework assignments. For example, when she has 11 points, it will take $12\div5=2.4\rightarrow3$ homework assignments to earn her 12th point. What is the smallest number of homework assignments necessary to earn a total of 25 homework points?

Answer: Let's think step by step

Noelle only has to do 1 homework assignment to earn her first point,

and the same is true for each of her first five points.

She must then do 2 homework assignments to earn her sixth point, seventh point, and so on, up to her tenth point.

Continuing, we see that Noelle must do a total of \[1+1+1+1+1+2+2+2+2+2+\dots+5+5+5+5+5\] homework assignments to earn 25 points.

This sum may be rewritten as $5(1+2+3+4+5)=5(15)=\boxed{75}$.

The answer is 75''',

APPENDIX A. APPENDIX

```
'''Question: The quadratic equation $x^2+mx+n=0$ has roots that
   are twice those of $x^2+px+m=0,$ and none of $m,$ $n,$ and $p$
    is zero. What is the value of $n/p?$
Answer: Let's think step by step
Let $r_1$ and $r_2$ be the roots of $x^2+px+m=0.$
Since the roots of $x^2+mx+n=0$ are $2r_1$ and $2r_2,$ we have
   the following relationships: \[
m=r_1 r_2,\quad n=4r_1 r_2,\quad p=-(r_1+r_2), \quad\text{and}\
   quad
m=-2(r_1+r_2).
\] So \[
n = 4m, \quad p = \frac{1}{2}m,
\quad\text{and}\quad
\frac{n}{p}=\frac{4m}{\frac{1}{2}m}=\boxed{8}.
\]
Alternatively, the roots of \[
\left(\frac{x}{2}\right)^2 + p\left(\frac{x}{2}\right) + m = 0
\] are twice those of $x^2 + px + m = 0.$
Since the first equation is equivalent to $x^2 + 2px + 4m = 0,$
we have \[m = 2p \quad\text{and}\quad n = 4m, \quad\text{so}\quad
    \frac{n}{p} = \boxed{8}.\]
```

The answer is 8''',

'''Question: Expand $(2z^2 + 5z - 6)(3z^3 - 2z + 1)$.

Answer: Let's think step by step

$$\begin{array}{crrrrrrr}

*& & & 3z^3 & & -2z & + 1 & *

*\times & & & & 2z^2 & +5z & -6 *

\cline{1-7}\rule{0pt}{0.17in}

*& & & -18z^3 & & +12z & -6 & *

*& & +15z^4 & & -10z^2 & +5z & & *

*+ & 6z^5 & & -4z^3 & +2z^2 & & & *

\cline{1-7}\rule{0pt}{0.17in}

& 6z^5 & +15z^4 & -22z^3 & - 8z^2 &+17z & -6 &

\end{array}$$

The answer is 6z^5+15z^4-22z^3-8z^2+17z-6''',

'''Question: Find the mean of all solutions for x when $x^3 + 3 x^2 - 10x = 0$.

Answer: Let's think step by step

First, we factor the equation as $x(x^2 +3x - 10) = 0$.

```
So, one solution is $x=0$ and the other two solutions are the
   solutions to $x^2 + 3x-10=0$.
We could either factor the quadratic, or note that the sum of the
    solutions to this quadratic is $-(3/1)=-3$,
so the mean of the three solutions to the original equation is $
   -3/3=\boxed{-1}$.
The answer is -1
''',
]
```

A.6.0.1 SL

```
sl_system_prompt = '''
You are a helpful assistant who is good at sloving math problems
   and writing code. You should should follow the guidelines
   below:
- Utilize the `pi` symbol and `Rational`` from Sympy for $\pi$
   and fractions, and simplify all fractions and square roots
   without converting them to decimal values
- You should only write code blocks and the function name should
```

```
   be `solution` and the returned value should be the final
   answer.
'''

sl_instruction_prompt = "Let's␣use␣python␣to␣solve␣the␣math␣
   problem.␣"


sl_math_shots= [
'''Question: Find the coefficient of $x^3$ when $3(x^2 - x^3+x)
   +3(x +2x^3- 3x^2 + 3x^5+x^3) -5(1+x-4x^3 - x^2)$ is simplifie.

```python
from sympy import symbols, simplify

def solution():
    """Find the coefficient of $x^3$ when $3(x^2 - x^3+x) +3(x +2
       x^3- 3x^2 + 3x^5+x^3) -5(1+x-4x^3 - x^2)$ is simplified
       ."""
    x = symbols('x')
    expr = 3*(x**2 - x**3 + x) + 3*(x + 2*x**3 - 3*x**2 + 3*x**5
       + x**3) - 5*(1 + x - 4*x**3 - x**2)
```
```

```
    simplified_expr = simplify(expr)

    x3_coefficient = simplified_expr.as_coefficients_dict()[x**3]
    result = x3_coefficient
    return result
```''',

'''Question: The surface area of a sphere with radius r is $4\
   pi r^2$. Including the area of its circular base, what is the
   total surface area of a hemisphere with radius 6 cm? Express
   your answer in terms of π.

```python
import math

def solution():
    """The surface area of a sphere with radius $r$ is $4\pi r^2$
       . Including the area of its circular base, what is the
       total surface area of a hemisphere with radius 6 cm?
       Express your answer in terms of $\pi$"""
    radius = 6
```

```
    # Surface area of the hemisphere
    hemisphere_area = 2 * math.pi * radius**2

    # Area of the circular base
    base_area = math.pi * radius**2

    # Total surface area
    total_surface_area = hemisphere_area + base_area

    # Formatting the result in LaTeX
    result = r'{}\\pi'.format(total_surface_area / math.pi)
    return result
```''',

'''Question: Monica tosses a fair 6-sided die. If the roll is a prime number, then she wins that amount of dollars (so that, for example, if she rolls 3, then she wins 3 dollars). If the roll is composite, she wins nothing. Otherwise, she loses 3 dollars. What is the expected value of her winnings on one die toss? Express your answer as a dollar value to the nearest cent.

```python
def solution():
    """Monica tosses a fair 6-sided die.  If the roll is a prime
       number, then she wins that amount of dollars (so that, for
        example, if she rolls 3, then she wins 3 dollars).  If
       the roll is composite, she wins nothing. Otherwise, she
       loses 3 dollars. What is the expected value of her
       winnings on one die toss? Express your answer as a dollar
       value to the nearest cent."""
    # Probabilities of each outcome
    prime_prob = 1 / 6
    composite_prob = 1 / 3
    otherwise_prob = 1 / 6

    # Expected value of each outcome
    prime_expected_value = (2 * prime_prob) + (3 * prime_prob) +
       (5 * prime_prob)
    composite_expected_value = 0 * composite_prob
    otherwise_expected_value = -3 * otherwise_prob

    # Total expected value
    total_expected_value = prime_expected_value +
```

```
        composite_expected_value + otherwise_expected_value


    # Dollar value to the nearest cent
    result = "{:.2f}".format(total_expected_value)
    return result
```''',


'''Question: Given $\mathbf{a} = \begin{pmatrix} -7 \\ 0 \\ 1 \
   end{pmatrix}$ and $\mathbf{b} = \begin{pmatrix} 4 \\ 2 \\ -1 \
   end{pmatrix},$ find $\mathbf{a} - 3 \mathbf{b}.$

Solution:
```python
import numpy as np


def solution()
    """Given $\mathbf{a} = \begin{pmatrix} -7 \\ 0 \\ 1 \end{
       pmatrix}$ and $\mathbf{b} = \begin{pmatrix} 4 \\ 2 \\ -1 \
       end{pmatrix},$ find $\mathbf{a} - 3 \mathbf{b}.$"""
    a = np.array([-7, 0, 1])
    b = np.array([4, 2, -1])
```

```
    result = a - 3 * b


    result = r'\begin{{pmatrix}} {} \\ {} \\ {} \end{{pmatrix}}'.
       format(result[0], result[1], result[2])
    return result
```''',

'''Question: The endpoints of a diameter of circle M are $
   (-1,-4)$ and $(-7,6)$. What are the coordinates of the center
   of circle M? Express your answer as an ordered pair.
```python
def solution():
    """The endpoints of a diameter of circle $M$ are $(-1,-4)$
       and $(-7,6)$. Find the coordinates of the center of circle
        $M$."""
    x1, y1 = -1, -4
    x2, y2 = -7, 6

    # Midpoint formula
    center_x = (x1 + x2) / 2
    center_y = (y1 + y2) / 2
```

```
    # Result as an ordered pair
    result = (center_x, center_y)
    return result
```''',

'''Question: Find the remainder when $2x^6-x^4+4x^2-7$ is divided
    by x^2+4x+3.
```python
from sympy import symbols, div

def solution():
    """Find the remainder when $2x^6-x^4+4x^2-7$ is divided by $x
       ^2+4x+3$."""
    x = symbols('x')
    numerator = 2*x**6 - x**4 + 4*x**2 - 7
    denominator = x**2 + 4*x + 3

    quotient, remainder = div(numerator, denominator)

    return remainder
''',
```

```
'''Question: Find $x$ so that the vectors $\begin{pmatrix} 2 \\ 5
    \end{pmatrix}$ and $\begin{pmatrix} x \\ -3 \end{pmatrix}$
   are orthogonal.
```python
from sympy import symbols, Eq, solve


def solution():
    """Find x so that the vectors (2, 5) and (x, -3) are
       orthogonal."""
    x = symbols('x')
    # Dot product of the vectors should be zero for orthogonality
    equation = Eq(2*x + 5*(-3), 0)
    result = solve(equation, x)
    return result[0]
```''',
'''Question: If x is positive and $x^2 = 729$, what is the
   value of x?
```python
from sympy import symbols, Eq, solve


def solution():
    """If $x$ is positive and $x^2 = 729$, what is the value of
```
```

```
        $x$?"""
    x = symbols('x')
    equation = Eq(x**2, 729)
    solutions = solve(equation, x)

    # Since x is positive, we select the positive solution
    result = [sol for sol in solutions if sol > 0][0]
    return result
```''',
]
```

### A.6.0.2 Second stage of SymbolNL

```
symbolnl_instructio_prompt = "Please␣think␣step␣by␣step␣about␣the
   ␣question␣based␣on␣the␣provided␣code(it␣may␣be␣wrong)␣and␣the␣
   executed␣output.␣"

symbolnl_math_shots = [
'''Question: How many distinct positive factors does 32 have?
Code: ```python
def solution():
    """How many distinct positive factors does 32 have?"""
```
```

```
    number = 32
    factors = set()

    for i in range(1, int(number**0.5) + 1):
        if number % i == 0:
            factors.add(i)
            factors.add(number // i)

    result = len(factors)
    return result
```
```

*Output: 6*

*Answer: To determine the number of distinct positive factors of 32, we can follow these steps:*

*1. Prime Factorization:*
   *- First, we find the prime factorization of 32.*
   *- \( 32 = 2^5 \)*

*2. Number of Factors:*
   *- The formula to find the number of distinct positive factors of a number based on its prime factorization is to add 1 to*
```

each of the exponents in the factorization and then multiply the results.

- For \(32 = 2^5 \), the exponent is 5.

- Adding 1 to the exponent: \(5 + 1 = 6 \).

3. Conclusion:

- Therefore, the number of distinct positive factors of 32 is 6.

Thus, the final result is:

\[\boxed{6}\]''',

'''Question: In parallelogram $ABCD$, angle B measures 110°. What is the number of degrees in the measure of angle C?

Code: ```python

```
def solution():
    """In parallelogram $ABCD$, angle $B$ measures $110^\circ$. What is the number of degrees in the measure of angle $C$?"""
    angle_B = 110
```

```
    # In a parallelogram, opposite angles are equal and adjacent
       angles are supplementary
    angle_C = 180 - angle_B

    return angle_C
```
```

Output: 70

Answer: In a parallelogram, opposite angles are equal and
   adjacent angles are supplementary. Given that angle \( B \)
   measures \( 110^\circ \), we need to find the measure of angle
    \( C \).

Step-by-step reasoning:

1. In a parallelogram, adjacent angles are supplementary. This
   means that the sum of the measures of angle \( B \) and angle
   \( C \) is \( 180^\circ \).
2. Therefore, we can write the equation: \( \text{angle } B + \
   text{angle } C = 180^\circ \).
3. Substituting the given measure of angle \( B \) into the
   equation: \( 110^\circ + \text{angle } C = 180^\circ \).
4. Solving for angle \( C \): \( \text{angle } C = 180^\circ -
   110^\circ = 70^\circ \).
```

```
Thus, the measure of angle \( C \) is \( \boxed{70} \).''',

'''Question: In triangle $ABC$, we have that $E$ and $F$ are
   midpoints of sides $\overline{AC}$ and $\overline{AB}$,
   respectively. The area of $\triangle ABC$ is 24 square units.
   How many square units are in the area of $\triangle CEF$?
Code: ```python
from sympy import Rational

def solution():
    """Calculate the area of triangle CEF given that E and F are
       midpoints of sides AC and AB respectively, and the area of
        triangle ABC is 24 square units."""
    # Area of triangle ABC
    area_ABC = 24

    # Triangle CEF is a median triangle, which is 1/4 the area of
        triangle ABC
    area_CEF = Rational(1, 4) * area_ABC

    return area_CEF
```

```
```

Output: 6

Answer: The problem involves finding the area of triangle CEF where E and F are midpoints of sides $\overline{AC}$ and $\overline{AB}$ respectively in triangle ABC. Given that the area of triangle ABC is 24 square units, we need to determine the area of triangle CEF.

Since E and F are midpoints of their respective sides, line segment $\overline{EF}$ is parallel to side $\overline{BC}$ and is half its length. This makes triangle CEF a median triangle of triangle ABC. A median triangle is one-fourth the area of the original triangle because it is formed by drawing lines from each vertex of the original triangle to the midpoints of the opposite sides.

Thus, the area of triangle CEF is calculated as:

\[\text{Area of } \triangle CEF = \frac{1}{4} \times \text{Area of } \triangle ABC = \frac{1}{4} \times 24\]

Simplifying this, we get:

\[\text{Area of } \triangle CEF = 6\]

Therefore, the area of triangle CEF is \(\boxed{6}\) square units.''',

'''Question: What is the greatest two-digit multiple of 13?

Code: ```python

```
def solution():
    """Find the greatest two-digit multiple of 13."""
    # Start from the greatest two-digit number and go downwards
    for number in range(99, 12, -1):
        if number % 13 == 0:
            return number
```

Output: 91

Answer: To find the greatest two-digit multiple of 13, we start by considering the largest two-digit number, which is 99. We need to find the largest number less than or equal to 99 that is divisible by 13.

We perform the division of 99 by 13 to find the largest integer quotient:

\[\frac{99}{13} \approx 7.615 \]

The integer part of this division is 7, indicating that the largest multiple of 13 within the two-digit range is \(13 \times 7 \).

Calculating this product:

\[13 \times 7 = 91 \]

Thus, the greatest two-digit multiple of 13 is 91. The Python code provided confirms this by iterating from 99 downwards and checking divisibility by 13, stopping at the first number that satisfies this condition, which is 91.

Therefore, the final answer is:

*\[\boxed{91} \]'''',

]

A.6.0.3 Second stage of NLSymbol

```
nlsymbol_instruction_prompt = "Write␣a␣Python␣code␣that␣
   translates␣a␣natural␣language␣(NL)␣reasoning␣path␣into␣
   executable␣code␣to␣answer␣a␣given␣question.␣The␣output␣of␣the␣
```

```
   generated␣code␣should␣be␣the␣final␣answer␣to␣the␣question.␣"

nlsymbol_math_shots = [
'''Question: The area of the floor in a square room is 225 square
    feet. The homeowners plan to cover the floor with rows of 6-
   inch by 6-inch tiles. How many tiles will be in each row?
Reasoning Path: Let's think step by step:

The area of the floor is given as 225 square feet. Since the room
    is square, each side of the room is the square root of the
   area. Thus, each side of the room is \(\sqrt{225} = 15\) feet.

Each tile is 6 inches by 6 inches, which is equivalent to \(0.5\)
    feet by \(0.5\) feet since there are 12 inches in a foot.

To find out how many tiles fit along one side of the room, we
   divide the length of the room by the length of one tile:
\[
\frac{15 \text{ feet}}{0.5 \text{ feet/tile}} = 30 \text{ tiles}
\]

Therefore, there will be \(\boxed{30}\) tiles in each row.
```

Code: ```python

```
from sympy import sqrt


def solution():
    # Area of the floor in square feet
    area = 225

    # Since the room is square, calculate the side length of the
       room
    side_length = sqrt(area)

    # Each tile's side length in feet (6 inches = 0.5 feet)
    tile_length = 0.5

    # Calculate the number of tiles in each row
    tiles_per_row = side_length / tile_length

    return tiles_per_row

```

```''',

*'''Question: When the set of natural numbers is listed in
   ascending order, what is the smallest prime number that occurs*

after a sequence of five consecutive positive integers all of which are nonprime?

Reasoning Path: Let's think step by step:

To find the smallest prime number that occurs after a sequence of five consecutive nonprime positive integers, we need to identify such a sequence first.

1. Start by checking small numbers. The first few nonprime numbers are 4, 6, 8, 9, and 10. However, these do not form a sequence of five consecutive nonprime numbers.

2. Continue checking further:
   - 8, 9, 10, 12, 14 are nonprime, but they are not consecutive.
   - 24, 25, 26, 27, 28 are nonprime and consecutive.

3. Check the number following 28:
   - 29 is the next number, and it is a prime number.

Thus, the smallest prime number that occurs after a sequence of five consecutive nonprime positive integers (24, 25, 26, 27, 28) is 29.

The answer is \(\boxed{29}\).

Code: ```python

from sympy import isprime

def solution():

*   # Start checking from a reasonable number*

*   n = 1*

*   while True:*

*   # Check if the next five numbers are all nonprime*

*   if all(not isprime(n + i) for i in range(5)):*

*   # Check the number right after the sequence of five nonprimes*

*   if isprime(n + 5):*

*   return n + 5*

*   n += 1*

```''',

'''Question: What is the slope of the line that is tangent to a circle at point (5,5) if the center of the circle is (3,2)? Express your answer as a common fraction.

Reasoning Path: Let's think step by step:

APPENDIX A. APPENDIX

The slope of the line connecting the center of the circle at point \((3,2)\) to the point on the circle \((5,5)\) is given by the formula for the slope between two points:

\[m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{5 - 2}{5 - 3} = \frac{3}{2}\]

The line that is tangent to the circle at the point \((5,5)\) will be perpendicular to the line connecting the center of the circle to this point. The slope of a line perpendicular to another line with slope \(m\) is the negative reciprocal of \(m\). Therefore, the slope of the tangent line is:

\[m_{\text{tangent}} = -\frac{1}{\frac{3}{2}} = -\frac{2}{3}\]

Thus, the slope of the line that is tangent to the circle at point \((5,5)\) is \(\boxed{-\frac{2}{3}}\).

Code: ```python

```
from sympy import Rational

def solution():
    # Slope of the line connecting the center of the circle to
       the point (5,5)
```

```
    slope_radius = Rational(5 - 2, 5 - 3)

    # Slope of the tangent line, which is the negative reciprocal
        of the slope of the radius
    slope_tangent = -1 / slope_radius

    return slope_tangent

```''',

'''Question: What is the greatest integer less than 100 for which
    the greatest common factor of that integer and 18 is 3?
Reasoning Path: Let's think step by step

We are looking for the greatest integer less than 100 for which
   the greatest common factor (GCF) with 18 is 3.

First, note that 18 can be factored into prime factors as \(18 =
   2 \times 3^2\).

For the GCF of a number \(n\) and 18 to be 3, \(n\) must:
1. Be divisible by 3 (to include the factor of 3).

2. Not be divisible by 2 or 9 (to avoid increasing the GCF beyond 3).

We need to find the largest integer less than 100 that meets these criteria. We look for numbers that are multiples of 3 but not multiples of 2 or 9.

The largest multiple of 3 under 100 is 99. We check if it is divisible by 2 or 9:

- 99 is not divisible by 2 (since it is odd).

- 99 is divisible by 9 (since \(9 + 9 = 18\), and 18 is divisible by 9).

Since 99 does not work (as it is divisible by 9), we check the next largest multiple of 3, which is 96.

- 96 is divisible by 2 (even number), so it does not work.

Next, we check 93:

- 93 is not divisible by 2 (odd number).

- 93 is not divisible by 9 (since \(9 + 3 = 12\), and 12 is not divisible by 9).

```
Thus, 93 meets the criteria of being divisible by 3 but not by 2
   or 9. Therefore, the greatest integer less than 100 for which
   the GCF with 18 is 3 is \(\boxed{93}\).
Code ```python
def solution():
    from math import gcd
    # Start from the largest number less than 100 and check
       downwards
    for n in range(99, 0, -1):
        if gcd(n, 18) == 3:
            return n

# The function will return the greatest integer less than 100 for
    which the GCD with 18 is 3
```''',
]
```

### A.6.1 Meta-Selection prompt

We show a 2-class meta-selection prompt for the MATH dataset below.

```
two_class_shots = [
    '''
```
```

APPENDIX A. APPENDIX

Question: Four standard, six-sided dice are to be rolled. If the product of their values turns out to be an even number , what is the probability their sum is odd? Express your answer as a common fraction.

Response: Below is an evaluation of the two methods for this specific four-dice probability problem:

Natural Language (Language Only):

Strengths: This method lets you break the problem down step by step. You can explain your reasoning about conditionals (the even product and odd sum) and manually handle the combinatorial cases. This approach can offer insight into why the answer is what it is.

Weaknesses: It requires careful bookkeeping to avoid mistakes in counting cases and may become tedious when managing several conditions simultaneously. There is more risk of a small oversight that leads to an error in the final probability.

Symbolic Language (Coding Only):

Strengths: By writing a short Python program to iterate over all 6^4 possible outcomes, you can systematically and accurately count the desired cases. It is very effective for this problem because the search space is small enough for exhaustive enumeration, thereby reducing human error.

Weaknesses: It might seem like overkill for a problem that is solvable by combinatorial reasoning, and you must be careful to correctly implement the conditions in code. However, given the manageable sample space, this method is straightforward and reliable.

For this specific question, because the number of outcomes is small and the conditions can be programmed explicitly, the program-aided approach minimizes error risk and offers a quick, systematic solution.

Selected method: SL (Symbolic Language, Coding Only)

'''',

'''

Question: A piece of cheese is located at $(12,10)$ in a coordinate plane. A mouse is at $(4,-2)$ and is running

up the line $y=-5x+18$. At the point (a,b) the mouse starts getting farther from the cheese rather than closer to it. What is $a + b$?

Response: Below is an evaluation of each approach for this particular problem:

Natural Language (NL):

Strengths:

- It is especially useful for problems that rely on geometric insight and visual reasoning. For instance, one can break down the problem by first finding the point on the line that is closest to the cheese (using ideas like the perpendicular from the cheese to the mouse's path) and then compute the coordinates, which directly leads to a conceptual understanding of why that point minimizes the distance.

- The natural language breakdown allows the solver to easily explain, verify, and understand each step in the geometric reasoning process.

Weaknesses:

- It relies on the solver's ability to carefully reason through each step without skipping important details.

Errors might occur if a step is overlooked or misinterpreted, but these issues are often easier to spot and correct when the reasoning is laid out in natural language.

Symbolic Language (SL):

Strengths:

- A Python program can be very effective for computing an answer if one sets up the right system (for example, defining the distance function and finding its minimum via analytic methods or optimization). It is useful if one has a routine set of tools (like computational geometry libraries or symbolic manipulators) that can automate the calculation accurately.

- It minimizes the potential for simple arithmetic mistakes, as the computer does the heavy lifting once the input is correctly specified.

Weaknesses:

- Setting up the correct algorithm in code (especially for a geometry problem that involves minimizing a distance) can be nontrivial. One must translate the geometric intuition correctly into code, and a mistake there might lead to an

incorrect answer without an immediately clear explanation.

- The program might properly produce a number but without conveying the underlying geometric reasoning, which can reduce the educational value of the solution process for problems like this.

For this specific problem-a geometry problem involving distances and perpendicular intersections-the Chain-of-Thought method is likely the more effective approach. It promotes understanding of the underlying geometric reasoning and highlights the process behind finding the point where the mouse's distance from the cheese is minimized.

Selected method: NL (Natural Language, Language Only)''',

'''

Question: Find the minimum of the function

\[\frac{xy}{x^2 + y^2}\]in the domain $\frac{2}{5} \le x \le \frac{1}{2}$ and $\frac{1}{3} \le y \le \frac{3}{8}.$

Response: Let's review what each approach offers for this problem.

APPENDIX A. APPENDIX

Natural Lauguage (NL) method:

Strengths: It allows step-by-step reasoning, considering the structure of the algebraic expression and boundary behavior, which may reveal insights such as where the minimum might occur (for example, at the endpoints or at a critical point) and can be done entirely by hand.

Weaknesses: The algebra can become quite messy given the rational function and the two-dimensional domain. Tracking all possible cases (including boundary and potential critical points) increases the risk of error or oversight. The method relies on human intuition and careful logical decomposition, which can be challenging under time constraints.

Symbolic Language (SL) method:

Strengths: A Python program can brute-force or use well-tested optimization routines to accurately sample the function over the given domain. Because the domain is compact and small in extent, a numerical search (or even more refined optimization) is straightforward and likely to yield a correct answer with less chance of human error.

Weaknesses: One must be careful to cover the entire domain correctly and account for potential issues with numerical precision. Additionally, writing the optimization code correctly also requires some programming skill, though for a two-variable bounded problem this is well within reach.

For this specific question-finding the minimum of a rational function over a small, compact domain-the program-aided approach is more directly effective. It reduces the intellectual overhead of dealing with all the subtle boundary and interior cases manually and tends to be more reliable given the tight constraints.

Selected method: SL (Symbolic Language, Coding Only)

'''

'''

Question: Someone observed that $6! = 8 \cdot 9 \cdot 10$. Find the largest positive integer n for which $n!$ can be expressed as the product of $n - 3$ consecutive positive integers.

Response: For this particular problem the trade-offs are as follows:

APPENDIX A. APPENDIX

Natural Language (NL) Strengths and Weaknesses:

- *Strength: A step-by-step reasoning process can provide insight into the structure of the equation (relating factorials to products of sequences) and might lead to an elegant, human-readable argument.*

  Weakness: The reasoning becomes quite intricate because one needs to relate rapidly growing factorials to a product of a long chain of consecutive integers. Handling large numbers and justifying bounds without resorting to heavy computations is tricky and error-prone for this specific task.

Symbolic Language (SL) Strengths and Weaknesses:

- *Strength: A computer program can iterate through candidate values of n (and compute factorials and products of consecutive integers) with precision. This method offloads the heavy number crunching and lets one quickly test many cases, which is especially valuable given the enormous growth of factorials.*
- *Weakness: A coding solution requires careful handling of very large numbers (or the use of logarithms/*

approximations) to avoid computational inefficiencies or overflow issues. Still, with modern programming languages and libraries this issue can be managed.

For this specific question, where the answer hinges on a precise numerical property and where direct computational verification seems feasible (once the search boundaries are known or estimated), the Symbolic Language (SL) approach is more effective. It allows for a systematic exploration of candidates with the precision that the problem demands without running into lengthy manual derivations.

Selected method: SL (Symbolic Language, Coding Only)''',

'''Question: Evaluate $(2-w)(2-w^2)\cdots(2-w^{10})$ where $w =e^{2\pi i/11}.$

Response: Below is an evaluation of both approaches for this problem:

Natural Language (nl, Language Only):

- *Strengths: This method allows one to recognize the structure of the product. In this particular problem the factors are (2 – w^k) for the nontrivial 11th roots of unity, and one can quickly connect this to the cyclotomic polynomial formula (x^11 – 1)/(x – 1). This insight leads directly to an exact, elegant solution without resorting to numerical approximation.*
- *Weaknesses: Using pure logical reasoning requires a strong familiarity with algebraic properties of roots of unity and cyclotomic polynomials. If one's background is less strong in these areas, there is a risk of overlooking key steps or making errors in derivation.*

Symbolic Language (SL, Coding Only):

- *Strengths: A Python program (or similar) can compute the product numerically, providing a quick verification of the result. It automatically handles the multiplication of many complex numbers, which is useful when manual calculation is cumbersome.*
- *Weaknesses: The numerical approach may introduce floating-point rounding errors, especially when working with complex exponentials. It also may hide the underlying*

structure of the problem and does not provide the exact algebraic insight that leads to a neat closed-form answer. Additionally, setting up the program correctly requires careful implementation of complex arithmetic.

For this specific question, since the underlying algebraic structure (via cyclotomic polynomials) offers an elegant, exact answer, the chain-of-thought method is more effective and insightful.

Selected method: NL (Natural Language, Language Only)

'''

'''

Question: In trapezoid $ABCD$ the lengths of the bases AB and CD are 8 and 17 respectively. The legs of the trapezoid are extended beyond A and B to meet at point E. What is the ratio of the area of triangle EAB to the area of trapezoid $ABCD$? Express your answer as a common fraction.

Response: For this trapezoid geometry problem, the NL method allows you to gradually set up the problem by assigning variables, drawing relationships (such as the similarity

of triangles), and then carefully deriving the necessary formulas. This process can enhance understanding and make it easier to check each individual step, especially when dealing with geometric properties and ratios.

On the other hand, the SL method would involve writing a Python program to symbolically manipulate or numerically approach the problem. While Python can be very powerful, setting up the symbolic algebra needed for a clear derivation of an algebraic ratio may require additional packages or careful coding. This approach might risk making the solution less transparent and could introduce potential programming errors for a problem that is more elegantly handled through traditional reasoning.

Given that this problem benefits from step-by-step geometric reasoning and clear algebraic manipulation, the chain-of-thought method is more effective and straightforward for this specific question.

Selected method: NL (Natural Language, Language Only)

'''',

```
    '''
    Question: A 4-inch by 6-inch picture is enlarged for framing
        by tripling its dimensions.  A 2-inch-wide border  is
        then placed around each side of the enlarged  picture, as
        shown.  Thin metal framing is sold only  in increments of
        one foot.  What is the minimum  number of linear feet of
        framing that must be  purchased to go around the perimeter
         of the border?

[asy]

draw((0,0)--(14,0)--(14,20)--(0,20)--cycle,linewidth(2));

draw((4,4)--(10,4)--(10,16)--(4,16)--cycle);

label("border",(7,17),N);

label("picture",(7,8),N);

label("frame",(14,5),E);

draw((17.5,7.5)--(14.5,7.5),Arrow);
```

draw((10.5,7.5)--(13.5,7.5),Arrow);

[/asy]

Response: Below is an evaluation of the two methods for this particular problem, followed by the selected method.

Natural Language (NL) – Language Only:

Strengths:

- It lets the solver reason through each step, clarifying the enlargement process, how the border is added to both dimensions, and the conversion from inches to feet.

- The explanation can detail the arithmetic (tripling dimensions, adding 2 inches on each side for the border, computing the new perimeter, and then converting inches to feet while accounting for the need to round up) so that every piece of the problem is understood.

- This method can easily communicate the reasoning to others.

Weaknesses:

- If not careful, details might be overlooked or arithmetic errors made.

- The reliance on manual computations (even if done carefully) might lead to mistakes if any step is not double-checked.

APPENDIX A. APPENDIX

Symbolic Language (SL) – Coding Only:

Strengths:

- *A short Python program can accurately perform the arithmetic once the proper logic is implemented.*
- *Code can handle unit conversion and rounding (for example, using math.ceil) without any miscalculation due to human error .*
- *It is especially useful when the operations are straightforward and mostly numerical, as in this case.*

Weaknesses:

- *The problem requires careful interpretation (tripling the dimensions, adding a border to both sides) which must be encoded correctly; any misinterpretation could lead to a coding error.*
- *Writing code for such a simple arithmetic problem might be overkill and could miss conveying the geometric understanding behind the steps.*
- *There can also be distraction in the form of programming syntax rather than focusing on the underlying mathematics.*

For THIS SPECIFIC QUESTION, the arithmetic and geometry are simple and direct. It is crucial to make sure that each conceptual step (enlargement, addition of borders, perimeter computation, and conversion unit) is clearly understood and communicated. The chain-of-thought method naturally lends itself to this clarity without the overhead of translating the problem into code. While coding could give the answer reliably, it may hide the reasoning steps that are important to verify each part of the computation.

Selected method: NL (Natural Language, Language Only)

'''

]

A.7 Scoring Rubric for CourtReasoner Meta-Evaluation

Legal Analysis Evaluation Rubric

You are tasked with evaluating legal analyses according to the following comprehensive rubric. Follow these criteria carefully to ensure consistent and fair assessment.

Conduct evaluations in all three of the following areas:

(If there are no citations, all the scores should be 0.)

A. Citation Relevance (Score: 0-4) Evaluate how applicable the cited precedent cases are to the legal question at hand. Please analyze each citation in the legal analysis in order to produce this score.

* **0 points**: No relevant cases cited, or all cited cases are completely irrelevant to the analysis. * **1 point**: All cited cases have only remote or tangential relevance to the core legal analysis. * **2 points**: Most cases cited have only distant relevance, with few directly applicable precedents. * **3 points**: About half of the cases cited are only remotely relevant to the analysis, while the rest are relevant. * **4 points**: All or nearly all cited cases are highly relevant and directly applicable to the legal analysis.

B. Constraints Extraction (Score: 0-4) In order to use a conclusion in the cited case, the analysis must first identify which constraints are needed to reach the conclusion in the case cited. This conclusion is useful for arguing the case this legal analysis is trying to argue. Evaluate how well the analysis identifies the necessary conditions (constraints) that must be satisfied in the case cited to reach the conclusion in the case cited that is useful for arguing the case this legal analysis is trying to argue.

* **0 points**: No legal constraints identified or the extraction is fundamentally incorrect. * **1 point**: Some constraints extracted but fewer than 3, or contains significant errors in interpretation. * **2 points**: At least 3 constraints extracted, but some are incorrectly formulated or incompletely articulated. * **3 points**: All necessary constraints (typically at least 3 plus any other applicable ones) are extracted, with only minor interpretive errors. * **4 points**: All constraints are fully and correctly extracted with precise legal terminology and interpretation.

C. Argument Validity per Constraint (Score: 0-4) Evaluate how well the legal arguments support each identified constraint, factual accuracy is important here. The legal analysis must not exaggerate or change key phrases in the background information or facts. This aspect should be evaluate independent of citation relevance and constraint extraction.

* **0 points**: No substantive arguments provided for any of the identified constraints. * **1 point**: Arguments provided for some constraints, but they are predominantly invalid, weak, or misapply legal principles. * **2 points**: Arguments provided for most constraints, but several are invalid or significant constraints lack supporting arguments. * **3 points**: Arguments provided for all identified constraints; most are valid but contain minor logical inconsistencies or gaps. * **4 points**: Strong, valid arguments provided for each identified constraint, with sound legal reasoning throughout.

Additional Context: Legal Question: legal question

“‘ legal analysis text “‘ Please analyze each citation in the legal analysis, then analyze whether constraints are satisfied for each cited case, then analyze whether each argument is valid for each constraint.

Besides explanations for your scoring, also produce a formatted score following ’ the example below in a json format. Make sure to add “‘json before the json dict. “‘json "Citation Relevance": "<number>", "Constraints Extraction": "<number>", "Argument Validity per Constraint", "<number>", “‘

Table S1: Annotation guidelines

Write a proof for each conclusion in a story. Write the relevant derivation at each inference step and the rule of inference. Please think about the forms of the input premises and conclusions carefully. If the forms do not correspond to a rule in the instructions, then you should not use it without verifying it. You can invent new rules yourselves. But please verify the correctness of the rules with the truth table calculator here. If the rule is not successfully verified, please do not use it. If it is successfully verified, you can add it to the updated list and use it.

Do's:

*** Write the inference rule used for each inference step. For inference rules, please refer to this page. If you think the inference rules needed are not included in this list, please add them to the updated list below.

... *Established inference rule definitions with examples*...

Intuitively and Logically Correct Rules: There are some rules that might be needed in the proof procedure but that are hard to be derived. Theses rules do not have a commonly agreed name, we are listing these rules as below:

*** $\neg(a \oplus b)$ is equivalent to $(\neg a \wedge \neg b)$ or $\neg(a \wedge b)$.

*** ANDEquivalent1: $a \rightarrow \neg b$ is equivalent to $\neg(a \wedge b)$

*** XOR1: $\neg(a \oplus \neg b)$ is equivalent to $(a \oplus b)$

*** XOR2: $(a \oplus b)$ is equivalent to $(\neg a \oplus \neg b)$

*** XOR3: $(\neg b \oplus b)$ is always true.

*** XOR4: From $(a \oplus b)$, we know that $a \rightarrow \neg b$.

*** XOR5: From $\neg(a \oplus b)$, we know that $a \rightarrow b$.

*** XOR6: From $a \rightarrow b$ and $b \rightarrow a$, we know that $\neg(a \oplus b)$.

*** IMPAND1: From $a \rightarrow \neg b$, we know that $\neg(a \wedge b)$.

*** XORUni1: From $\forall x(C(x) \rightarrow \neg A(x))$, $\forall x(A(x) \oplus B(x))$, we know that $\forall x(C(x) \rightarrow B(x))$.

*** XORUni2: From $\forall x(A(x) \rightarrow B(x))$, $\forall x(A(x) \oplus B(x))$, we know that $\forall x(B(x))$.

*** XORUni3: From $\forall x(A(x) \oplus B(x))$ and $\forall x(A(x) \rightarrow C(x))$, we know that $\forall x(C(x) \vee B(x))$.

*** OREquivalent1: $(a \vee b) \wedge (a \vee \neg b)$ is equivalent to a

*** ANDUni1: From $\forall x(A(x) \rightarrow C(x))$ and $A(paul) \wedge B(paul)$", we know that "$C(paul) \wedge B(paul)$".

*** ORUni1: From $\forall x(A(x) \vee B(x))$ and $\forall x(A(x) \rightarrow C(x))$, we know that $\forall x(C(x) \oplus B(x))$.

*** ORUni2: From $\forall x(A(x) \rightarrow C(x))$, A(paul) $\vee$ B(paul), we know that C(paul) $\vee$ B(paul).

*** ORUni3: From $\forall x(\neg C(x) \rightarrow A(x))$, A(paul) $\vee$ B(paul), we know that $\neg C$(paul) $\vee$ B(paul).

*** ORUni4: From $\forall x(A(x) \rightarrow C(x))$ and $\forall x(B(x) \rightarrow C(x))$, we know that $\forall x((A(x) \vee B(x)) \rightarrow C(x))$.

*** Note: From $\neg a \rightarrow b$, we cannot know $\neg(a \wedge b)$.

*** Note: From $\forall x(A(x) \oplus B(x))$ and $\forall x(A(x) \rightarrow C(x))$, we cannot infer $\forall x(C(x) \oplus B(x))$.

*** Also note: From $\forall x(A(x) \rightarrow D(x))$ and A(paul) $\vee$ B(paul) $\rightarrow$ C(paul), we cannot infer D(paul) $\vee$ B(paul) $\rightarrow$ C(paul).

*** ORUni5: From $\forall x(C(x) \rightarrow \neg A(x))$, $\forall x(A(x) \vee B(x))$, we know that $\forall x(C(x) \rightarrow B(x))$.

*** Iff introduction: If A, then B. If B, then A. Then, either both A and B or neither A nor B.

*** For complicated conclusions, sometimes proving part of them is sufficient. A or B. A is true -> the conclusion is true. A -> B. A is false -> the conclusion is false. For conclusions with truth values of False, prove the opposite is True.

*** For the sake of annotation purposes, either..or in any context is exclusive, or is inclusive. For example, if I can either sleep at home today or go to class, this means that I can choose to sleep at home, go to class, but not both. Whereas if I can I can drink soda or drink coffee, it means I can drink soda, coffee, or both.

*** When we say that someone is not an energetic person or a sloth, this means that someone is neither an energetic person nor a sloth.

*** $\neg$(Indian(jesse) $\oplus$ Human(jesse)) : Jessie is either Indian and human, or that he is neither Indian nor human. In several instances, this sentence is rendered as "Jessee is not either an Indian or a human."

Proof consistency check:

*** Please involve at most two premises in one inference step.

*** Be sure to write the conjunction introduction steps.

*** Existential generalization check: Broadway Sheetmetals was a business owned by Edwin Smith, who was a rower. -> There was a business owned by a rower. There are two steps of existential generalization involved. Be sure to write both steps.

*** Do not use derivations used for a previous conclusion in a new conclusion. If conclusion A uses certain inference steps of conclusion B you have written the proof for, be sure to copy those inference steps in the proof of conclusion A.

*** Check for missing modus tollens steps.

*** Note the difference between modus tollens and transposition.

*** Note the difference between hypothetical syllogism, modus ponens and universal instantiation.

*** Hypothetical syllogism should be easy to distinguish from the other two since hypothetical syllogism would involve two implications (->). Both the premises need to have implications in order to derive the conclusion.

*** Universal instantiation and modus pones both involve only one implication

*** Universal instantiation involves the universal quantifier, you can check the FOL part if you are not sure if a statement involves the universal quantifier.

*** Modus ponens does not involve the universal quantifier.

*** Note: A(a) xor B(a) imply A(a) or B(a); A(a) or B(a) cannot imply A(a) xor B(a).

*** Hypothetical syllogism instead of universal instantiation: If you have room for dessert, you have room for broccoli. Everyone at Luis's dinner party has room for dessert, including Luis. -> Everyone at Luis's dinner party has room for broccoli.

Category	Algebra	Counting & Probability	Geometry	Number & Theory	Intermediate & Algebra	Precalculus	Prealgebra	Total
Number	1187	474	479	540	903	546	871	5000

Table S2: Number of samples in the testing set by categories for MATH.

	MATH		FOLIO	
Method	**Freq.**	**Prop.**	**Freq.**	**Prop.**
NL	4705	94.10%	133	58.84%
SL	124	2.48%	93	41.16%
SymbolNL	163	3.26%	-	-
NLSymbol	8	0.16%	-	-

Table S3: The frequency and proportion of selected choices by HybridMind on MATH and FOLIO *test* set. We implemented 4-choice selector for MATH dataset and 2-choice selector for FOLIO.

Category	`WikiLogic`	`HybLogic`	Total
Number	111	115	226

Table S4: Number of samples in the testing set by categories for FOLIO.

Method	Significance Level
GPT-3.5-Turbo compare prompt (less)	< 0.05
GPT-4o compare prompt (less)	< 0.05
o3-mini prompt (less)	0.257
FOLIO (more)	< 0.05

Table S5: Testing of significance levels of MATH dataset between FOLIO and NL.

Method	Significance Level
GPT-4o compare prompt (less)	0.080
o3-mini prompt (less)	< 0.05
FOLIO (more)	< 0.05

Table S6: Testing of significance levels of FOLIO dataset between FOLIO and NL.

Question

For how many integers a satisfying $1 \leq a \leq 23$ is it true that $a^{-1} \equiv a \pmod{24}$?

NL

Let's think step by step. We need to find integers a such that $1 \leq a \leq 23$ and $a^{-1} \equiv a \pmod{24}$. This condition implies that $a \cdot a \equiv 1 \pmod{24}$, or equivalently, $a^2 \equiv 1 \pmod{24}$. This congruence can be rewritten as $a^2 - 1 \equiv 0 \pmod{24}$, which factors as $(a-1)(a+1) \equiv 0 \pmod{24}$. Therefore, $(a-1)(a+1)$ must be divisible by 24. The prime factorization of 24 is $2^3 \times 3$. Thus, $(a-1)(a+1)$ must be divisible by $2^3 \times 3$. Since $a-1$ and $a+1$ are consecutive integers, one of them is even, and the other is odd. Therefore, one of them must be divisible by 4, and the other by 2, to ensure divisibility by $2^3 = 8$. Additionally, one of $a-1$ or $a+1$ must be divisible by 3.
We solve these systems of congruences using the Chinese Remainder Theorem:
Case 1: ..... $a \equiv 17 \pmod{24}$
Case 2: ..... $a \equiv 7 \pmod{24}$
Thus, there are two integers a that satisfy the condition: $a = 7$ and $a = 17$. Therefore, the number of integers a is 2.

SL

Python

```
from sympy import symbols, Eq, solve, mod_inverse
def solution():
  count = 0
  for a in range(1, 24):
    try:
     # Calculate the modular inverse of a modulo 24
     a_inv = mod_inverse(a, 24)
     # Check if the modular inverse is equal to a
     if a_inv == a:
       count += 1
    except ValueError:
     # If a modular inverse does not exist, continue
     continue
  return count
solution()
```

Executed Result
8

Table S7: An example where SL outperforms NL. While NL reasoning leads to logical errors in analysis and overlooks some possible scenarios, resulting in incorrect conclusions, writing Python code is more elegant and intuitive.

Question

How many days in a year have a 3 in the tens digit of the day of the month?

NL

Let's think step by step. We need to count the number of days in a year where the tens digit of the day of the month is 3. This means we are looking for days numbered 30 through 39 in each month.
1. Identify the months with days 30-39:
- The months with at least 30 days are January, March, April, May, June, July, August, September, October, November, and December.
- February has only 28 or 29 days, so it does not have any days with a tens digit of 3.
2. Count the days for each applicable month:
-For each of the months with at least 30 days, the days 30 and 31 have a tens digit of 3.
- Therefore, each of these months contributes 2 days (30 and 31) to our count.
3. Calculate the total number of such days:
- There are 11 months with at least 30 days.
- Each of these months contributes 2 days.
- Total number of days = $11 \times 2 = 22$. Thus, the number of days in a year that have a 3 in the tens digit of the day of the month is 22.

SL

Python

```
def solution():
  # Not all months have 31 days, so we need to count carefully
  # Months with 31 days:  January, March, May, July, August, October, December
  months_with_31_days = 7
  # Months with 30 days:  April, June, September, November
  months_with_30_days = 4
  # February has 28 or 29 days, so it never has a 30th or 31st
  # Days with a 3 in the tens digit:
  # - 30th:  occurs in every month except February
  # - 31st:  occurs only in months with 31 days
              days_with_3_in_tens_digit = (months_with_30_days + months_with_31_days) +
months_with_31_days
  return days_with_3_in_tens_digit
solution()
```

Executed Result

18

Table S8: An example where SL outperforms NL. Python code offers a more accurate and structured approach. While NL reasoning overlooks some edge cases and leads to incorrect conclusions, SL systematically handles the details and produces the correct result.

Question

Find the product of all positive integer values of c such that $3x^2 + 7x + c = 0$ has two real roots.

NL

To determine the product of all positive integer values of c such that the quadratic equation $3x^2 + 7x + c = 0$ has two real roots, we need to ensure that the discriminant of the quadratic is positive. The discriminant Δ of a quadratic equation $ax^2 + bx + c = 0$ is given by:

$$\Delta = b^2 - 4ac$$

.....
Solving this inequality for c, we get:

$$49 > 12c \quad \Rightarrow \quad c < \frac{49}{12}$$

Calculating $\frac{49}{12}$, we find:

$$\frac{49}{12} \approx 4.0833$$

Since c must be a positive integer, the possible values for c are $1, 2, 3$, and 4. Now, we find the product of these values:

$$1 \times 2 \times 3 \times 4 = 24$$

Thus, the product of all positive integer values of c such that the quadratic equation has two real roots is 24.

SL

Python

```
from sympy import symbols, solve, Rational
def solution():
  x, c = symbols('x c')
  # For the quadratic equation to have two real roots, the discriminant must be
non-negative
  # Discriminant of 3x^2 + 7x + c = 0 is 7^2 - 4*3*c
  discriminant = 7**2 - 4*3*c
  # Solve for c when discriminant is greater than 0
  c_values = solve(discriminant > 0, c)
  # c must be a positive integer
  positive_integer_c_values = [i for i in range(1, int(c_values[0]) + 1)]
  # Calculate the product of all positive integer values of c
  product = 1
  for value in positive_integer_c_values:
    product *= value
  return product solution()
```

Executed Result

TypeError: 'And' object is not subscriptable

Table S9: An example where natural language (NL) outperforms symbolic logic (SL) arises when solving problems through Python code. A key factor in such cases is that the language model must not only understand the problem but also possess a strong command of the syntax and semantics of the symbolic language. Without this, the model may generate code that is syntactically invalid or semantically incorrect, even if its underlying reasoning about the problem is sound.

Question

How many vertical asymptotes does the graph of $y = \frac{x-3}{x^2+7x-30}$ have?

NL

Let's think step by step. To find the vertical asymptotes of the function $y = \frac{x-3}{x^2+7x-30}$, we need to determine where the denominator is equal to zero, as these are the points where the function is undefined and may have vertical asymptotes. First, we set the denominator equal to zero:

$$x^2 + 7x - 30 = 0.$$

Next, we factor the quadratic equation:

$$x^2 + 7x - 30 = (x + 10)(x - 3) = 0.$$

Setting each factor equal to zero gives us the solutions:

$$x + 10 = 0 \quad \Rightarrow \quad x = -10,$$

$$x - 3 = 0 \quad \Rightarrow \quad x = 3.$$

These are the potential vertical asymptotes. However, we must check if these points are also zeros of the numerator, which would indicate a hole instead of an asymptote. The numerator is $x - 3$. Setting it equal to zero gives:

$$x - 3 = 0 \quad \Rightarrow \quad x = 3.$$

Since $x = 3$ is a zero of both the numerator and the denominator, it is a removable discontinuity (a hole) rather than a vertical asymptote.
Therefore, the only vertical asymptote occurs at $x = -10$.
Thus, the graph of $y = \frac{x-3}{x^2+7x-30}$ has 1 vertical asymptote.

SL

Python

```
from sympy import symbols, solve
def solution():
  x = symbols('x')
  denominator = x**2 + 7*x - 30

  # Find the roots of the denominator
  roots = solve(denominator, x)

  # The number of vertical asymptotes is the number of distinct real roots
  num_vertical_asymptotes = len(roots)

  return num_vertical_asymptotes
solution()
```

Executed Result

2

Table S10: An example where NL outperforms SL. SL doesn't consider the case where the numerator equals zero, which highlights the weakness of SL reasoning when facing certain edge cases or special conditions.

A.8 Per-citation Relevance Rubric used in CourtReasoner

0: Not relevant: The citation does not support the claim at all (possibly erroneous or misleading).
1: Slightly relevant: The citation touches on a loosely related topic but doesn't provide strong evidence for the exact claim. 2: Relevant: The citation supports the claim in a general way, though not precisely or directly.
3: Very relevant: The citation directly supports the claim, but may not be the strongest or most specific possible source.
4: Highly relevant: The citation is exactly on point—direct, precise, and authoritative evidence for the claim.

A.9 Example of model input and output for the CourtReasoner benchmark

A.9.0.0.1 GPT4o Deep Research input

Use the information provided below to answer this question: Does Neb. Rev. Stat. § 25-1329 apply to a district court's judgment resolving a petition in error, thereby tolling the time for appeal, despite prior case law holding that the statute does not apply to district courts acting as intermediate appellate courts?

Do not cite anything from McEwen v. Neb. State. Do not cite any judgments rendered after Jul. 12, 2019. When conducting the research and analysis, use the facts provided below and not any facts that you might know from the underlying case. For example, if any facts from the underlying case are missing from the information provided below, do not assume those facts. The analysis should be based only on the facts provided below.

Cite precedent cases with in-text citations. Expand out the full case citations in the text instead of using popup citations. Do not use Wikipedia.

* * * * *

Dr. Robert McEwen filed a petition in error in the district court for Dawes County, Nebraska, against the Nebraska State *125College System (NSCS), a system of three state colleges in Nebraska. He alleged that he was wrongfully terminated from his position as a tenured professor at Chadron State College.

Neither party disputes that the petition in error was timely filed. Responding to the petition in error, NSCS' answer admitted that McEwen was discharged on March 16, 2016, that McEwen timely requested an additional hearing before NSCS' board of trustees under a provision of the collective bargaining agreement, and that on April 18, NSCS' chancellor **556denied the additional hearing, thereby finalizing the discharge. McEwen's petition was filed on May 17. District court proceedings followed.

By a judgment styled as a memorandum order, the district court "overruled" his petition on March 31, 2017 (March judgment).

Exactly 10 days later, on April 10, 2017, McEwen moved for a new trial or, in the alternative, for an order vacating the March judgment. The alternative motion stated that it was based on Neb. Rev. Stat. § 25-2001 (Reissue 2016).

After a hearing, the district court overruled both aspects of the motion, doing so by an order entered on May 25, 2017 (May order). Because the court had not conducted a trial and reviewed only a transcript of the administrative proceedings, it concluded that a motion for new trial was not proper. Turning to McEwen's alternative motion to vacate judgment, the court explained that it had made a mistake of fact regarding the presence of an individual at an administrative hearing. But the court concluded that the individual's presence was not the "determining fact" in the court's conclusions regarding the " '17.3' " issue, referring to a section of a collective bargaining agreement. Thus, the court did not change its decision regarding the merits of McEwen's petition in error.

Within 30 days after the May order, McEwen filed a notice of appeal. In case No. A-17-638, the Court of Appeals summarily dismissed the appeal for lack of jurisdiction. The court's summary order explained that McEwen's motion for new trial did not "toll" the time to file a notice of appeal and that McEwen's notice of appeal was not timely filed.

—three paragraphs abbreviated—

After oral argument in this court, we requested supplemental briefing by the parties addressing whether, in light of the distinction traditionally recognized between petitions in error and appeals created by various statutes, the Legislature intended for motions to alter or amend a judgment under § 25-1329 to apply to judgments entered in error proceedings and, if so, the proper application of that statute to the case before us. The parties promptly submitted supplemental briefs, which we have considered.

**558III. ASSIGNMENTS OF ERROR

McEwen assigns that the Court of Appeals erred in (1) dismissing his appeal for lack of jurisdiction and (2) overruling his subsequent motion for rehearing.

IV. STANDARD OF REVIEW

A jurisdictional question which does not involve a factual dispute is determined by an appellate court as a matter of law, which requires the appellate court to reach a conclusion independent of the lower court's decision.8

The meaning of a statute is a question of law.9

A.9.0.0.2 GPT4o Deep Research Output (Excerpt)

Understood. I will research whether Neb. Rev. Stat. § 25-1329 applies to a district court's judgment resolving a petition in error, despite prior case law suggesting that it does not apply to district courts acting in an appellate capacity. I will craft the remaining portion of an appellate court judgment with legal analysis, discussion, and a reasoned conclusion, using only precedent and legal interpretations available up to July 12, 2019, and excluding McEwen v. Neb. State.

ANALYSIS

District Court as Appellate Tribunal and Tolling Motions

Nebraska law has long held that when a district court exercises appellate jurisdiction – reviewing a decision of a lower court or tribunal – certain postjudgment motions are not available to extend the time for appeal. In particular, a motion for new trial is confined to trial courts; if the district court acted as an appellate court, a "motion for new trial" is not a proper pleading and does not arrest the running of the appeal clock. This rule has been applied uniformly whether the district court was hearing an appeal from a county court or from "some other lower tribunal". Accordingly, our cases repeatedly refused to recognize such motions as tolling the time for appeal in that context. We reasoned that in an appellate review, the district court's decision is not a true "judgment" as contemplated by the tolling statutes, but rather an appellate disposition of another body's judgment. We thus defined "judgment" for purposes of a motion to alter or amend (§ 25-1329) as *"the final determination of the rights of the parties in an action"* – i.e. the decision of a trial court – and not an appellate decision of a district court. Under that view, a district court sitting as an intermediate appellate court could not entertain a tolling motion (whether styled as new trial or to alter/amend), and any such motion would not extend the 30-day deadline for filing a notice of appeal. This doctrine was well established in our jurisprudence.

Petition in Error Proceedings as Distinct from Appeals

Notwithstanding the above principles, we recognize a clear distinction in Nebraska law between a direct *appeal* to the district court and a *proceeding in error*. A proceeding in error in the district court (pursuant to Neb. Rev. Stat. §§ 25-1901 to 25-1908) is *"distinct and independent"*, whereas an appeal is *"a mere continuation of the same cause in another court."* Unlike a direct appeal which removes the entire cause to the higher court for retrial on law and fact, an error proceeding is initiated by filing a petition in the district court (along with a transcript of the inferior tribunal's record) and issuing summons to the adverse party. In other words, the petition in error functions as a new civil action filed in the district court, the purpose of which is to obtain review of the inferior body's decision on the record. We have said that the two procedures – appeal and error – are so different that the statutory provisions governing each "cannot be taken together and construed as if they were one law". Consistent with that view, the outcome of a petition in error is a judgment of the district court concluding the independent error proceeding, whereas on a direct appeal the district court's ruling might simply remand the cause for further proceedings below. This historical distinction between an error proceeding and a regular appeal is crucial in assessing the applicability of § 25-1329.

Earlier Nebraska decisions did not always articulate this distinction when considering postjudgment motions. The broad rule stated in cases like **Goodman v. City of Omaha** (a zoning appeal) and **Timmerman v. Neth** (an administrative license appeal) was that no tolling motion lies when the district court acts as an appellate reviewer. Those cases, however, involved ordinary appeals where the district court was unquestionably sitting as an intermediate appellate court. We acknowledge that our jurisprudence at times used loose language referring to an "appeal by petition in error" and thus lumped error proceedings together with other appeals. Upon closer examination, however, the district court's role in a petition in error is more akin to that of a trial court entering a judgment in an original action (albeit on a limited record review) than that of a purely intermediate appellate tribunal. The question, then, is whether the tolling provisions of § 25-1329 should apply to the judgment rendered by a district court at the conclusion of a petition in error proceeding. We conclude that they should.

Legislative Intent of § 25-1329

Section 25-1329 was enacted as part of L.B. 921 in 2000, against the backdrop of the above-described case law. By its plain terms, the statute imposes only two requirements for a motion to alter or amend: it must be filed within ten days after the entry of *"the judgment,"* and it must seek a substantive change to that judgment. Nothing in the text of § 25-1329 limits the term "judgment" to a particular court or type of proceeding. In contrast to other contexts where the Legislature has explicitly differentiated appeals from other actions, here the statute is written generally, suggesting it encompasses any final judgment properly so called.

Application to the Present Case

In light of the foregoing, we hold that Neb. Rev. Stat. § 25-1329 applies to a district court's judgment resolving a petition in error. When a party timely files a motion to alter or amend such a judgment (seeking a substantive change in the outcome), the motion terminates the running of the time for appeal,...